銀河力学で学ぶ数値計算法

配布版 v1.2

馬場淳一（鹿児島大学・国立天文台）

2026 年 10 月 16 日

目次

はじめに **iv**

1 球対称質量モデルと数値積分 **1**
1.1 球対称密度分布と内包質量 1
1.2 数値積分の考え方 3
1.3 基本的な数値積分公式 3
1.4 Plummer 球による 4 つの積分法の比較 7
1.5 収束次数の測定 8
1.6 離散化誤差と丸め誤差 12
1.7 累積積分による内包質量プロファイル 13
1.8 内包質量と円運動速度 15

2 Poisson 方程式と有限差分法 **19**
2.1 Plummer 球の重力を求める二つの方法 19
2.2 球対称 Poisson 方程式の 1 次元化 20
2.3 Poisson 方程式の中心差分による離散化 21
2.4 中心と外側の境界条件 22
2.5 連立一次方程式の組み立て 25
2.6 Plummer 球の Poisson 方程式の数値解 26
2.7 Poisson 方程式の格子収束 29
2.8 数値微分による重力加速度 31
2.9 前進・後退・中心差分の収束比較 33

3 球対称ポテンシャルと時間積分法 **37**
3.1 Plummer 球の重力場と軌道計算の準備 37
3.2 初期値問題としての運動方程式 39
3.3 Euler 法 39
3.4 古典的 4 次 Runge–Kutta 法（RK4） 41
3.5 リープフロッグ法（蛙跳び法） 42
3.6 円軌道による時間積分法の検証 44
3.7 時間積分法の収束次数 48
3.8 保存量による数値軌道の診断 49
3.9 高次精度と長時間保存性の違い 52

A 軸対称円盤の質量積分と回転曲線 **57**
A.1 球対称系と軸対称円盤の質量要素 57
A.2 指数円盤の内包質量 57
A.3 累積台形則による数値積分 59
A.4 累積台形則の格子収束 60
A.5 指数円盤の回転曲線と球対称近似 62
A.6 Mestel 円盤と平坦な回転曲線 63

B 軸対称 Poisson 方程式の数値解法 **65**
B.1 軸対称 Poisson 方程式と検証用モデル 65
B.2 2 次元有限差分 66
B.3 2 次元格子での連立一次方程式の組み立て 68
B.4 疎行列の大きさと構造 70
B.5 数値解と誤差 72
B.6 2 次元 Poisson 方程式の格子収束 74

C 軸対称・非軸対称ポテンシャル中の軌道 **76**
C.1 異なる重力場への時間積分法の適用 76
C.2 軸対称ポテンシャルとエピサイクル運動 76
C.3 Miyamoto–Nagai ポテンシャル中の軌道 82
C.4 非軸対称ポテンシャル 84

D 連立一次方程式の解法 **89**
D.1 Gauss の消去法：小さな例 89
D.2 LU 分解：消去手順の行列表現 90
D.3 疎行列の消去と代入 91
D.4 反復法：Jacobi 法 94

参考図書 **97**

はじめに

本書は、宇宙流体シミュレーションや重力多体系シミュレーションに進む学部生が、数値計算法の基礎と検証の考え方を学ぶための準備教材である。B. Ryden “*Celestial and Stellar Dynamics*” (2025) の輪講と並行して用い、そこで学ぶ質量モデル・重力場・恒星軌道を数値計算の題材とする。Ryden の物理内容の説明を置き換えるのではなく、その式から実際に数値を求め、誤差と収束を検証する方法を学ぶ。

宇宙流体・重力多体系シミュレーションでは、物理量の離散化、重力場の計算、時間積分などを組み合わせる。本書では、それらを最初から同時に扱わず、単純な問題で一つずつ学ぶ。本文では基本的な計算法と検証方法を扱い、脚注や補足では、今学んでいる計算が将来のシミュレーションのどの処理に相当するかを示す。

本書のねらい

数値計算では、コードがエラーなく最後まで実行され、もっともらしい図が得られても、その答えが正しいとは限らない。格子幅や時間刻みによる離散化誤差、丸め誤差、計算領域や境界条件の設定による誤差が、気づかないうちに結果に入り込むからである。本書で最も身につけてほしいのは、個々の公式よりも、得られた数値解が信頼できるかを自分で確かめる習慣である。そのために、本書では次の 3 つの確認を繰り返し行う。

- **解析解との比較**：解析解が分かっている問題で、コードと数値計算法が正しく働くことを確かめる。
- **保存量による診断**：エネルギーや角運動量など、物理的に保存されるべき量がどの程度保たれているかを調べる。
- **分解能を変えた収束の確認**：格子幅や時間刻みを変えて計算し、答えが収束するか、誤差が期待どおりの次数で減るかを調べる。

実際の研究で解く問題の多くには解析解がない。だからこそ、まず解析解のある問題で手法を検証し、そのうえで保存量と収束を組み合わせて結果を確かめることが重要になる。また、誤差が見つかったときには、それが離散化、丸め誤差、有限の計算領域や境界条件、あるいは近似したモデルそのもののどれに由来するのかを区別して考える。

本書の構成

本編は第 1–3 章で完結する。第 1・2 章では Ryden §6.4 の球対称系を題材に数値積分と有限差分法を学び、第 3 章では §7.1 の球対称ポテンシャル中の軌道を題材に時間積分法を学ぶ。本編は球対称系のみを扱うが、付録 A・B・C は、それぞれ第 1・2・3 章で学んだ方法を軸対称系や非軸対称系へ広げる発展内容であり、関心に応じて選んでよい。また、連立一次方程式の解法を詳しく知りたい場合は、数値線形代数の補足である付録 D を参照してほしい。各 Notebook は単独で先頭から実行でき、本文との対応表と実行順序を冒頭に示している。

本書	Ryden テキストとの対応	物理的な題材	本書で学ぶ数値計算法
第 1 章 Notebook 1	§6.4 特に §6.4.2–6.4.3	Plummer・Hernquist・NFW などの球対称密度分布	数値積分、誤差、収束次数
第 2 章 Notebook 2	§6.4	球対称密度分布からポテンシャル・重力場を求める問題	1 次元 Poisson 境界値問題、有限差分、数値微分
第 3 章 Notebook 3	§7.1	球対称ポテンシャル中の恒星軌道	常微分方程式の初期値問題、Euler 法、古典的 4 次 Runge–Kutta 法（RK4）、リープフロッグ法
付録 A Notebook A	§6.5	薄い軸対称円盤、指数円盤、Mestel 円盤、回転曲線	第 1 章の数値積分の応用、格子収束
付録 B Notebook B	§6.5	軸対称密度・ポテンシャル、Miyamoto–Nagai モデル	2 次元有限差分、対称軸の処理、疎行列、格子収束
付録 C Notebook C	§7.2–7.3	軸対称・非軸対称ポテンシャル、エピサイクル運動、box・loop 軌道	第 3 章の時間積分法の応用、保存量による検証
付録 D	直接の対応なし	第 2 章・付録 B で現れる連立一次方程式の背景	Gauss の消去法、LU 分解、疎行列、Jacobi 法

(注) 表の「Ryden テキストとの対応」は、主として物理的な題材の対応を示しており、Ryden テキストが有限差分法や Poisson 方程式の数値ソルバーを扱っているという意味ではない。

本書で扱う範囲

本書で学ぶ数値計算法は、次の 4 つである。

- **数値積分**（第 1 章、付録 A）
- **数値微分**（第 2 章）
- **有限差分法による境界値問題の解法**（第 2 章、付録 B）
- **初期値問題に対する時間積分法**（第 3 章、付録 C）

一方、補間法そのもの（スプライン補間など）、非線形方程式の解法（Newton 法など）、固有値問題は扱わない。連立一次方程式はライブラリのソルバーで解き、その基本的な考え方を付録 D で紹介するにとどめる。また、流体の方程式のような偏微分方程式を実際に時間発展させる数値解法そのものは扱わない。ただし、境界条件、陽的・陰的な時間積分、粒子の初期位置分布や重力計算、および宇宙流体計算で使われる 2 次・3 次精度 Runge–Kutta 法（RK2・RK3）との関係は、脚注や補足で紹介する。これらは今後の学習への案内であり、本書で新たに実装する数値実験ではない。これらの内容をさらに学ぶ場合は、巻末の参考図書を参照してほしい。

次の学習に向けて

本教材で扱った数値積分、数値微分、有限差分法、時間積分法は、宇宙流体シミュレーションや重力多体系シミュレーションで使われる数値計算法の基礎となる。一方、実際のシミュレーションでは、保存則に基づく離散化、数値流束、重力場の効率的な計算、複数の物理過程の結合など、さらに多くの数値計算法を組み合わせる必要がある。本教材のあとに学ぶ内容は、目的に応じて次のように選ぶとよい。

- **宇宙流体シミュレーションへ進む場合：**福江純・和田桂一・梅村雅之『宇宙流体力学の基礎［改訂版］』[4] では、宇宙流体の基礎方程式、自己重力流体、波動、衝撃波、流体不安定性などを通して、宇宙流体力学そのものの物理的な基礎を学ぶことができる。富阪幸治・花輪知幸・牧野淳一郎編『シミュレーション天文学』[8] では、常微分方程式と偏微分方程式の数値解法から、流体力学・磁気流体力学、自己重力を含む流体計算までを、天文シミ

ュレーションとの関係を意識しながら学ぶことができる。藤井孝藏・立川智章『Python で学ぶ流体力学の数値計算法』[10] では、圧縮性流体の数値計算法を Python による実装とともに学ぶことができる。松本倫明・大須賀健・須佐元『輻射電磁流体シミュレーションの基礎』[9] では、有限体積法、数値流束、SPH 法、磁気流体力学、輻射流体力学など、宇宙物理シミュレーションで用いられる数値計算法をより詳しく学ぶことができる。

- **重力多体系・銀河力学シミュレーションへ進む場合：** James Binney & Scott Tremaine, “*Galactic Dynamics*”[2] では、銀河の重力場の表現、多重極展開、基底関数展開、N 体計算における重力計算法などを詳しく学ぶことができる。富阪幸治・花輪知幸・牧野淳一郎編『シミュレーション天文学』[8] では、無衝突系・衝突系の重力多体系計算を含め、天文シミュレーションにおける N 体計算を広く学ぶことができる。より専門的には、Sverre J. Aarseth, “*Gravitational N-Body Simulations: Tools and Algorithms*”[11] では、直接重力計算、時間積分、近接遭遇、正則化、個別時間刻み、精度と計算効率など、重力 N 体シミュレーションのアルゴリズムを体系的に学ぶことができる。特に、星団のような衝突系の計算を詳しく学ぶ際の参考になる。Walter Dehnen & Justin I. Read, “*N-body simulations of gravitational dynamics*”[12] は、衝突系と無衝突系の両方について、重力 N 体シミュレーションで用いられる代表的な数値手法を整理したレビューであり、重力多体系計算の全体像を把握するための参考になる。

これらの専門的な教科書やレビューを読むときにも、本教材で繰り返した解析解との比較、収束試験、保存量による診断、誤差の原因を区別する考え方は、そのまま役立つ。

第 1 章　球対称質量モデルと数値積分

銀河や星団のような自己重力系を記述するとき、最初に必要になる量の一つが、密度分布から求める内包質量（enclosed mass）である。球対称な密度分布 $\rho(r)$ が与えられれば、半径 r より内側の質量は積分で求められる。しかし、密度分布が複雑になると積分を解析的に実行できるとは限らない。そこで本章では、Ryden §6.4 の球対称質量モデルを題材として、**数値積分（numerical integration）**の基本を学ぶ [1]。

中心となる例は Plummer 球である。Plummer 密度分布では内包質量を解析的にも求められるため、数値積分の答えがどの程度正しいかを直接確認できる。このように、**解析解が分かる問題を数値計算法の検証に使う**ことは、以後の章でも繰り返し用いる基本的な考え方である。

本章で学ぶこと

- 球対称な密度分布から内包質量を求める問題について、数値積分の被積分関数と積分区間を設定できる。
- 左長方形則、中点則、台形則、Simpson 則の近似の考え方を説明し、Python で実装できる。
- 累積積分を用いて、積分上限が異なる複数の積分値を効率よく計算できる。
- 解析解との比較と格子収束試験から、各積分公式の誤差と収束次数 p を評価できる。
- 離散化誤差と丸め誤差を区別し、格子を細かくしても精度が向上しなくなる理由を説明できる。

本章に対応する数値実験は、Notebook 1「球対称質量モデル」で行う。本文中の Listing には、導いた公式に対応する短い Python コードを示す。計算全体を実行して結果を図にするコードは Notebook にある。

1.1 球対称密度分布と内包質量

球殻の質量要素

球対称な密度分布を $\rho(r)$ とする。半径 r、厚さ dr の薄い球殻の体積は

$$dV = 4\pi r^2\, dr \tag{1.1}$$

であるから、その球殻に含まれる質量は

$$dM = \rho(r)\, dV = 4\pi r^2 \rho(r)\, dr \tag{1.2}$$

である。したがって、半径 r より内側の**内包質量（enclosed mass）**は

$$\boxed{M(<r) = \int_0^r 4\pi \rho(r')\, r'^2\, dr'} \tag{1.3}$$

となる。

ここで数値積分するのは $\rho(r)$ そのものではなく、

$$f_M(r) \equiv \frac{dM}{dr} = 4\pi r^2 \rho(r) \tag{1.4}$$

である。半径 r、厚さ dr の球殻の体積は $4\pi r^2 dr$ であるため、内包質量への寄与は密度 $\rho(r)$ だけでなく、球殻の面積 $4\pi r^2$ にも依存する。

Plummer 密度分布

Ryden §6.4.2 で扱われる Plummer 密度分布は、中心で密度が有限となる球対称モデルである [1]。本章では全質量（total mass）を M、スケール半径（scale radius）を a として、

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2} \tag{1.5}$$

を用いる。この分布は、中心では

$$\rho(0) = \frac{3M}{4\pi a^3} \tag{1.6}$$

で有限である。一方、$r \gg a$ では

$$\rho(r) \propto r^{-5} \tag{1.7}$$

となる。

Plummer 分布の被積分関数（integrand）は

$$f_M(r) \equiv \frac{dM}{dr} = 4\pi r^2 \rho(r) = \frac{3M}{a}\,\frac{(r/a)^2}{[1 + (r/a)^2]^{5/2}} \tag{1.8}$$

であり、中心では $dM/dr \propto r^2$、外側では $dM/dr \propto r^{-3}$ である。密度と被積分関数の違いを図1.1に示す。

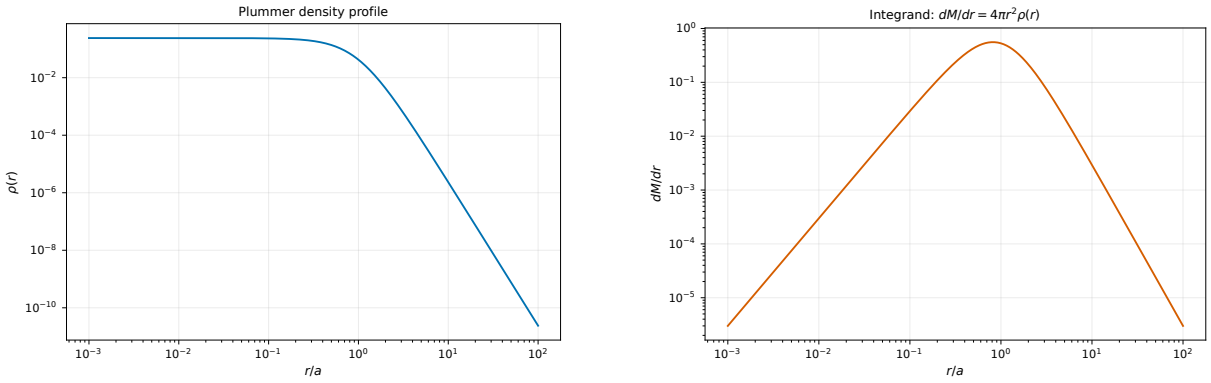


図 1.1　Plummer 密度分布の密度（左）と、内包質量の被積分関数 $dM/dr = 4\pi r^2\rho$（右）。縦軸はそれぞれ M/a^3、M/a を単位としている。密度は中心で最大であるが、同じ厚さの球殻の体積は r^2 に比例するため、dM/dr は中心で 0 になる。

解析的な内包質量

式(1.5)の Plummer 密度分布では式(1.3)を解析的に積分できる。無次元半径

$$x \equiv \frac{r}{a} \tag{1.9}$$

を導入する。積分変数を $u = r'/a$ に取り替えると、$r' = au$、$dr' = a\,du$ であり、積分の上端は $u = x$ になるので、

$$M(<r) = 4\pi \int_0^r \rho(r') r'^2\, dr' = 3M \int_0^x \frac{u^2}{(1+u^2)^{5/2}}\, du. \tag{1.10}$$

実際、

$$\frac{d}{du}\left[\frac{u^3}{(1+u^2)^{3/2}}\right] = \frac{3u^2}{(1+u^2)^{5/2}} \tag{1.11}$$

なので、この積分は解析的に計算できる。したがって、

$$\boxed{M(<r) = M\frac{x^3}{(1+x^2)^{3/2}} = M\frac{(r/a)^3}{[1+(r/a)^2]^{3/2}}} \tag{1.12}$$

を得る。$r \to \infty$ では $M(<r) \to M$ となり、M が全質量であることも確認できる。本章では式(1.12)を「正解」として使い、数値積分の精度を検証する。

以後、本章では半径を a、質量を M で規格化し、r/a と $M(<r)/M$ を用いて結果を表す。Notebook 1 では、この規格化を $M = a = 1$ として実装する。これは質量と長さの単位を選んで無次元量を扱っているだけであり、両者を同じ物理量とみなすことを意味しない。

1.2 数値積分の考え方

数値積分の積分公式を導くため、ここでは x を無次元半径に限らず一般の積分変数として用い、定積分

$$I = \int_{x_0}^{x_N} f(x)\,dx \tag{1.13}$$

を考える。区間 $[x_0, x_N]$ を N 個の小区間に等分し、格子点を

$$x_i = x_0 + ih, \qquad h = \frac{x_N - x_0}{N} \tag{1.14}$$

とする。ここで h は隣り合う格子点の間隔であり、**格子幅（grid spacing）**と呼ぶ。N は区間数であり、両端を含む格子点数は $N+1$ である。この分割によって定積分 I は

$$I = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x)\,dx \tag{1.15}$$

と書けることになる。

数値積分では、各小区間で $f(x)$ を簡単な関数で近似して積分する。例えば一定値で近似すれば長方形、1 次関数なら台形、2 次関数なら Simpson 則が得られる。

1.3 基本的な数値積分公式

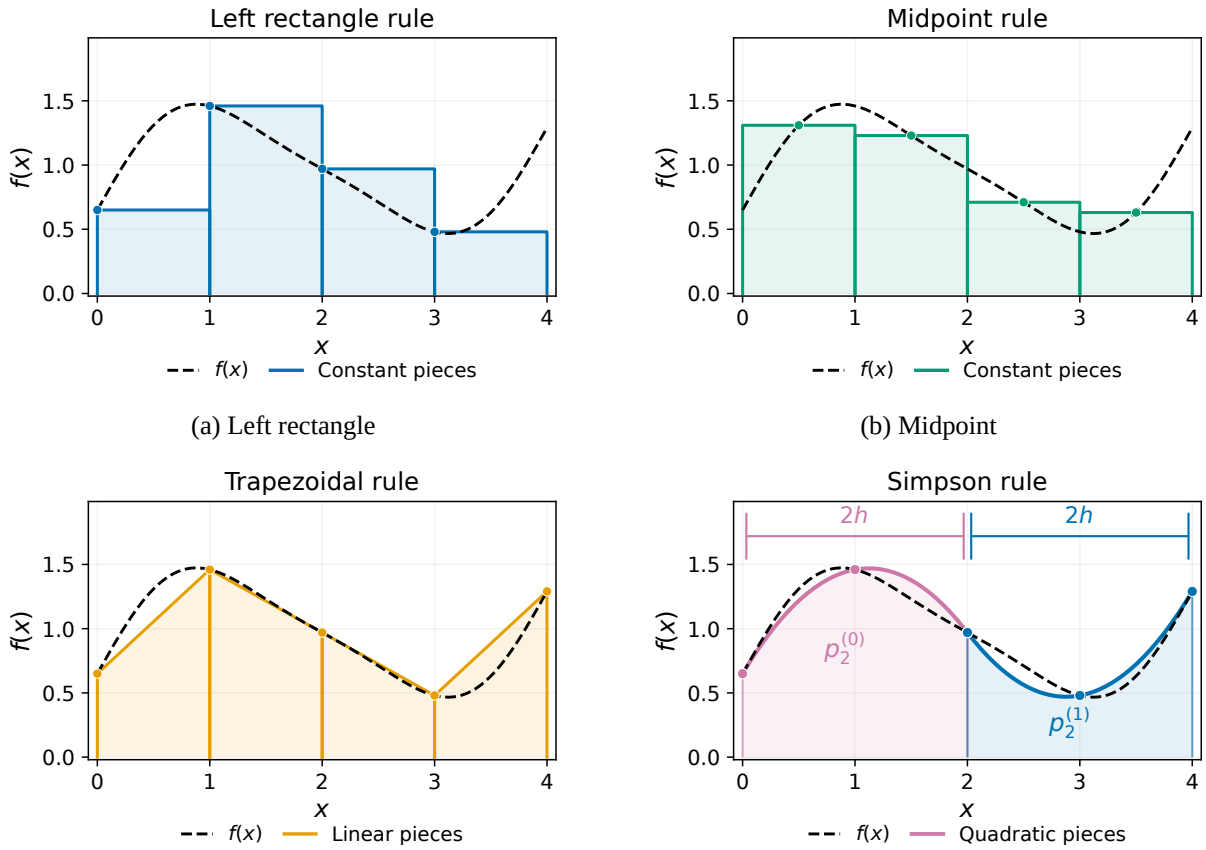
図1.2に、4 種類の積分公式の幾何学的な考え方を示す[1]。以下では、各公式が被積分関数をどのように近似するかと、その実装方法を順に見ていく。それぞれの方法で誤差がどの速さで減少するかは、後の収束試験で数値的に調べる。

左長方形則（left-rectangle rule）

小区間 $[x_i, x_{i+1}]$ で $f(x)$ を左端の値 $f_i \equiv f(x_i)$ で一定と近似すると、

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \simeq h f_i \tag{1.16}$$

[1] ここでは 4 つの積分公式を紹介するが、格子点上の値を積分する実際の計算では台形則がよく使われる。左長方形則は台形則と同じ格子点の値を使うにもかかわらず、誤差の減り方が遅い（§1.5）。中点則は区間の中点での値を必要とするため、格子点の値しかないデータにはそのまま使えない。ただし、セル中心に値を置く計算や、端点で被積分関数が発散する積分では中点則が適している。台形則は隣り合う 2 点の値だけで計算でき、不等間隔の格子にも使え、区間ごとの面積を順に足すだけで累積積分が得られる（§1.7）。Simpson 則は被積分関数が滑らかなときに誤差が速く減少するが、区間を 2 つずつ組にする必要があり、折れ曲がりや雑音を含むデータでは、その利点は小さくなる。格子法の流体計算で用いる有限体積法では、密度などのセル平均値を未知数とする。一次元の Cartesian 座標で、幅 h のセルを中心のまわりに対称に取ると、滑らかな関数のセル平均とセル中心の値との差は $O(h^2)$ となる。この関係は、中点則の誤差と同じように Taylor 展開から説明できる [9]。



(a) Left rectangle (b) Midpoint

(c) Trapezoidal (d) Simpson

図 1.2 数値積分公式の幾何学的な意味。黒破線は全パネルで共通の被積分関数 $f(x)$、色付き実線は各方法の近似である。Simpson 則では幅 $2h$ の各区間で、3 個の評価点を通る別々の二次多項式 $p_2^{(0)}, p_2^{(1)}$ を積分する。色付きの面積が積分の近似値に対応する。

である。この近似では、各小区間の左端 $x_i = x_0 + ih$ だけで関数を評価していることになる。全区間を足すと、左長方形則は次のように書ける。

$$\boxed{I_{\text{left}} = h \sum_{i=0}^{N-1} f_i} \tag{1.17}$$

Python では Listing 1.1 のように書ける。

Listing 1.1 左長方形則

```
import numpy as np
def left_rule(func, x0, xN, N):
    h = (xN - x0) / N
    x = x0 + np.arange(N) * h
    return h * np.sum(func(x))
```

中点則（midpoint rule）

小区間 $[x_i, x_{i+1}]$ で $f(x)$ を中点

$$x_{i+1/2} = x_i + \frac{h}{2} \tag{1.18}$$

での関数値を使えば、

$$\boxed{I_{\text{mid}} = h \sum_{i=0}^{N-1} f(x_{i+1/2})} \tag{1.19}$$

を得る。

中点 $x_{i+1/2} = x_i + h/2$ は、格子番号 i に $1/2$ を加えて作れる。式(1.19)の実装が Listing 1.2 である。

Listing 1.2　中点則

```
def midpoint_rule(func, x0, xN, N):
    h = (xN - x0) / N
    x_mid = x0 + (np.arange(N) + 0.5) * h
    return h * np.sum(func(x_mid))
```

台形則（trapezoidal rule）

台形則では、小区間内の $f(x)$ を両端 (x_i, f_i) と (x_{i+1}, f_{i+1}) を結ぶ直線で近似する。直線補間（linear interpolation）は

$$f(x) \simeq f_i + \frac{f_{i+1} - f_i}{h}(x - x_i) \tag{1.20}$$

である。これを x_i から x_{i+1} まで積分すると、

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \simeq \int_{x_i}^{x_{i+1}} \left[f_i + \frac{f_{i+1} - f_i}{h}(x - x_i) \right] dx \tag{1.21}$$

$$= \frac{h}{2}(f_i + f_{i+1}). \tag{1.22}$$

これが台形の面積そのものである。全区間を足すと内部格子点は 2 回ずつ現れるため、

$$\boxed{I_{\text{trap}} = h \left[\frac{1}{2} f_0 + \sum_{i=1}^{N-1} f_i + \frac{1}{2} f_N \right]} \tag{1.23}$$

を得る。

式(1.23)の各項をそのまま Python に対応させると、Listing 1.3 のように書ける。

Listing 1.3　台形則

```
def trapezoid_rule(func, x0, xN, N):
    h = (xN - x0) / N
    x = np.linspace(x0, xN, N + 1)
    y = func(x)

    return h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
```

左長方形則と中点則では各小区間の代表点 N 個を使ったが、台形則では両端を含む $N+1$ 個の格子点を使う。この実装では、それらを `np.linspace(x0, xN, N + 1)` で作っている。f_0 と f_N が `y[0]`, `y[-1]` に、内部点の和 $\sum_{i=1}^{N-1} f_i$ が `np.sum(y[1:-1])` に対応する。

Simpson 則（Simpson's rule）

Simpson 則では、隣り合う 2 つの小区間を 1 組として扱う。まず 1 組 $[x_i, x_{i+2}]$ を取り出し、等間隔の 3 点

$$x_i, \qquad x_{i+1} = x_i + h, \qquad x_{i+2} = x_i + 2h \tag{1.24}$$

における関数値を f_i, f_{i+1}, f_{i+2} とする。この 3 点を通る 2 次多項式で $f(x)$ を近似し、その多項式を積分する。

計算を簡単にするため、区間の中央 x_{i+1} を原点とする無次元変数

$$t = \frac{x - x_{i+1}}{h} \tag{1.25}$$

を導入する。このとき、3 点は $t = -1, 0, 1$ に対応する。2 次多項式を

$$p_2(t) = At^2 + Bt + C \tag{1.26}$$

とおく。3 点を通る条件は

$$p_2(-1) = A - B + C = f_i, \tag{1.27}$$
$$p_2(0) = C = f_{i+1}, \tag{1.28}$$
$$p_2(1) = A + B + C = f_{i+2} \tag{1.29}$$

である。これらを解くと、

$$A = \frac{f_i - 2f_{i+1} + f_{i+2}}{2}, \qquad B = \frac{f_{i+2} - f_i}{2}, \qquad C = f_{i+1} \tag{1.30}$$

を得る。

$dx = h\,dt$ なので、

$$\int_{x_i}^{x_{i+2}} f(x)\,dx \simeq h \int_{-1}^{1} p_2(t)\,dt \tag{1.31}$$
$$= h \int_{-1}^{1} (At^2 + Bt + C)\,dt$$
$$= h \left(\frac{2A}{3} + 2C \right)$$
$$= \frac{h}{3}(f_i + 4f_{i+1} + f_{i+2}) \tag{1.32}$$

を得る。これが 1 組に対する Simpson の 1/3 公式である [5]。

次に、積分区間全体を幅 h の N 個の小区間に分割する。Simpson 則は 2 つの小区間を 1 組として適用するため、N は偶数とする。例えば、$N = 4$ の場合を考える。最初の組 $[x_0, x_2]$ と次の組 $[x_2, x_4]$ に Simpson 則を適用して足し合わせると、

$$I_{\text{Simpson}} = \frac{h}{3}(f_0 + 4f_1 + f_2) + \frac{h}{3}(f_2 + 4f_3 + f_4) \tag{1.33}$$
$$= \frac{h}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + f_4) \tag{1.34}$$

となる。ここで、f_2 は両方の組に含まれるため、係数が 2 になる。同様に、N が一般の偶数の場合には、積分区間の両端の係数が 1、奇数番号の点の係数が 4、両端を除く偶数番号の点の係数が 2 となる。したがって、

$$\boxed{I_{\text{Simpson}} = \frac{h}{3}\left[f_0 + f_N + 4\sum_{j=1}^{N/2} f_{2j-1} + 2\sum_{j=1}^{N/2-1} f_{2j} \right]} \tag{1.35}$$

を得る。

この公式では、奇数番号と偶数番号の内部点を別々に足す必要がある。NumPy のスライスを使うと、Listing 1.4 のように数式の構造をほぼそのまま表現できる。

Listing 1.4　Simpson 則

```
def simpson_rule(func, x0, xN, N):
    if N % 2 != 0:
        raise ValueError("Simpson 則では N を偶数にしてください。")

    h = (xN - x0) / N
    x = np.linspace(x0, xN, N + 1)
    y = func(x)

    odd_sum = np.sum(y[1:-1:2])
    even_sum = np.sum(y[2:-1:2])

    return h / 3.0 * (y[0] + y[-1] + 4.0 * odd_sum + 2.0 * even_sum)
```

`y[1:-1:2]` は $y_1, y_3, y_5, \ldots$、`y[2:-1:2]` は $y_2, y_4, y_6, \ldots$ を取り出す。したがって、それぞれ式(1.35)の奇数番号の和と偶数番号の和に対応する。

1.4 Plummer 球による 4 つの積分法の比較

Plummer 球について、半径 $r = a$ までの内包質量を求める。$x = r/a$ を用いると、式(1.10)より、比較する積分は

$$I \equiv \frac{M(<a)}{M} = \int_0^1 \frac{3x^2}{(1+x^2)^{5/2}}\, dx \tag{1.36}$$

である。すなわち、$f(x) = 3x^2/(1+x^2)^{5/2}$ に前節で導いた 4 つの積分公式を適用する。この $f(x)$ は、式(1.4)の被積分関数 $f_M(r) = dM/dr$ を M/a で割り、$r = ax$ としたものである。Notebook 1 では $M = a = 1$ とするため、`dMdr_plummer` の返す値がそのまま $f(x)$ に対応する。

数値積分の精度を評価するために、格子幅 h で求めた数値積分値を I_h とし、解析値 I との差の大きさ

$$\epsilon(h) \equiv |I_h - I| \tag{1.37}$$

を**絶対誤差（absolute error）**と定義する。これは、全質量 M で規格化した内包質量の誤差である。なお、ここでは $r = a$ を評価点とし、解析値

$$I = \frac{M(<a)}{M} = \frac{1}{2\sqrt{2}} \simeq 0.353553390593 \tag{1.38}$$

を比較基準とする。

Notebook 1 ｜ 4 つの積分法の比較

$M = a = 1$、$N = 40$ として、4 つの積分法を同じ条件で比較する。数値積分値と解析値との差から絶対誤差を計算し、その結果を同じセルの出力として表示する。

対応セル（計算部分）

```
# 積分区間を 0 <= r <= a とし、区間数を N=40 にする
r_eval = 1.0 * a
N = 40

# 被積分関数 dM/dr = 4 pi r^2 rho(r)
func = lambda r: dMdr_plummer(r, M, a)

# 同じ条件で 4 つの数値積分法を適用する
m_left = left_rule(func, 0.0, r_eval, N)
```

```
m_mid = midpoint_rule(func, 0.0, r_eval, N)
m_trap = trapezoid_rule(func, 0.0, r_eval, N)
m_simp = simpson_rule(func, 0.0, r_eval, N)

# 数値積分値と比較する解析値
m_exact_eval = mass_plummer_exact(r_eval, M, a)

print(f" 解析値      : {m_exact_eval:.12f}")
print(f" 左長方形則  : {m_left:.12f}   誤差 = {abs(m_left - m_exact_eval):.3e}")
print(f" 中点則      : {m_mid:.12f}   誤差 = {abs(m_mid - m_exact_eval):.3e}")
print(f" 台形則      : {m_trap:.12f}   誤差 = {abs(m_trap - m_exact_eval):.3e}")
print(f"Simpson 則 : {m_simp:.12f}   誤差 = {abs(m_simp - m_exact_eval):.3e}")
```

実行結果

```
解析値                  : 0.353553390593
左長方形則              : 0.346910451143   誤差 = 6.643e-03
中点則                  : 0.353560298293   誤差 = 6.908e-06
台形則                  : 0.353539577216   誤差 = 1.381e-05
Simpson 則              : 0.353553401385   誤差 = 1.079e-08
```

この結果から、$N = 40$ では Simpson 則の誤差が最も小さいことがわかる。しかし、この 1 つの格子だけでは、格子を細かくしたときに誤差がどのように減るかは分からない。そこで次に、格子幅と絶対誤差 $\epsilon(h)$ の関係を調べる収束試験を行う。

1.5 収束次数の測定

誤差の h 依存性

前節では、$N = 40$ の 1 つの格子で 4 つの積分法の誤差を比較した。次に、格子幅 h を小さくしたときに、絶対誤差 $\epsilon(h)$ がどの速さで減少するかを調べる。多くの数値計算法では、格子を十分に細かくすると、誤差は格子幅 h のべき乗に従って規則的に減少する。この誤差の減少の速さを表す量を**収束次数**という。

収束次数

十分小さい格子幅 h に対して、誤差が

$$\epsilon(h) \simeq Ch^p \tag{1.39}$$

と振る舞うとき、指数 $p > 0$ をその数値計算法の**収束次数（order of convergence）**と呼ぶ。ここで $C > 0$ は h に依存しない定数である。

収束次数が p の方法を、p **次精度**の方法と呼ぶ。例えば $p = 2$ なら、格子幅を半分にすると誤差はおよそ $1/4$ になる。$p = 4$ なら、およそ $1/16$ になる。したがって、収束次数が大きい方法ほど、格子を細かくしたときに誤差が速く減少する。

数値計算上の注意｜収束次数と誤差の大きさは同じではない

式(1.39)では、収束次数 p は格子幅を小さくしたときに誤差がどの速さで減るかを表し、係数 C は誤差の大きさに関係する。したがって、同じ収束次数をもつ方法でも、同じ h での誤差が同じとは限らない。また、有限の h では、収束次数が高い方法が必ず最小の誤差を与えるとは限らない。

数値実験による収束次数の測定

それでは、Plummer 球の内包質量を用いて、実際に収束次数を測ってみる。収束次数を数値的に測定するには、式(1.39)の対数を取り、

$$\log \epsilon = p \log h + \log C \tag{1.40}$$

とする。したがって、$\log \epsilon$ を $\log h$ に対してプロットすると、式(1.39)が成り立つ範囲では点列はほぼ直線上に並び、その傾きが収束次数 p となる。対数の底を変えても傾き p は変わらない。Notebook 1 では、`np.log` と `np.polyfit` を用いて複数の格子幅の点を直線で近似し、その傾きから p を求める。

そこで、複数の格子幅で同じ積分を計算し、解析値との差から得た絶対誤差を用いて収束次数を測定する。どの積分公式で p がどの程度になるかを、まず数値実験から確かめてみる。

Notebook 1 ｜収束試験

左長方形則・中点則・台形則・Simpson 則について、$N = 10, 20, 40, 80, 160, 320$ と格子を細かくする。各方法について $\log \epsilon$ と $\log h$ を直線で近似し、その傾きから収束次数 p を求める。そのために、ここでは NumPy の `np.polyfit(x, y, 1)` を用いる。この関数は、データ (x, y) を直線 $y = ax + b$ で最小二乗近似し、傾き a と切片 b をこの順に返す。したがって、末尾の `[0]` で傾き、すなわち収束次数 p を取り出す。

対応セル（計算部分）

```
# 区間数 N を 2 倍ずつ増やして格子幅を小さくする
N_values = np.array([10, 20, 40, 80, 160, 320])
h_values = r_eval / N_values

# 4 つの積分法を同じ形で呼び出せるようにまとめる
methods = {
    "Left rectangle": lambda N: left_rule(func, 0.0, r_eval, N),
    "Midpoint": lambda N: midpoint_rule(func, 0.0, r_eval, N),
    "Trapezoidal": lambda N: trapezoid_rule(func, 0.0, r_eval, N),
    "Simpson": lambda N: simpson_rule(func, 0.0, r_eval, N),
}

errors = {}
orders = {}
for name, method in methods.items():
    # 各 N で解析値との差を絶対誤差として測る
    err = np.array([abs(method(int(N)) - m_exact_eval) for N in N_values])
    errors[name] = err

    # log(error)-log(h) の傾きから収束次数 p を求める
    orders[name] = np.polyfit(np.log(h_values), np.log(err), 1)[0]

method_names_ja = {
```

```
    "Left rectangle": " 左長方形則",
    "Midpoint": " 中点則",
    "Trapezoidal": " 台形則",
    "Simpson": "Simpson 則",
}

for name in methods:
    print(f"{method_names_ja[name]:10s}: 測定した収束次数 = {orders[name]:.3f}"
```

実行結果

```
左長方形則      : 測定した収束次数 = 1.002
中点則          : 測定した収束次数 = 2.001
台形則          : 測定した収束次数 = 2.001
Simpson 則      : 測定した収束次数 = 4.001
```

この結果から、左長方形則はほぼ 1 次、中点則と台形則はほぼ 2 次、Simpson 則はほぼ 4 次で収束していることがわかる。また、誤差と格子幅の関係を図1.3に示す。各方法の点列は両対数表示でほぼ直線となり、その傾きがそれぞれの収束次数に対応している。

このような**格子収束試験（grid-convergence test）**は、数値計算法の収束次数を実際の計算から調べる基本的な検証方法である。ただし、格子を極端に細かくすると、誤差がこのべき則から外れる場合がある。その主な原因の一つが、次節で扱う浮動小数点演算の丸め誤差である。

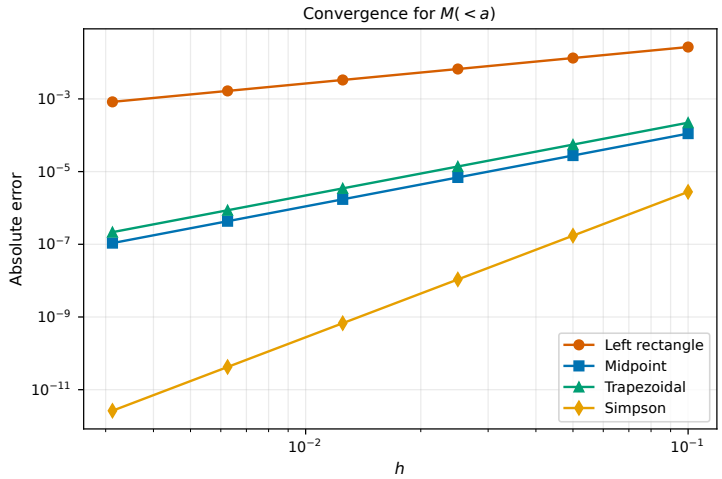


図 1.3 $M(<a)/M$ の絶対誤差と格子幅 h の関係。両対数表示での傾きが収束次数 p である。ここで h は無次元半径 $x = r/a$ に対する格子幅であり、$h = 1/N$ である。

補足｜Taylor 展開から収束次数を理解する

ここまでの収束試験では、数値実験から左長方形則、中点則、台形則、Simpson 則の収束次数を測定した。ここでは中点則を例に、なぜ局所的な近似誤差から積分全体の収束次数が決まるのかを見る。この補足は、以後の節を理解するための必須事項ではない。

以下で $O(h^q)$ は、$h \to 0$ のとき、その項の大きさが h^q の定数倍以下に抑えられることを表す。1 区間で生じる誤差を**局所打ち切り誤差**、積分区間全体で得られる誤差を**大域誤差**と呼ぶ。

中点則を区間 $[x_i, x_{i+1}]$ に適用する。区間幅を $h = x_{i+1} - x_i$、中点を

$$x_{i+1/2} = \frac{x_i + x_{i+1}}{2} \tag{1.41}$$

とする。中点からのずれを $\xi = x - x_{i+1/2}$ とおけば、積分範囲は $-h/2 \le \xi \le h/2$ となる。中点のまわりで Taylor 展開すると、

$$f(x_{i+1/2} + \xi) = f_{i+1/2} + \xi f'_{i+1/2} + \frac{\xi^2}{2} f''_{i+1/2} + \frac{\xi^3}{6} f'''_{i+1/2} + O(\xi^4) \tag{1.42}$$

である。これを $-h/2$ から $h/2$ まで積分する。対称な区間では ξ と ξ^3 のような奇数次の項の積分が 0 になるので、

$$\begin{aligned} \int_{x_i}^{x_{i+1}} f(x)\,dx &= \int_{-h/2}^{h/2} f(x_{i+1/2} + \xi)\,d\xi \\ &= h f_{i+1/2} + \frac{h^3}{24} f''_{i+1/2} + O(h^5) \end{aligned} \tag{1.43}$$

を得る。

中点則は右辺の最初の項 $h f_{i+1/2}$ だけを使う。したがって、1 区間での局所打ち切り誤差は

$$E_i = \frac{h^3}{24} f''_{i+1/2} + O(h^5) = O(h^3) \tag{1.44}$$

である。

次に、積分区間全体 $[a, b]$ を考える。区間数は

$$N = \frac{b - a}{h} \propto h^{-1} \tag{1.45}$$

である。f'' が積分区間で有界なら、各区間の局所打ち切り誤差は一様に $O(h^3)$ なので、それらを N 区間にわたって合わせた大域誤差は

$$E = \sum_{i=0}^{N-1} E_i = O(N h^3) = O(h^2) \tag{1.46}$$

となる。したがって、中点則は **2 次精度**である。

他の積分公式でも考え方は同じである。局所打ち切り誤差が $O(h^{p+1})$ なら、固定した積分区間では区間数が $O(h^{-1})$ であるため、大域誤差は $O(h^p)$ となる。本章で用いた 4 つの公式では以下のようになる。なお、Simpson 則では 2 区間 1 組を単位として局所誤差を数えるが、組の数も $O(h^{-1})$ なので同じ考え方で 4 次精度が得られる。

方法	局所打ち切り誤差	大域誤差
左長方形則	$O(h^2)$	$O(h)$
中点則	$O(h^3)$	$O(h^2)$
台形則	$O(h^3)$	$O(h^2)$
Simpson 則	$O(h^5)$	$O(h^4)$

この結果は、Notebook 1 の収束試験で測定した $p \simeq 1, 2, 2, 4$ と一致する。より一般的な積分則の誤差評価については杉浦第 3 章 [6] を、Newton–Cotes 型積分公式の導出と実装については幸谷第 12 章 [5] を参照されたい。

1.6 離散化誤差と丸め誤差

前節の収束試験では、格子幅 h を小さくすると、数値積分の誤差が h^p に従って減少することを確認した。数値計算では、このような数値公式による近似だけでなく、計算機が有限桁の数を扱うことによっても誤差が生じる。本節では、これらに対応する**離散化誤差**と**丸め誤差**について考える。

離散化誤差（discretization error）

数値計算では、連続な問題をそのまま計算機で扱うのではなく、有限個の格子点や時間刻みを用いた離散的な問題に置き換える。本章の数値積分では、積分区間を有限個の小区間に分け、それぞれの区間で被積分関数を簡単な関数で近似した。このような離散化によって、数値積分値には厳密な積分値との差が生じる。この差を**離散化誤差（discretization error）**という。

離散化によって生じる誤差は、「1 回の近似でどれだけずれるか」と「計算全体の結果がどれだけずれるか」という 2 つの観点から見ることができる。数値積分では、各小区間で近似を行うたびに誤差が生じ、それらを全区間に適用した最終的な積分値にも誤差が残る。この 2 つの見方に対応するのが、局所打ち切り誤差と大域誤差である。

局所打ち切り誤差と大域誤差

微分、積分、時間発展などの連続な関係を有限の近似式へ置き換えたときに生じるずれを **打ち切り誤差（truncation error）**という。本書では、1 つの小区間、1 ステップ、または 1 格子点で評価したものを **局所打ち切り誤差（local truncation error）**と呼ぶ。
一方、離散公式を計算全体に適用して得られた数値結果と厳密な結果との差を **大域誤差（global error）**という。本章の数値積分では、丸め誤差などを無視できる場合、大域誤差は各区間の局所的な打ち切り誤差を積分範囲全体で合わせた結果として現れる。時間発展を伴う問題では、局所的な誤差の伝播や増幅も重要になる。

前節の収束試験で測定した収束次数 p は、この大域誤差が格子幅 h に対してどの速さで減少するかを表している。本章の例では、左長方形則の誤差はおよそ h、中点則と台形則では h^2、Simpson 則では h^4 に比例して減少した。離散化誤差だけを考えるなら、格子幅 h を小さくするほど数値積分値は解析値に近づく。

丸め誤差（round-off error）

実際の計算機では、実数を無限の精度で表すことはできない。数値計算では、実数を有限桁の**浮動小数点数（floating-point number）**で近似して扱う。浮動小数点数とは、例えば 1.23×10^4 のように、数を有効数字と指数に分けて表す方法である。計算機では通常、2 進数を用いて数を表すが、表現できる有効桁数には限りがある。そのため、演算結果が表現可能な数に一致しない場合には、近くの値へ丸められる。このように、有限桁の浮動小数点数で数値を表し、演算結果を丸めることによって生じる誤差を **丸め誤差（round-off error）**という。

浮動小数点数の精度を表す目安の一つが、**機械イプシロン（machine epsilon）**である。機械イプシロン ϵ_{mach} は、1 と、それより大きい最も近い浮動小数点数との差として定義される。つまり、1 付近でどれほど小さな数の差を区別できるかを表している。Notebook 1 で用いる倍精度浮動小数点数（64 ビット）では、

$$\epsilon_{\mathrm{mach}} \simeq 2.220 \times 10^{-16} \tag{1.47}$$

である [5]。ただし、機械イプシロンは計算結果全体の誤差を表すものではない。丸め誤差は演算を繰り返すことで蓄積する場合がある。また、近い値どうしを引くと、もとの数値に含まれる小さな誤差が計算結果に大きく影響することがある。

一般に、格子を細かくすると離散化誤差は小さくなる。しかし、離散化誤差が十分に小さくなると、丸め誤差の影響を無視できなくなる場合がある。そのため、格子を細かくしても計算

精度が上がり続けるとは限らない。Simpson 則では離散化誤差がおよそ h^4 に比例して速く減少するため、この変化を確認しやすい。

Notebook 1 ｜ Simpson 則と丸め誤差

$N \sim 10^3$–10^4 以上まで増やし、離散化誤差が十分小さくなった後に丸め誤差の影響が現れる様子を確認する。

対応セル（計算部分）

```
# Simpson 則で、さらに細かい格子まで調べる
N_large = np.array([320, 640, 1280, 2560,
                    5120, 10240, 20480])
err_large = []

for N_i in N_large:
    # 各 N で Simpson 則を適用し、解析値との誤差を保存する
    val = simpson_rule(func, 0.0, r_eval, int(N_i))
    err_large.append(abs(val - m_exact_eval))

# 倍精度浮動小数点数の機械イプシロン
machine_eps = np.finfo(float).eps
```

図1.4に示すように、この計算では区間数を増やすと、最初は h^4 に従って誤差が減少する。しかし、誤差が 10^{-15}–10^{-16} 程度まで小さくなると丸め誤差の影響が現れ、この関係から外れる。この領域では、格子をさらに細かくしても精度は系統的には向上しない。

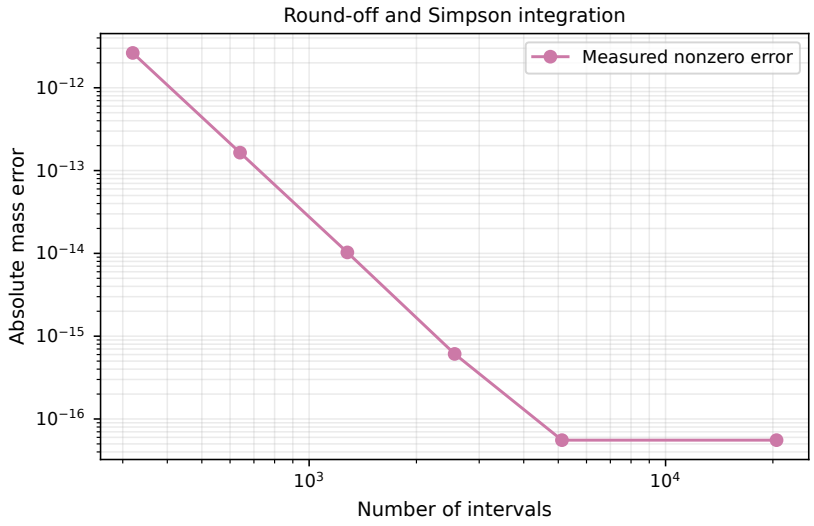


図 1.4　Simpson 則の区間数 N を増やしたときの、$M(<a)/M$ の絶対誤差。離散化誤差が十分小さくなると、倍精度浮動小数点数の丸め誤差の影響が見えてくる。

1.7 累積積分による内包質量プロファイル

これまでは固定した半径 $r = a$ までの 1 つの定積分を比較した。次に、すべての格子点 r_i について $M(< r_i)$ を求める。しかし、各半径について 0 から積分をやり直すと、同じ区間の寄与を何度も計算することになる。隣の区間の寄与を順に加える**累積積分（cumulative integration）**を使えば、この重複を避けられる。台形則では、隣り合う 2 点の値から各区間の積分を求めら

れるため、寄与を順に足し上げやすい。また、区間幅が一定でない格子にも同じ形で適用できる。

中心 $r_0 = 0$ から外側へ並ぶ格子 $r_0 < r_1 < \cdots < r_N$ を考え、$M(< r_i)$ の数値近似を M_i と書く。格子は不等間隔でもよいように、

$$\Delta r_i = r_{i+1} - r_i \tag{1.48}$$

とする。被積分関数を

$$f_{M,i} = 4\pi r_i^2 \rho(r_i) \tag{1.49}$$

とすると、台形則で各区間の質量は

$$\Delta M_i \simeq \frac{f_{M,i} + f_{M,i+1}}{2} \Delta r_i \tag{1.50}$$

である。したがって

$$M_{i+1} = M_i + \Delta M_i, \qquad M_0 = 0 \tag{1.51}$$

として中心から外側へ順に加えれば、全格子点の内包質量を得られる。

この手順を NumPy の配列演算で書くと、隣り合う 2 点の台形面積を一度に計算し、`np.cumsum` で左から累積できる。

Listing 1.5　累積台形積分

```
def cumulative_trapezoid_manual(x, y):
    result = np.zeros_like(x, dtype=float)

    dx = np.diff(x)
    area = 0.5 * (y[:-1] + y[1:]) * dx

    result[1:] = np.cumsum(area)
    return result
```

Notebook 1 ｜内包質量プロファイル

$r_{\max} = 20a$ まで 400 格子点を置き、Listing 1.5 の累積台形積分を使って $M(< r)$ を求める。

対応セル（計算部分）

```
# 0 <= r <= 20a に 400 個の格子点を置く
N_profile = 400
r_max = 20.0 * a
r = np.linspace(0.0, r_max, N_profile)

# 各格子点で被積分関数 dM/dr を評価する
dMdr = dMdr_plummer(r, M, a)

# 累積台形則で M(<r) を外側へ順に積み上げる
m_num = cumulative_trapezoid_manual(r, dMdr)

# 同じ格子点で解析解も計算して比較する
m_exact = mass_plummer_exact(r, M, a)
```

ここで `np.linspace` は、指定した範囲の両端を含む格子点を等間隔に配置する。ここでは 400 個の格子点を置くため、隣り合う格子点の間の区間数は 399 となる。図1.5に示すように、得られた数値積分の結果は解析式とほぼ重なる。

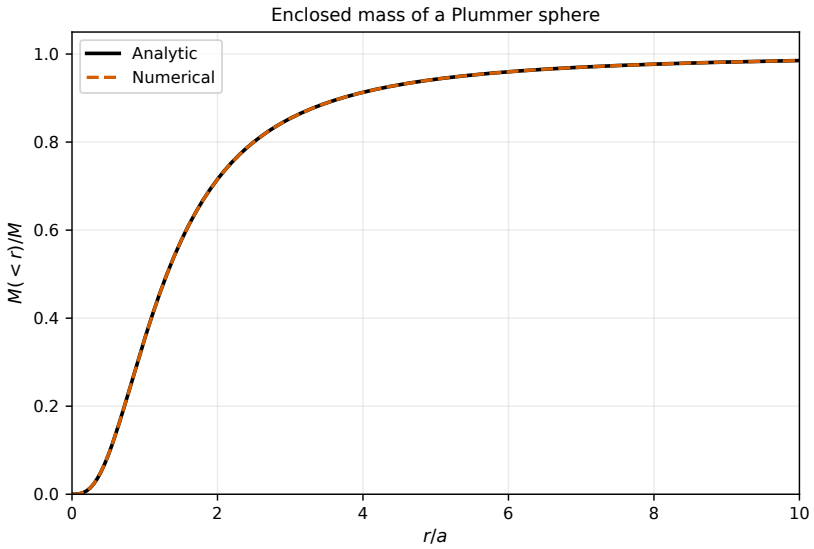


図 1.5　Plummer 球の内包質量。台形則による累積数値積分と解析解を比較している。計算は $r_{\max} = 20a$ まで行い、図は $r \le 10a$ の範囲を示す。

1.8 内包質量と円運動速度

球対称系では Newton の球殻定理（shell theorem）により、半径 r における重力加速度は、その半径より内側の質量 $M(<r)$ だけで決まる。ここで G は万有引力定数、g_r は重力加速度の動径成分とし、動径方向は外向きを正に取る。このとき

$$g_r(r) = -\frac{GM(<r)}{r^2} \tag{1.52}$$

である。

半径 r の円軌道を速さ $v_{\rm circ}$ で運動する粒子の半径方向加速度は

$$a_r = -\frac{v_{\rm circ}^2}{r} \tag{1.53}$$

である。円運動では、この加速度を重力が与えるので $a_r = g_r$ である。したがって、

$$-\frac{v_{\rm circ}^2}{r} = -\frac{GM(<r)}{r^2} \tag{1.54}$$

となり、

$$\boxed{v_{\rm circ}(r) = \sqrt{\frac{GM(<r)}{r}}} \tag{1.55}$$

を得る [1]。式(1.55)は $r > 0$ について書いた式である。Plummer 球では中心付近で $M(<r) \simeq M(r/a)^3$ なので、$v_{\rm circ} \simeq \sqrt{GM/a^3}\, r$ となり、$r \to 0$ で $v_{\rm circ} \to 0$ である。Notebook 1 では、この極限に対応して $r = 0$ の値を 0 として扱う。円運動速度を半径の関数として表したものを**回転曲線（rotation curve）**と呼ぶ。したがって、数値積分によって $M(<r)$ が得られれば、式(1.55)から回転曲線を計算できる。

Plummer 球では内包質量の解析式も分かっているため、回転曲線を解析的に求めることもできる。式(1.12)を式(1.55)に代入すると、

$$v_{\rm circ}^2(r) = \frac{GMr^2}{(r^2+a^2)^{3/2}} \tag{1.56}$$

となる。この解析式は、数値積分で得た $M(<r)$ から計算した回転曲線を検証する基準として使える。

ここで、速度の単位を

$$v_* \equiv \sqrt{\frac{GM}{a}} \tag{1.57}$$

とし、無次元半径 $x = r/a$ を導入する。式(1.56)は

$$\frac{v_{\rm circ}^2}{v_*^2} = \frac{x^2}{(1+x^2)^{3/2}} \tag{1.58}$$

となる。Notebook 1 では $G = M = a = 1$ としているため、$v_* = 1$ である。物理単位へ戻すには、得られた無次元速度に $v_* = \sqrt{GM/a}$ を掛ければよい。

この解析式を使えば、回転曲線が最大となる半径も求められる。$v_{\rm circ} > 0$ なので、$v_{\rm circ}$ を最大にする半径は $v_{\rm circ}^2$ を最大にする半径と同じである。式(1.58)の右辺を

$$q(x) = x^2(1+x^2)^{-3/2} \tag{1.59}$$

と置く。微分すると

$$\frac{dq}{dx} = \frac{x(2-x^2)}{(1+x^2)^{5/2}}. \tag{1.60}$$

$x > 0$ で $dq/dx = 0$ を満たすのは $x = \sqrt{2}$ である。導関数はこの点の前後で正から負に変わるため、円運動速度が最大となる半径は

$$r_{\rm peak} = \sqrt{2}\,a \tag{1.61}$$

である。このとき

$$\frac{v_{\rm peak}^2}{GM/a} = \frac{2}{3\sqrt{3}}, \qquad \frac{v_{\rm peak}}{\sqrt{GM/a}} = \sqrt{\frac{2}{3\sqrt{3}}} = 0.620403. \tag{1.62}$$

Notebook 1 ｜円運動速度

数値積分で得た内包質量 $M(<r)$ から式(1.55)を用いて回転曲線を計算し、Plummer 球の解析式と比較する。また、数値的な回転曲線が最大となる格子点を求め、解析値 $r_{\rm peak} = \sqrt{2}\,a$ と比較する。

対応セル（計算部分）

```
# 数値積分で得た M(<r) から円運動速度を求める
vc_num = vcirc_from_mass(r, m_num, G)

# 解析的な M(<r) から求めた円運動速度を比較用に用意する
vc_exact = vcirc_from_mass(r, m_exact, G)

# 数値的な回転曲線が最大となる格子点を探す
i_peak = np.argmax(vc_num)
print(f" 数値解の最大位置: r/a = {r[i_peak] / a:.4f}, v_circ = {vc_num[i_peak]: 6f}")

# 解析的な最大位置 r_peak/a = sqrt(2) も表示する
print(f" 解析的な最大半径 sqrt(2) = {np.sqrt(2.0):.6f}")
```

実行結果

```
数値解の最大位置: r/a = 1.4035, v_circ = 0.620347
解析的な最大半径 sqrt(2) = 1.414214
```

図1.6では、数値積分で得た内包質量から計算した回転曲線と解析式を比較している。両者はよく一致しているが、400 格子点の計算で求めた最大位置は

$$r_{\mathrm{peak,num}}/a = 1.403509 \tag{1.63}$$

となり、解析値 $\sqrt{2} = 1.414214$ から少しずれる。これは回転曲線そのものの誤差だけでなく、最大位置を連続的に探索せず、離散的な格子点の中から最大値を選んでいるためである。より正確な最大位置が必要な場合は、最大付近を補間するか、1 変数の最適化を用いる必要がある。

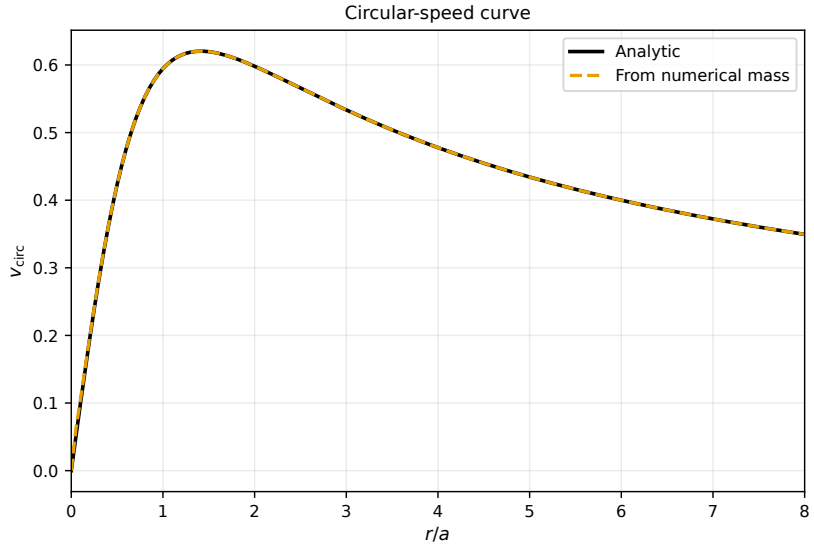


図 1.6　Plummer 球の回転曲線。縦軸は $v_* = \sqrt{GM/a}$ を単位としている。数値積分で得た内包質量から計算した回転曲線は解析式とよく一致する。解析的な最大位置は $r = \sqrt{2}a$ である。

補足｜高次元の積分とモンテカルロ積分

ここまでで、本章で扱う 1 次元数値積分とその物理的応用は完結した。最後に、積分の次元が高くなると計算量と誤差の関係がどのように変わるかを簡単に見ておく。

本章では、積分区間を格子に分け、台形則や Simpson 則で 1 次元の定積分を求めた。1 次元で誤差が h^p で減る積分公式では、滑らかな関数に対して格子を細かくするほど急速に精度が上がる。

一方、d 次元の積分で各方向を n 分割すると、総評価点数はおよそ $N_{\mathrm{eval}} = n^d$ になる。1 次元の積分公式を各方向に同じように適用すると、$h \sim N_{\mathrm{eval}}^{-1/d}$ なので、誤差はおおよそ

$$\epsilon \sim N_{\mathrm{eval}}^{-p/d}$$

で減る。したがって、次元が高くなるほど、同じ評価点数で得られる精度は低下する。

これに対して、積分領域から無作為に点を選び、被積分関数の平均値から積分値を推定する方法を **モンテカルロ積分（Monte Carlo integration）** という。区間 $[x_{\min}, x_{\max}]$ では

$$I = \int_{x_{\min}}^{x_{\max}} f(x)\,dx \simeq \frac{x_{\max} - x_{\min}}{N_{\mathrm{samp}}} \sum_{k=1}^{N_{\mathrm{samp}}} f(x_k)$$

と見積もる。ここで x_k は区間内から独立に、一様に選んだ点であり、N_{samp} はサンプル数である。被積分関数の分散が有限なら、この推定値の標準誤差は $N_{\mathrm{samp}}^{-1/2}$ に比例して減り、その指数は積分の次元に直接依存しない。したがって、1 次元の滑らかな積分では本章の方法のほうがはるかに効率的である。一方、次元が高い場合や積分領域の形が複雑な

場合には、モンテカルロ積分が有力な選択肢になる。天文学では、輻射輸送でも乱数によるサンプリングが利用されている [9]。

重力多体系シミュレーションで、等質量粒子の初期位置を作るときにも、乱数と内包質量を使う。球対称分布の全質量を M とし、$0 < X < 1$ の一様乱数を生成して $M(<r)/M = X$ を満たす半径 r を求める。この操作を**逆関数法（inverse transform sampling）**という。半径の確率密度は $\rho(r)$ そのものではなく $4\pi r^2\rho(r)/M$ であり、方向も等方的に選ぶことで、密度分布 $\rho(r)$ に対応する粒子配置を作れる。Plummer 球では式(1.12)を逆に解き、

$$r = a\left(X^{-2/3} - 1\right)^{-1/2}$$

と求められる。一般の密度分布では、累積積分で得た内包質量の表を逆にたどり、必要に応じて補間して半径を決める。有限半径 $r_{\max}$ までの表を使う場合は、その半径までの質量で規格化し、$M(<r) = XM(<r_{\max})$ を満たす半径を求める。これは位置分布の作成であり、平衡な初期条件を作るには、質量分布と整合する速度分布も別途与える必要がある。粒子を独立に抽出すると、一定の半径内に入る粒子数の割合には統計的なゆらぎが残る。その標準偏差は粒子数 $N_{\rm p}$ に対して $N_{\rm p}^{-1/2}$ に比例する。これは乱数サンプリングのゆらぎであり、累積積分の離散化誤差や、その後の時間積分の誤差とは区別する。

詳しくは、乱数とモンテカルロ法は三井田・須田 [7] 第 9 章、重力多体系の計算は富阪ほか編 [8] 第 6 章を参照すること。

第 2 章　Poisson 方程式と有限差分法

第 1 章では、球対称な密度分布を数値積分して重力場を求めた。本章では、重力ポテンシャルを介して同じ重力場を求める過程を通して、境界値問題の数値解法と数値微分を学ぶ。

基本となる手法は、格子点上の関数値の差を用いて微分を近似する**有限差分法（finite-difference method）**である。この考え方は、さまざまな常微分方程式や偏微分方程式に適用できる。本章で Poisson 方程式を取り上げるのは、離散化、境界条件の組み込み、連立一次方程式の構成、数値解の検証を、解析解の分かる重力モデルで学べるためである。実際の重力計算法は質量分布の表現や計算の目的によって異なるが[1]、ここでは手順を追いやすい有限差分法を用い、1 次元の境界値問題として解く。さらに、得られたポテンシャルから重力加速度を求めるため、前進差分・後退差分・中心差分による**数値微分（numerical differentiation）**を行い、それぞれの精度を収束試験で比較する。

本章で学ぶこと

- 球対称 Poisson 方程式を、変数変換 $u = r\Phi$ により有限区間上の 1 次元境界値問題として設定できる。
- 二階微分を中心差分で近似し、中心と外側の境界条件を組み込んだ三重対角の連立一次方程式 $A\boldsymbol{u} = \boldsymbol{b}$ を構成できる。
- 前進差分・後退差分・中心差分を Taylor 展開から導き、数値微分に適用できる。
- 解析解との比較と格子収束試験から、境界値問題の数値解と数値微分の誤差・収束次数を評価できる。
- 連立一次方程式の残差と離散化誤差を区別し、残差が小さくても元の微分方程式に対する誤差が残る理由を説明できる。

2.1 Plummer 球の重力を求める二つの方法

第 1 章と同じ Plummer 密度分布

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2} \tag{2.1}$$

を使う [1]。

密度分布から重力ポテンシャル Φ を求めるには、両者を結ぶ Poisson 方程式

$$\nabla^2 \Phi = 4\pi G\rho \tag{2.2}$$

を、適切な境界条件のもとで解く。球対称系では、得られたポテンシャル $\Phi(r)$ の微分から、重力加速度の動径成分を $g_r(r) = -d\Phi/dr$ と求められる。ここでは動径方向の外向きを正とする。

Plummer 球では、ポテンシャルと重力加速度の解析式が分かっているため、数値計算の結果を直接検証できる。無限遠でポテンシャルが 0 になるように基準を選び、$\Phi(\infty) = 0$ とすると、

$$\Phi_{\mathrm{exact}}(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{2.3}$$

である。この式を微分すると、重力加速度は次のように求められる。

$$g_{r,\mathrm{exact}}(r) = -\frac{d\Phi_{\mathrm{exact}}}{dr} = -\frac{GMr}{(r^2 + a^2)^{3/2}} \tag{2.4}$$

[1] 重力計算法の使い分けは、付録Bの補足「シミュレーション天文学で使う重力計算法」で紹介する。

また、式(1.12)で得た内包質量

$$M(<r) = M\frac{(r/a)^3}{[1+(r/a)^2]^{3/2}} \tag{2.5}$$

からも、球対称性を使って

$$g_r(r) = -\frac{GM(<r)}{r^2} \tag{2.6}$$

と求められる。

したがって Plummer 球では、同じ g_r を

$$\rho \to M(<r) \to g_r, \qquad \rho \to \Phi \to g_r \tag{2.7}$$

という二つの方法で計算でき、それぞれを解析解 $g_{r,\mathrm{exact}}$ と比較できる。同じ物理量を独立した方法で求めて比較することは、数値計算を検証するうえで有用である。数値積分、Poisson 方程式の求解、数値微分を組み合わせた比較は、Notebook 2 の総合演習で行う。

Notebook 2 ｜ Plummer 球

Notebook では $G = M = a = 1$ の無次元単位を用いる。密度と内包質量には式(2.1)、(2.5)を、ポテンシャルと重力加速度には式(2.3)、(2.4)を使い、検証用の関数として定義する。

対応セル（計算部分）

```
# 無次元単位を設定する
G = 1.0
M = 1.0
a = 1.0

# Plummer 球の密度と解析解
def rho_plummer(r, M, a):
    return 3.0 * M / (4.0 * np.pi * a**3) * (1.0 + (r / a)**2)**(-2.5)

def mass_plummer_exact(r, M, a):
    s = r / a
    return M * s**3 / (1.0 + s**2)**1.5

def phi_plummer_exact(r, G, M, a):
    return -G * M / np.sqrt(r**2 + a**2)

def gravity_plummer_exact(r, G, M, a):
    return -G * M * r / (r**2 + a**2)**1.5
```

2.2 球対称 Poisson 方程式の 1 次元化

前節で示した Poisson 方程式を球対称系で考える。球対称なら、密度とポテンシャルは半径 r だけの関数となり、Poisson 方程式は

$$\frac{1}{r^2}\frac{d}{dr}\left(r^2\frac{d\Phi}{dr}\right) = 4\pi G\rho(r) \tag{2.8}$$

と書ける。したがって、3 次元の偏微分方程式は、半径 r だけを独立変数とする二階の常微分方程式へ帰着する。

式(2.8)の左辺を展開すると

$$\frac{d^2\Phi}{dr^2} + \frac{2}{r}\frac{d\Phi}{dr} = 4\pi G\rho(r) \tag{2.9}$$

である。この形でも離散化できるが、$r = 0$ では $2/r$ の係数をそのまま扱えない。そこで、球対称 Poisson 方程式をより単純な形にするため、

$$u(r) \equiv r\Phi(r) \tag{2.10}$$

を導入する。以下、プライムは r に関する微分を表す。$r > 0$ では $\Phi = u/r$ なので、

$$\frac{d\Phi}{dr} = \frac{u'}{r} - \frac{u}{r^2}, \tag{2.11}$$

$$r^2\frac{d\Phi}{dr} = ru' - u, \tag{2.12}$$

$$\frac{d}{dr}\left(r^2\frac{d\Phi}{dr}\right) = ru'' \tag{2.13}$$

となる。この結果を式(2.8)に代入し、$r > 0$ で両辺に r を掛けると、

$$\frac{d^2u}{dr^2} = 4\pi G r\rho(r) \tag{2.14}$$

を得る。さらに、右辺を

$$s(r) \equiv 4\pi G r\rho(r) \tag{2.15}$$

と書けば、

$$\boxed{u''(r) = s(r)} \tag{2.16}$$

となる。変数変換 $u = r\Phi$ によって、球対称座標に由来する $1/r$ を含む係数が消え、左辺は単純な二階微分だけになった。このため、次節では u'' を中心差分で近似することで、内部格子点の方程式を直接構成できる。

式(2.16)は二階の常微分方程式であり、解を定めるには二つの独立な条件が必要である。本章では $0 \le r \le r_{\max}$ を計算領域とし、その両端で u の値を指定する。微分方程式に加えて、境界で与えた条件を満たす解を求めることになる。

境界値問題

微分方程式と、計算領域の境界で与える条件を組み合わせて解を求める問題を **境界値問題（boundary-value problem）**と呼ぶ。

中心と外側で具体的に与える境界値は、§2.4で説明する。

2.3 Poisson 方程式の中心差分による離散化

式(2.16)では、未知量は半径 r の連続関数 $u(r)$ である。数値計算では、計算領域を有限個の格子点に分け、各格子点での関数値を未知数として求める。そのため、微分方程式に含まれる微分を、格子点上の有限個の関数値を用いて近似する。

この置き換えによって、連続な微分方程式は、有限個の未知数に対する代数方程式へ近似的に置き換えられる。本章で扱う Poisson 方程式は線形なので、内部格子点で得られる差分方程式と境界条件をまとめると、連立一次方程式として数値解を求めることができる。このように、連続な問題を有限個の未知数からなる問題へ置き換えることを **離散化（discretization）**という。

ここでは、二階微分 u'' を中心差分で近似して、式(2.16)を離散化する。計算領域 $0 \le r \le r_{\max}$ を N 個の区間に等分し、

$$r_i = i\Delta r, \qquad \Delta r = \frac{r_{\max}}{N}, \qquad i = 0, 1, \ldots, N \tag{2.17}$$

という等間隔格子を導入する。格子点は $N + 1$ 個である。滑らかな関数 $u(r)$ の格子点での値を $u_i = u(r_i)$ とし、$u'_i = u'(r_i)$ などと書く。

内部格子点 r_i のまわりで $u_{i\pm1} = u(r_i \pm \Delta r)$ を Taylor 展開すると、

$$u_{i+1} = u_i + \Delta r\, u_i' + \frac{(\Delta r)^2}{2} u_i'' + \frac{(\Delta r)^3}{6} u_i''' + O((\Delta r)^4), \tag{2.18}$$

$$u_{i-1} = u_i - \Delta r\, u_i' + \frac{(\Delta r)^2}{2} u_i'' - \frac{(\Delta r)^3}{6} u_i''' + O((\Delta r)^4) \tag{2.19}$$

である。2 式を足して $2u_i$ を引くと、表示した 1 次と 3 次の項が消え、

$$u_{i+1} - 2u_i + u_{i-1} = (\Delta r)^2 u_i'' + O((\Delta r)^4) \tag{2.20}$$

となる。両辺を $(\Delta r)^2$ で割ると、

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta r)^2} = u_i'' + O((\Delta r)^2) \tag{2.21}$$

を得る。したがって、二階微分は

$$\boxed{u_i'' \simeq \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta r)^2}} \tag{2.22}$$

と近似できる。これが二階微分の 3 点中心差分であり、この近似の打ち切り誤差は $O((\Delta r)^2)$ である。

次に、この中心差分を式(2.16)へ代入し、各内部格子点で数値近似が満たす方程式を作る。ここまでの Taylor 展開では $u_i = u(r_i)$ を厳密な格子点値として用いたが、以下では記号を簡単にするため、格子点 r_i における数値近似も u_i と書く。また、既知の右辺を $s_i = s(r_i)$ とする。各内部格子点では

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta r)^2} = s_i \tag{2.23}$$

となる。後で係数行列の対角成分を正に書くため、両辺に $-(\Delta r)^2$ を掛けると、

$$\boxed{-u_{i-1} + 2u_i - u_{i+1} = -(\Delta r)^2 s_i} \tag{2.24}$$

を得る。

ここで、差分公式がどの格子点を使っているかを整理しておく。

差分ステンシル

ある格子点の差分式を構成するために用いる格子点の組を、**差分ステンシル（finite-difference stencil）**と呼ぶ。本節の二階中心差分では r_{i-1}, r_i, r_{i+1} の 3 点を使うため、式(2.24)は 3 点ステンシルである（図2.1）。

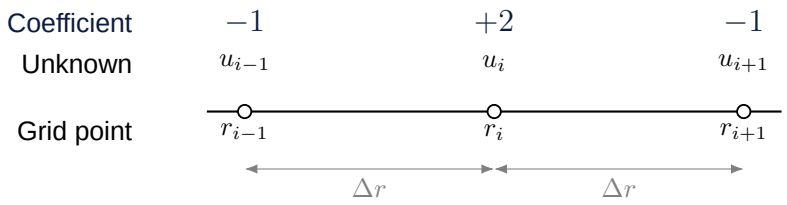


図 2.1　式(2.24)の 3 点ステンシル。上段は左辺における u_{i-1}, u_i, u_{i+1} の係数である。二階微分の中心差分そのものの係数 $1, -2, 1$ ではなく、式(2.24)で用いる係数 $-1, 2, -1$ を示している。

2.4 中心と外側の境界条件

前節で得た内部格子点の差分式に加えて、計算領域の両端で境界条件を与える。本章では $0 \le r \le r_{\max}$ で式(2.16)を解くため、中心 $r = 0$ と外側 $r = r_{\max}$ の値を指定する。

中心境界条件と $\Phi(0)$ の復元

中心でポテンシャル $\Phi(0)$ が有限なら、$u = r\Phi$ なので

$$u(0) = 0 \tag{2.25}$$

である。したがって、u について Poisson 方程式を解くときの中心境界条件は

$$u_0 = 0 \tag{2.26}$$

と与えればよい。

一方、Poisson 方程式を解いた後で u からポテンシャル Φ を復元するときには、中心だけ別の扱いが必要になる。$r_i > 0$ では

$$\Phi_i = \frac{u_i}{r_i} \tag{2.27}$$

とすればよいが、中心では $u_0 = r_0 = 0$ なので、この式は $0/0$ となり直接使えない。これは u の中心境界条件に問題があるのではなく、変数変換 $u = r\Phi$ から Φ に戻すときに生じるものである。

中心でポテンシャルが滑らかなら

$$\Phi(0) = \lim_{r\to 0} \frac{u(r)}{r} = u'(0) \tag{2.28}$$

である。この滑らかさと球対称性による中心まわりの対称性を用いると、中心値 $\Phi_0 = \Phi(0)$ は中心近傍の 2 点から

$$\boxed{\Phi_0 \simeq \frac{4\Phi_1 - \Phi_2}{3}} \tag{2.29}$$

と近似できる（「補足｜中心ポテンシャルの外挿公式の導出」を参照）。実際に Poisson 方程式で求める未知数は u なので、$\Phi_1 = u_1/\Delta r$、$\Phi_2 = u_2/(2\Delta r)$ を代入すると

$$\boxed{\Phi_0 \simeq \frac{8u_1 - u_2}{6\Delta r}} \tag{2.30}$$

となる。数値計算では、この式を用いて中心ポテンシャルを復元する。

補足｜中心ポテンシャルの外挿公式の導出

中心を通る任意の直線上に、中心を $\ell = 0$ とする符号付き座標 ℓ を取る。球対称性から、中心から同じ距離にある ℓ と $-\ell$ ではポテンシャルが等しいので、

$$\Phi(\ell) = \Phi(-\ell) \tag{2.31}$$

が成り立つ。中心で Φ が滑らかなら Φ は ℓ の偶関数であり、中心まわりの Taylor 展開には偶数次の項だけが現れる。半径 $r = |\ell|$ を用いると

$$\Phi(r) = \Phi_0 + c_2 r^2 + c_4 r^4 + O(r^6) \tag{2.32}$$

と書ける。

$r_1 = \Delta r$、$r_2 = 2\Delta r$ とすると

$$\Phi_1 = \Phi_0 + c_2(\Delta r)^2 + c_4(\Delta r)^4 + O((\Delta r)^6), \tag{2.33}$$

$$\Phi_2 = \Phi_0 + 4c_2(\Delta r)^2 + 16c_4(\Delta r)^4 + O((\Delta r)^6). \tag{2.34}$$

ここで $4\Phi_1 - \Phi_2$ を作ると $(\Delta r)^2$ の項が消え、

$$\frac{4\Phi_1 - \Phi_2}{3} = \Phi_0 - 4c_4(\Delta r)^4 + O((\Delta r)^6) \tag{2.35}$$

となる。したがって、

$$\Phi_0 = \frac{4\Phi_1 - \Phi_2}{3} + O((\Delta r)^4) \tag{2.36}$$

を得る。さらに $\Phi_1 = u_1/\Delta r$、$\Phi_2 = u_2/(2\Delta r)$ を代入すると、式(2.30)が得られる。

ただし、この $O((\Delta r)^4)$ は、Φ_1, Φ_2 が厳密に分かっている場合の外挿誤差である。実際には、これらも有限差分法で求めた近似値であり、誤差を含むため、中心ポテンシャルが 4 次精度で求まるとは限らない。

外側境界条件

もう一つの境界条件は、計算領域の外側 $r = r_{\max}$ に与える。まず有限差分法そのものの精度を検証するため、本章の Plummer 球では解析解を用いて

$$u_N = r_{\max}\Phi_{\text{exact}}(r_{\max}) \tag{2.37}$$

とする。これにより、外側境界の近似による誤差を入れずに、主として有限差分による離散化誤差を調べることができる。

解析解が分からなくても、全質量 M が有限な孤立系では、十分遠方で質量分布を点質量として近似し、

$$\Phi(r) \simeq -\frac{GM}{r}, \qquad u(r) \simeq -GM \tag{2.38}$$

を外側境界条件として用いることができる。ただし、本章の Plummer 球は $r_{\max}$ より外側にも広がっているため、有限の $r_{\max}$ でこの近似を境界値に用いると、解析値との差が生じる。この誤差は有限差分の離散化誤差とは別のものであり、$r_{\max}$ を十分大きく取るか、より適切な外側境界条件を用いることで小さくする必要がある。この影響は Notebook 2 の発展課題「境界条件による誤差」で調べる。

本章では、有限の計算領域で Poisson 方程式を解くために、外側境界でポテンシャルの値を与えた。自己重力を含む数値シミュレーションでも、密度分布から重力ポテンシャルを有限の計算領域上で求める場合には、外側境界条件を指定する必要がある[2]。

補足｜孤立系の境界条件と James 法

本章では Plummer 球の解析解が分かるため、外側境界のポテンシャルを直接与えられる。一方、一般の非球対称な質量分布では、有限の計算領域の境界上でどのようなポテンシャルを与えるかが問題になる。孤立系では無限遠で $\Phi \to 0$ となるが、有限の距離にある境界で単に $\Phi = 0$ としても、一般には正しい解にならない。

James 法（James 1977）は、孤立系の境界条件を有限の計算領域で扱う方法である [3, 2]。まず、境界上でポテンシャルを 0 と仮定して Poisson 方程式を解く。しかし、この仮定は実際の孤立系の境界条件とは異なる。そこで、境界を 0 としたことで生じる影響を計算して取り除き、孤立系に対応するポテンシャルを求める。

James 法は有限差分法を置き換えるものではなく、Poisson 方程式の数値解法と組み合わせて境界条件を扱う方法である。ただし、計算領域の外側にも質量がある場合は、その寄与を別途考慮する必要がある。

[2] 質量分布が境界より十分内側に収まる場合には、点質量による項を先頭とし、非球対称な分布の効果を高次の項で表す **多重極展開（multipole expansion）**を用いることがある [8]。これは質量配置の近似を改良する方法であり、境界より外側の質量を無視したことによる誤差とは区別する。

2.5 連立一次方程式の組み立て

式(2.24)には、格子点 r_i における u_i だけでなく、隣の格子点の値 u_{i-1} と u_{i+1} も含まれる。したがって、各 u_i を独立に求めることはできず、すべての内部格子点の差分式をまとめて連立一次方程式として解く必要がある。

まず、差分式がどのような連立一次方程式になるかを見るため、r_0, r_1, r_2, r_3, r_4 の 5 個の格子点だけを持つ簡単な例を考える。両端の r_0, r_4 は境界点であり、r_1, r_2, r_3 は内部格子点である。この例では、左境界に隣接する r_1、両隣が内部点である r_2、右境界に隣接する r_3 の 3 種類の差分式を確認できる。

この例では、両端の u_0 と u_4 は境界条件によって与えられる。したがって、求める未知数は内部 3 点の

$$u_1, \quad u_2, \quad u_3 \tag{2.39}$$

である。式(2.24)を r_1, r_2, r_3 にそれぞれ適用すると、

$$2u_1 - u_2 = -(\Delta r)^2 s_1 + u_0, \tag{2.40}$$

$$-u_1 + 2u_2 - u_3 = -(\Delta r)^2 s_2, \tag{2.41}$$

$$-u_2 + 2u_3 = -(\Delta r)^2 s_3 + u_4. \tag{2.42}$$

これら 3 本の式をまとめると、

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} -(\Delta r)^2 s_1 + u_0 \\ -(\Delta r)^2 s_2 \\ -(\Delta r)^2 s_3 + u_4 \end{pmatrix} \tag{2.43}$$

となる。

ここで、左端の行列を係数行列 A、内部格子点の未知数を並べた列ベクトルを $\boldsymbol{u}$、既知の量をまとめた右辺の列ベクトルを $\boldsymbol{b}$ と書くと、式(2.43)は

$$A\boldsymbol{u} = \boldsymbol{b} \tag{2.44}$$

と簡潔に表せる。一般の N では、

$$\boldsymbol{u} = \begin{pmatrix} u_1 & u_2 & \cdots & u_{N-1} \end{pmatrix}^{\mathsf{T}} \tag{2.45}$$

が求める未知数ベクトルである。上付きの T は転置を表し、横に並べた成分を列ベクトルにしている。係数行列 A の非零成分は、主対角とその上下の対角にだけ現れる。このような行列を **三重対角行列（tridiagonal matrix）**と呼ぶ。

Listing 2.1 に、一般の N に対して係数行列 A と右辺ベクトル $\boldsymbol{b}$ を組み立てる部分を示す。

Listing 2.1　三重対角行列と境界値の組み込み

```
# 内部格子点の値 u_1, ..., u_{N-1} を未知数とする
n = N - 1
A = np.zeros((n, n))

# 三重対角行列の係数 (-1, 2, -1) を入れる
np.fill_diagonal(A, 2.0)
np.fill_diagonal(A[1:], -1.0)
np.fill_diagonal(A[:, 1:], -1.0)

# 内部格子点に対応する右辺ベクトルを作る
b = -(dr**2) * source[1:N].copy()

# 既知の境界値 u_0, u_N を右辺へ組み込む
b[0] += u0
b[-1] += uN
```

一般の格子では、未知数は $u_1, \ldots, u_{N-1}$ の $N-1$ 個である。コード中の `n = N - 1` はこの未知数の数を表す。また、`dr` は格子幅 Δr、`source[1:N]` は内部格子点の $s_1, \ldots, s_{N-1}$ を取り出している。係数行列 A の主対角の 2 と上下の副対角の -1 は、式(2.24)の左辺の係数に対応する。

5 格子点の例と同様に、一般の N 区間でも、両端の境界値 u_0 と u_N は未知数に含めず、右辺に組み込む。コード中の `b[0] += u0` は、最初の内部格子点 r_1 の式に中心境界値 u_0 を加える操作に対応する。同様に、`b[-1] += uN` は、最後の内部格子点 r_{N-1} の式に外側境界値 u_N を加える操作である。本章では $u_0 = 0$ なので最初の加算は値を変えないが、差分式とコードの対応を明示するために残している。

2.6 Plummer 球の Poisson 方程式の数値解

ここまで、球対称 Poisson 方程式を有限差分法で解くために、球対称性による 1 次元化と変数変換、中心差分による離散化、境界条件の設定、連立一次方程式の組み立てを順に説明してきた。本節では、これらの手順をまとめて Plummer 球に適用し、重力ポテンシャルを数値的に求める。

計算手順｜球対称 Poisson 方程式を有限差分で解く

1. $r_i = i\Delta r$ の格子を作り、密度 $\rho_i = \rho(r_i)$ と右辺 $s_i = 4\pi G r_i \rho_i$ を計算する。
2. 中心境界 $u_0 = 0$ と外側境界 u_N を与える。
3. 内部格子点の差分式から、係数行列 A と右辺ベクトル $\boldsymbol{b}$ を組み立てる。
4. 連立一次方程式 $A\boldsymbol{u} = \boldsymbol{b}$ を解き、$u_1, \ldots, u_{N-1}$ を求める。
5. 境界値と内部の数値解を合わせて u_i を構成し、$r_i > 0$ では $\Phi_i = u_i / r_i$ とする。中心値 Φ_0 は式(2.30)から復元する。

以下では、この一連の手順を Python で実行する。連立一次方程式の求解には、NumPy の `np.linalg.solve` を使う。[3]

得られた数値解を検証するため、ポテンシャルを同じ格子点で評価した解析解と比較する。あわせて、連立一次方程式をどの程度満たしているかを残差で確認する。

[3] 一般の密行列には Gauss の消去法や LU 分解が用いられる。本章のような三重対角行列では、構造を利用した Thomas 法によって計算量を $O(N)$ にできる。これらの解法の基本的な考え方は付録D、詳しい説明は幸谷第 7 章 [5] を参照。

残差

連立一次方程式 $A\boldsymbol{u} = \boldsymbol{b}$ に数値解を代入したときの左右のずれ

$$\boldsymbol{r}_{\mathrm{res}} = A\boldsymbol{u} - \boldsymbol{b} \tag{2.46}$$

を**残差（residual）**という。残差は、数値解が離散化した連立一次方程式をどの程度満たしているかを表す。残差が小さくても、解析解との誤差が小さいとは限らない。

Notebook 2 ｜ Poisson 方程式の数値解

Notebook では、一連の計算手順を関数 `solve_spherical_poisson_plummer` にまとめる。関数の内部では、格子と右辺の作成、境界条件の設定、連立一次方程式の組み立てと求解、u から Φ への復元を順に行う。外側境界は引数 `boundary` で選び、本節では式(2.37)の解析的な境界条件を用いる。

対応セル（関数定義）

```
def solve_spherical_poisson_plummer(N, r_max, G, M, a, boundary="exact"):
    # 1. 格子を作り、密度と Poisson 方程式の右辺を計算する
    r = np.linspace(0.0, r_max, N + 1)
    dr = r[1] - r[0]
    rho = rho_plummer(r, M, a)
    source = 4.0 * np.pi * G * r * rho

    # 2. 中心境界と外側境界を与える
    u0 = 0.0
    if boundary == "exact":
        uN = r_max * phi_plummer_exact(r_max, G, M, a)
    elif boundary == "point_mass":
        uN = -G * M
    else:
        raise ValueError("boundary は 'exact' または 'point_mass' を指定")

    # 3. 係数行列 A と右辺ベクトル b を組み立てる
    n = N - 1
    A = np.zeros((n, n))
    np.fill_diagonal(A, 2.0)
    np.fill_diagonal(A[1:], -1.0)
    np.fill_diagonal(A[:, 1:], -1.0)

    b = -(dr**2) * source[1:N].copy()
    b[0] += u0
    b[-1] += uN

    # 4. A u = b を解き、境界値と内部解を合わせる
    u = np.empty(N + 1)
    u[0] = u0
    u[1:N] = np.linalg.solve(A, b)
    u[N] = uN

    # 5. u = r Phi からポテンシャルを復元する
    phi = np.empty(N + 1)
    phi[1:] = u[1:] / r[1:]
```

```
    phi[0] = (8.0 * u[1] - u[2]) / (6.0 * dr)

    return r, rho, u, phi, A, b
```

この関数が返す配列 u は、両端の境界値を含む $N+1$ 個の値からなる。連立一次方程式の未知数ベクトル $\boldsymbol{u}$ に対応するのは、内部の $N-1$ 個を取り出した u[1:N] である。この関数を $N=200$、$r_{\max}=10a$ で実行すると、$\Delta r = 0.05a$ となる。以下では、数値ポテンシャルの最大絶対誤差と、残差の各成分の絶対値の最大値を計算する。

対応セル（計算部分）

```
# 計算領域と区間数を設定する
N = 200
r_max = 10.0 * a

# Poisson 方程式を解いて数値ポテンシャルを得る
r, rho, u, phi_num, A, b = solve_spherical_poisson_plummer(
    N=N, r_max=r_max, G=G, M=M, a=a, boundary="exact"
)

# 同じ格子点で解析解を計算する
phi_exact = phi_plummer_exact(r, G, M, a)

# 離散方程式 A u = b の残差を確認する
residual = A @ u[1:N] - b

# ポテンシャルの最大誤差と最大残差
phi_error = np.max(np.abs(phi_num - phi_exact))
residual_max = np.max(np.abs(residual))
```

図2.2では、数値ポテンシャルと解析解はほぼ重なり、解析解をよく再現できていることが確認できる。

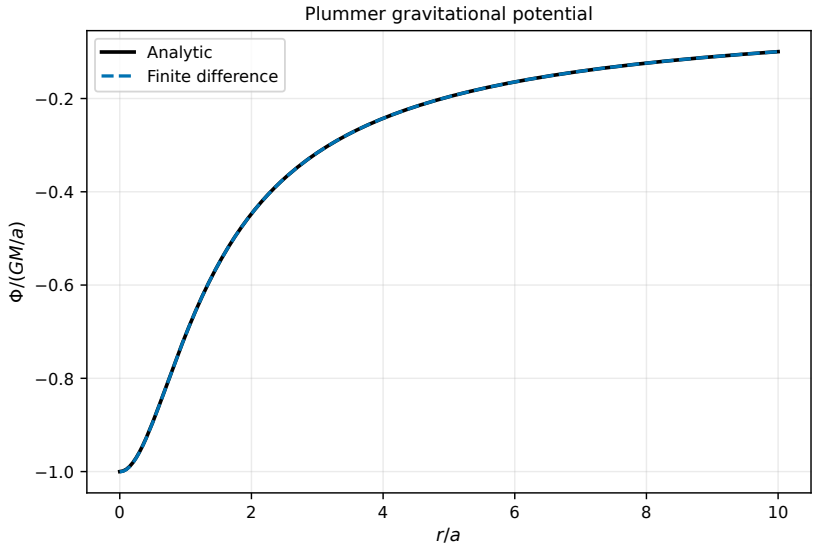


図 2.2　Plummer ポテンシャルの有限差分解と解析解。

一方、差を拡大した図2.3では、有限差分による離散化誤差が確認できる。今回の $N = 200$ では、ポテンシャルの最大絶対誤差は

$$\max_{0 \le i \le N} |\Phi_{\rm num}(r_i) - \Phi_{\rm exact}(r_i)| = 6.222636 \times 10^{-4} \, \frac{GM}{a} \tag{2.47}$$

である。同じ計算を $G = M = a = 1$ の無次元単位で行ったとき、連立一次方程式の最大残差は $\max_i |(A\boldsymbol{u} - \boldsymbol{b})_i| \simeq 3.85 \times 10^{-16}$ である。残差の大きさは方程式の規格化に依存するが、今回用いた形では、左右のずれは浮動小数点演算の精度に近い大きさまで小さくなっている。つまり、数値解は離散方程式 $A\boldsymbol{u} = \boldsymbol{b}$ を高い精度で満たしている。それにもかかわらず解析解との差が残るのは、$A\boldsymbol{u} = \boldsymbol{b}$ が Poisson 方程式そのものではなく、二階微分を中心差分で近似して得られた離散方程式だからである。このため、連立一次方程式を高い精度で解いても、元の連続な Poisson 方程式に対する離散化誤差は残る。

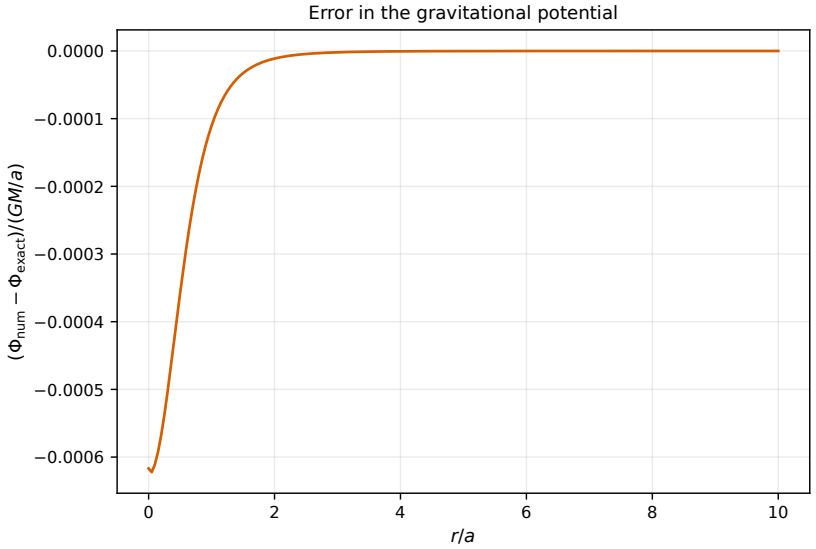


図 2.3 有限差分で得た Plummer ポテンシャルの誤差。縦軸は符号付きの差 $(\Phi_{\rm num} - \Phi_{\rm exact})/(GM/a)$ であり、絶対値ではない。外側境界には解析解を与えている。

2.7 Poisson 方程式の格子収束

前節では、$N = 200$ の一つの格子について数値ポテンシャルを解析解と比較し、有限差分による離散化誤差が残ることを確認した。しかし、一つの格子で誤差を測るだけでは、格子を細かくしたときに数値解がどのように解析解へ近づくかは分からない。そこで本節では、格子幅 Δr を変えて同じ計算を繰り返し、ポテンシャルの誤差がどのように減少するかを調べる。二階中心差分そのものの近似誤差は $O((\Delta r)^2)$ なので、ポテンシャル全体の誤差についても 2 次収束が得られるかを格子収束試験によって確認する。

計算領域は $r_{\rm max} = 10a$ に固定し、外側境界には前節と同じく解析値を与える。そのうえで区間数を

$$N = 50, 100, 200, 400, 800 \tag{2.48}$$

と変化させる。各格子では、ポテンシャルの最大絶対誤差を GM/a で無次元化し、

$$\epsilon_\Phi \equiv \max_{0 \le i \le N} \frac{|\Phi_{\rm num}(r_i) - \Phi_{\rm exact}(r_i)|}{GM/a} \tag{2.49}$$

として評価する。

Notebook 2 ｜ Poisson 方程式の格子収束

Notebook では 5 種類の格子で ϵ_Φ を測り、両対数直線の傾きから収束次数を求める。

対応セル（計算部分）

```
# 1 つの格子でポテンシャルの最大誤差を測る
def poisson_error(N):
    r, _, _, phi_num, _, _ = solve_spherical_poisson_plummer(
        N=N, r_max=10.0 * a, G=G, M=M, a=a
    )
    phi_exact = phi_plummer_exact(r, G, M, a)
    dr = r[1] - r[0]
    err = np.max(np.abs(phi_num - phi_exact)) / (G * M / a)
    return dr, err

# 区間数を増やし、格子幅と誤差を記録する
N_values = np.array([50, 100, 200, 400, 800])
dr_values = []
phi_errors = []

for N_i in N_values:
    dr_i, err_i = poisson_error(int(N_i))
    dr_values.append(dr_i)
    phi_errors.append(err_i)

dr_values = np.array(dr_values)
phi_errors = np.array(phi_errors)

# log(error)-log(dr) の傾きが収束次数 p
order_phi = np.polyfit(
    np.log(dr_values), np.log(phi_errors), 1
)[0]
```

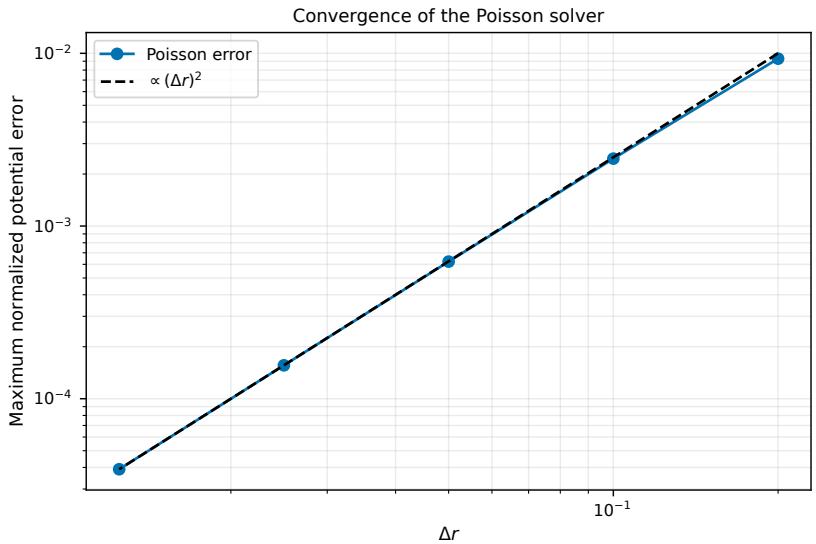


図 2.4　Poisson 方程式ソルバーの格子収束。破線は $\epsilon_\Phi \propto (\Delta r)^2$ を示す。

5 つの格子で得た誤差を両対数上で直線近似すると、$p = 1.977$ を得る。p はほぼ 2 であるため、ポテンシャルの数値解は格子幅 Δr に対して 2 次精度で収束していることが分かる（図2.4）。すなわち、格子幅を半分にすると、誤差はおよそ $1/4$ に減少する。

2.8 数値微分による重力加速度

Poisson 方程式から $\Phi(r)$ が得られたら

$$g_r(r) = -\frac{d\Phi}{dr} \tag{2.50}$$

を数値微分で計算する。ポテンシャルが格子上の値として与えられる場合には、差分して重力加速度を求める方法がある [9]。本節では、この計算を題材に数値微分の基本を学ぶ。以下では前進差分（forward difference）、後退差分（backward difference）、中心差分（central difference）を Taylor 展開から導く[4][5]。以下の Φ_i は、滑らかなポテンシャルを格子点 r_i で評価した値を表す。まず差分公式自体の近似誤差を調べる。

前進差分（forward difference）

r_i のまわりで

$$\Phi_{i+1} = \Phi_i + \Delta r\,\Phi_i' + \frac{(\Delta r)^2}{2}\Phi_i'' + \frac{(\Delta r)^3}{6}\Phi_i''' + O((\Delta r)^4) \tag{2.51}$$

なので

$$\boxed{\Phi_i' \simeq \frac{\Phi_{i+1} - \Phi_i}{\Delta r}} \tag{2.52}$$

である。差分商から厳密な微分値 Φ_i' を引いた差の主要項は $+(\Delta r/2)\Phi_i''$ である。この誤差は Δr に比例するので 1 次精度である。重力加速度に使うと Listing 2.2 のようになる。

Listing 2.2　前進差分

```
def gravity_forward(r, phi):
    dr = r[1] - r[0]
    return r[:-1], -(phi[1:] - phi[:-1]) / dr
```

返す半径が `r[:-1]` なのは、差分商 $(\Phi_{i+1} - \Phi_i)/\Delta r$ を r_i での微分の近似とするためである。

後退差分（backward difference）

同様に

$$\Phi_{i-1} = \Phi_i - \Delta r\,\Phi_i' + \frac{(\Delta r)^2}{2}\Phi_i'' - \frac{(\Delta r)^3}{6}\Phi_i''' + O((\Delta r)^4) \tag{2.53}$$

より

$$\boxed{\Phi_i' \simeq \frac{\Phi_i - \Phi_{i-1}}{\Delta r}} \tag{2.54}$$

である。差分商から厳密な微分値を引いた差の主要項は $-(\Delta r/2)\Phi_i''$ であり、やはり 1 次精度である。

[4] ここでは 3 つの差分公式を紹介するが、内部の格子点では、同じく 2 点の値だけで 2 次精度が得られる中心差分を使うのが基本である。前進・後退差分が必要になるのは、片側にしか格子点がない計算領域の端と、時間方向のように一方向の値しか使えない場合である。後者の例が第 3 章の Euler 法である。端点でも 2 次精度を保ちたい場合は、片側の 3 点を使う公式 $f_0' \simeq (-3f_0 + 4f_1 - f_2)/(2\Delta r)$ がよく使われる。ただし、流体の移流のように、流れの向きに応じて片側差分を選ばないと計算が不安定になる問題もあり、中心差分が常に適しているわけではない。詳しくは藤井・立川 [10]、松本ほか [9] を参照すること。

Listing 2.3　後退差分

```
def gravity_backward(r, phi):
    dr = r[1] - r[0]
    return r[1:], -(phi[1:] - phi[:-1]) / dr
```

前進差分と後退差分のコードは同じ 2 点差を計算している。違いは、その差をどの格子点の微分とみなすかである。後退差分では `r[1:]` を返す。

中心差分（central difference）

前進側と後退側の Taylor 展開を 4 次の項まで含めて引き算すると、偶数次の項が消えて

$$\Phi_{i+1} - \Phi_{i-1} = 2\Delta r\, \Phi_i' + \frac{(\Delta r)^3}{3}\Phi_i''' + O((\Delta r)^5) \tag{2.55}$$

である。したがって

$$\boxed{\Phi_i' \simeq \frac{\Phi_{i+1} - \Phi_{i-1}}{2\Delta r}} \tag{2.56}$$

を得る。差分商から厳密な微分値を引いた差の主要項は $(\Delta r)^2\Phi_i'''/6$ なので、中心差分の誤差は $O((\Delta r)^2)$ であり、2 次精度である。これは、同じ点で評価した前進差分と後退差分を平均すると、主要な 1 次誤差が互いに相殺されることに対応している。

Listing 2.4　中心差分

```
def gravity_central(r, phi):
    dr = r[1] - r[0]
    return r[1:-1], -(phi[2:] - phi[:-2]) / (2.0 * dr)
```

この 3 点中心差分は左右の格子点を必要とするため、計算領域の両端にはそのまま適用できない。したがって、この関数が返す半径配列は `r[1:-1]` である。

Plummer 球への適用

以上の 3 つの差分公式を、Plummer 球のポテンシャルに適用する。Poisson 方程式の数値解 $\Phi_{\rm num}$ を微分すると、ポテンシャルに含まれる離散化誤差と数値微分そのものの誤差が重力加速度に同時に含まれる。そこで、差分公式そのものの性質を調べるため、解析ポテンシャル $\Phi_{\rm exact}$ を格子点で評価して用いる。

$N = 200$、$r_{\rm max} = 10a$ の格子上で、同じ解析ポテンシャルに前進差分・後退差分・中心差分を適用する。各関数が返す半径配列は差分公式によって異なるため、それぞれの評価位置における解析解と比較する。

Notebook 2 ｜数値微分による重力加速度

同じ格子上で評価した解析ポテンシャルに、前進差分・後退差分・中心差分を適用する。

対応セル（計算部分）

```
# 一つの格子で解析ポテンシャルを評価する
N = 200
r_max = 10.0 * a
r_diff = np.linspace(0.0, r_max, N + 1)
phi_for_diff = phi_plummer_exact(r_diff, G, M, a)

# 3 つの差分公式で重力加速度を求める
r_f, g_forward = gravity_forward(r_diff, phi_for_diff)
```

```
r_b, g_backward = gravity_backward(r_diff, phi_for_diff)
r_c, g_central = gravity_central(r_diff, phi_for_diff)
```

図2.5に、この計算で得られた重力加速度を解析解とともに示す。この格子では、3 つの差分公式はいずれも解析解に近い値を与える。

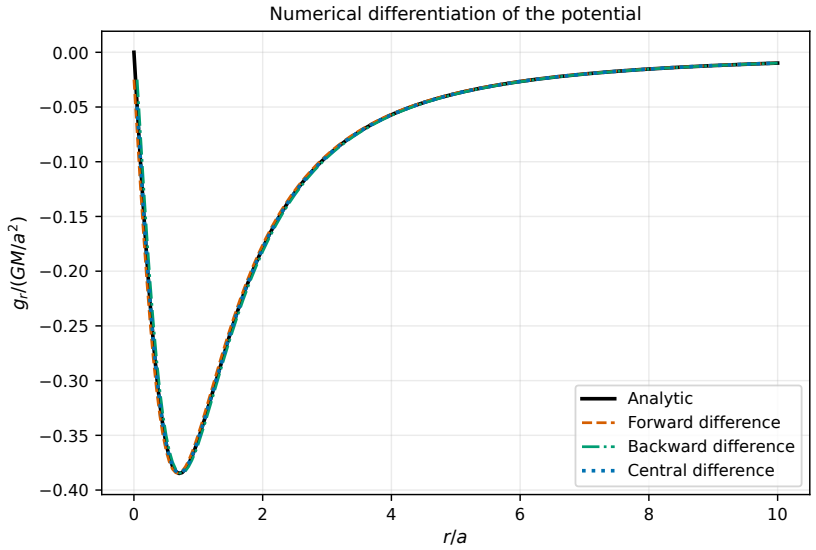


図 2.5 $N = 200$、$r_{\max} = 10a$ の格子上で、解析的な Plummer ポテンシャルを前進・後退・中心差分して得た重力加速度。

一つの格子で結果を比較するだけでは、格子を細かくしたときに誤差がどの速さで減少するかは分からない。この点を次節の格子収束試験で調べる。

2.9 前進・後退・中心差分の収束比較

前節では、一つの格子上で解析ポテンシャルを数値微分し、3 つの差分公式から得られる重力加速度を比較した。本節では格子幅を変えて同じ計算を繰り返し、前進差分・後退差分の 1 次収束と中心差分の 2 次収束を数値的に確認する。

数値微分そのものの収束性を調べるため、前節と同じく解析ポテンシャル $\Phi_{\rm exact}$ を入力として用いる。計算領域を $r_{\max} = 10a$ に固定し、区間数を

$$N = 50, 100, 200, 400, 800 \tag{2.57}$$

と変えて、5 種類の格子で誤差を測る。

格子を変えると、各差分公式が重力加速度を返す格子点の位置も変わる。そこで、すべての格子で同じ物理的な半径範囲を比較するため、

$$0.2a \le r \le 5a \tag{2.58}$$

に含まれる評価点だけを用いる。各差分公式について、規格化した最大絶対誤差を

$$\epsilon_{g,\rm fixed} \equiv \max_{\substack{i \\ 0.2a \le r_i \le 5a}} \frac{|g_{r,\rm num}(r_i) - g_{r,\rm exact}(r_i)|}{GM/a^2} \tag{2.59}$$

と定義する。ここで $g_{r,\rm num}$ は各差分公式から得た重力加速度である。分母の GM/a^2 は一定の基準量なので、$\epsilon_{g,\rm fixed}$ は規格化した絶対誤差であり、各点での相対誤差ではない。

Notebook 2 ｜数値微分の格子収束

各格子について、解析ポテンシャルを 3 つの差分公式で微分し、式(2.59)の最大誤差を測る。この処理を補助関数 `differentiation_errors` にまとめる。

対応セル（補助関数）

```
def differentiation_errors(N):
    r_max = 10.0 * a
    r = np.linspace(0.0, r_max, N + 1)
    dr = r[1] - r[0]
    phi = phi_plummer_exact(r, G, M, a)

    r_f, g_f = gravity_forward(r, phi)
    r_b, g_b = gravity_backward(r, phi)
    r_c, g_c = gravity_central(r, phi)

    def max_error(r_eval, g_num):
        mask = (r_eval >= 0.2 * a) & (r_eval <= 5.0 * a)
        g_exact = gravity_plummer_exact(r_eval, G, M, a)
        return np.max(np.abs(g_num[mask] - g_exact[mask])) / (G * M / a**2)

    return dr, max_error(r_f, g_f), max_error(r_b, g_b), max_error(r_c, g_c)
```

対応セル（収束次数の測定）

```
# 5 種類の格子で 3 手法の最大誤差を測る
N_diff = np.array([50, 100, 200, 400, 800])
results = np.array([differentiation_errors(int(N_i)) for N_i in N_diff])

dr_diff = results[:, 0]
err_forward = results[:, 1]
err_backward = results[:, 2]
err_central = results[:, 3]

# log(error)-log(dr) の傾きから収束次数を求める
order_forward = np.polyfit(np.log(dr_diff), np.log(err_forward), 1)[0]
order_backward = np.polyfit(np.log(dr_diff), np.log(err_backward), 1)[0]
order_central = np.polyfit(np.log(dr_diff), np.log(err_central), 1)[0]
```

5 つの格子で得た誤差を両対数上で直線近似すると、

$$p_{\mathrm{f}} = 0.952, \qquad p_{\mathrm{b}} = 1.030, \qquad p_{\mathrm{c}} = 1.988 \tag{2.60}$$

を得る。添字 $\mathrm{f}, \mathrm{b}, \mathrm{c}$ は、それぞれ前進・後退・中心差分を表す。図2.6に示すように、前進差分と後退差分はほぼ 1 次精度、中心差分はほぼ 2 次精度で収束しており、前節で求めた各差分公式の誤差評価と一致する。前進差分と後退差分の測定値が厳密に 1 にならないのは、有限の格子幅では主要な 1 次誤差だけでなく、より高次の誤差項も寄与するためである。格子をさらに細かくすると、それらの影響は小さくなり、測定される収束次数は 1 に近づく。

以上で、密度分布から Poisson 方程式を解いてポテンシャルを求め、その数値微分から重力加速度を計算するための手順がそろった。ここまでの収束試験では、Poisson 方程式の求解と数値微分を分けて検証した。Notebook 2 の総合演習では、数値ポテンシャル Φ_{num} を中心差分

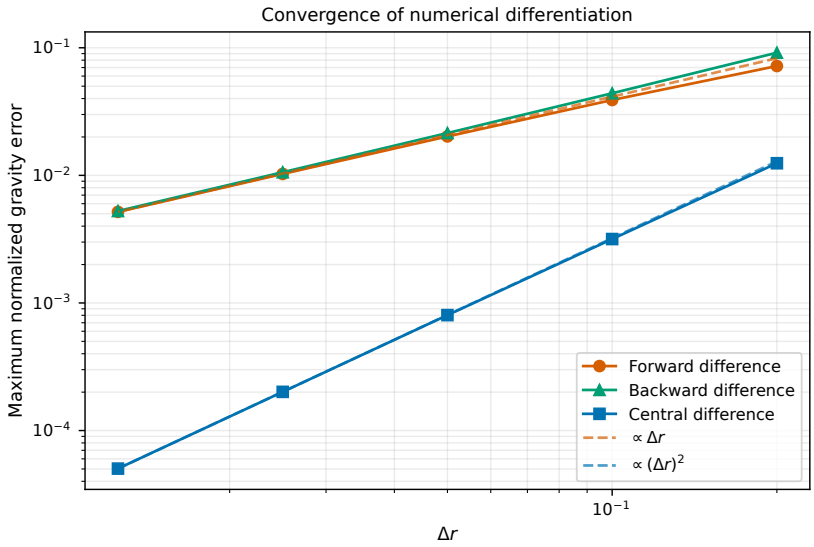


図 2.6 解析的な Plummer ポテンシャルを数値微分したときの格子収束。$0.2a \le r \le 5a$ で測った規格化最大絶対誤差を示す。前進・後退差分はほぼ 1 次、中心差分はほぼ 2 次で収束する。

して重力加速度を求め、解析解および第 1 章の累積台形則による内包質量から得た結果と比較する。

補足｜偏微分方程式の型と境界条件

本章で扱った Poisson 方程式は、**楕円型（elliptic）**偏微分方程式の代表例である。この境界値問題では、内部の方程式と境界条件を合わせ、領域全体で互いに結合した未知数を求める。本章では中心と外側に条件を与え、有限差分で得た連立一次方程式を解いた。付録Dの反復法を使う場合も、領域全体の方程式を満たす解へ近づける点は同じである。

熱伝導や拡散を記述する**放物型（parabolic）**と、波や移流を記述する**双曲型（hyperbolic）**の方程式も、宇宙流体計算で現れる。有限領域でこれらの時間発展を求める問題は、初期条件と境界条件を組み合わせた**初期境界値問題（initial-boundary value problem）**として扱う。正の拡散係数を持つ拡散方程式では、初期分布に加え、空間領域の境界全体で各時刻の条件を定める。一方、双曲型では情報が有限の速さで伝わる経路を**特性曲線**と呼び、境界で与える独立な条件の数は、領域へ流入する特性の数で決まる。例えば一定方向に進む移流では、流入側に値を与え、流出側の値は内部から伝わる解によって決まる。この場合、流出側に独立な値を追加して固定してはいけない。ただし、流出境界近くの数値更新に必要な値を、内部から外側へ延長する処理は必要になり得る。

本章では既知の境界値を右辺へ移したが、別の実装方法として、計算領域の外側に仮想的な値を置き、境界近傍でも内部と同じ形の更新式を使う方法がある。格子法の流体計算で用いる**ゴーストセル（ghost cell）**はこの考え方に対応し、付録Bの対称軸の仮想点とも共通する。例えば、周期境界では反対側の値をコピーし、反射境界では境界をはさんで対称な位置の値をコピーして、境界に垂直な速度成分の符号を反転する。流出境界では内部の値を外挿する方法があるが、すべての波を無反射で通す保証はない。必要な物理的条件と、その条件を実装する処理を分けて考えることが重要である。境界条件とゴーストセルの扱いは、藤井・立川 [10]、松本ほか [9] を参照すること。

補足｜時間発展問題の時間積分と連立一次方程式

時間発展を含む偏微分方程式でも、空間方向を有限差分などで離散化すると、各格子点の値についての連立常微分方程式が得られる。これを時間方向へ進める方法には、陽的な方法と陰的な方法がある。

既知の状態と、すでに計算した途中の状態から、順に更新量を求める方法を**陽的な時間積分（explicit time integration）**という。第 3 章で扱う Euler 法と陽的 Runge–Kutta 法では、時間更新の未知量について連立方程式を解く必要はない。一方、空間離散化と組み合わせた時間刻みの制約を満たす必要がある。例えば、一次元の圧縮性流体で全セルに共通の時間刻みを使う場合には、

$$\Delta t \le C_{\mathrm{CFL}} \min_i \frac{\Delta x_i}{|v_i| + c_{\mathrm{s},i}}$$

という **CFL 条件（Courant–Friedrichs–Lewy condition）**が重要になる。Δx_i はセル幅、v_i は流速、$c_{\mathrm{s},i}$ は音速であり、最も厳しいセルの条件を使う。C_{CFL} の許容値は数値手法に依存し、この条件を満たすだけで任意の空間離散化が安定になるわけではない。一定の拡散係数を持つ拡散方程式の陽解法では、許される時間刻みの上限は典型的に $\Delta t_{\max} \propto (\Delta x)^2$ となる。高次精度の公式を選ぶことと、安定に時間発展を計算することは別である。

次の時刻や計算途中の未知の値を含む式を立て、その式を解いて更新する方法を**陰的な時間積分（implicit time integration）**という。線形な問題では連立一次方程式に帰着するが、非線形な問題では非線形方程式を解く必要がある。例として、本章の $u = r\Phi$ とは区別して拡散する量を $w(x,t)$ と書き、一定の拡散係数 $D > 0$ を持つ方程式

$$\frac{\partial w}{\partial t} = D \frac{\partial^2 w}{\partial x^2}$$

を考える。一様格子上の値を $w_i^n \simeq w(x_i, t_n)$ とし、二階微分を中心差分、時間方向を後退 Euler 法で離散化すると、

$$\frac{w_i^{n+1} - w_i^n}{\Delta t} = D \frac{w_{i+1}^{n+1} - 2w_i^{n+1} + w_{i-1}^{n+1}}{(\Delta x)^2}$$

である。これを整理すると、

$$-\alpha w_{i-1}^{n+1} + (1 + 2\alpha) w_i^{n+1} - \alpha w_{i+1}^{n+1} = w_i^n, \qquad \alpha = \frac{D\Delta t}{(\Delta x)^2}$$

となる。境界値を指定すれば、本章と同じ三重対角構造の連立一次方程式を各ステップで解くことになる。この線形拡散問題に対する後退 Euler 法では、陽解法の拡散安定条件を避けられるが、大きな時間刻みでも時間変化を正確に追えるという意味ではない。安定性による制約と、精度のために必要な時間刻みを区別する。

宇宙流体計算では、流体部分を陽的に進め、熱伝導や輻射の拡散など、時間刻みの制約が厳しい過程を陰的に扱うことがある。また、流体の時間更新が陽的でも、自己重力を求めるには Poisson 方程式を解く。本章では密度を固定したが、自己重力流体では密度分布が変わるため、時間積分の各段階と整合する密度から重力場を更新する必要がある。詳しい解法は藤井・立川 [10]、松本ほか [9]、自己重力との組合せは富阪ほか編 [8] を参照すること。

第 3 章　球対称ポテンシャルと時間積分法

第 1–2 章では、球対称な密度分布から重力場を求めた。本章では、同じ Plummer 球の重力場の中で星の位置と速度の時間変化を計算し、常微分方程式の時間積分法を学ぶ。時間積分そのものの誤差を調べるため、加速度には解析式を用い、第 2 章の数値重力場に含まれる誤差とは切り分けて検証する。

運動方程式に初期位置と初期速度を与えると、その後の軌道を求める初期値問題となる。Euler 法、古典的 4 次 Runge–Kutta 法（RK4）、リープフロッグ法（leapfrog method）を同じ軌道に適用し、時間刻みを変えた収束試験と保存量の変化から結果を検証する。軌道の物理は Ryden §7.1[1]、Euler 法と RK4 は幸谷第 13 章 [5]、リープフロッグ法の長時間挙動は松本ほか §9.6[9] を参照する。

本章で学ぶこと

- 運動方程式を、位置と速度を未知数とする 1 階の常微分方程式系に書き換え、初期値問題として設定できる。
- Euler 法、RK4、リープフロッグ法の更新の考え方を説明し、時間ループとして実装できる。
- 円軌道の解析解との比較と時間刻みを変えた収束試験から、時間積分法の誤差と収束次数を評価できる。
- エネルギーと角運動量の変化を用いて数値軌道を診断し、保存量だけでは位置の精度を判断できないことを説明できる。
- 収束次数と長時間積分での保存量の振る舞いを区別し、必要な精度・積分時間・計算コストに応じて時間積分法を選ぶ際の考え方を説明できる。

本章の数値実験は Notebook 3 で行う。本文では更新式と計算部分の対応を示し、完全な可視化コードは Notebook に置く。軸対称・非軸対称の軌道は付録Cで扱い、本章は球対称モデルだけで学習できる。

3.1 Plummer 球の重力場と軌道計算の準備

第 1–2 章で用いた Plummer ポテンシャル

$$\Phi(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{3.1}$$

の中で星の軌道を計算する。重力場は時間に依存せず、計算する星の運動によっても変化しないものとする。中心から測った位置ベクトルを $\boldsymbol{x}$、その大きさを $r = |\boldsymbol{x}|$ と書く。第 2 章と同様に、動径方向の重力加速度を $g_r = -d\Phi/dr$ とする。軌道計算では重力加速度をベクトルとして扱うため、以下では重力加速度ベクトルを $\boldsymbol{g}$ と書く。球対称系では、中心以外で

$$\boxed{\boldsymbol{g} = g_r \frac{\boldsymbol{x}}{r} = -\frac{GM}{(r^2 + a^2)^{3/2}} \boldsymbol{x}} \tag{3.2}$$

となる。この右辺は中心でもそのまま評価でき、$\boldsymbol{g}(\boldsymbol{0}) = \boldsymbol{0}$ である。[1]

[1] このポテンシャルは、無衝突系を扱う重力 N 体計算で用いられる **Plummer 型ソフトニング（softening）**と同じ関数形を持つ [9]。粒子間距離を r、粒子質量を m とし、ポテンシャルを $-Gm/\sqrt{r^2 + \epsilon^2}$ とすると、$\epsilon > 0$ なら $r \to 0$ でも重力加速度は発散しない。ただし、本章の a は質量モデル全体のスケール半径であり、ϵ は粒子間の重力を平滑化する長さである。同じ関数形に現れるが、両者の役割は異なる。

Notebook 3 では $G = M = a = 1$ の無次元単位を用いる。質量単位を M、長さ単位を a とし、時間単位 t_* を $a/t_*^2 = GM/a^2$ となるように選ぶと、

$$t_* = \sqrt{\frac{a^3}{GM}}, \qquad v_* = \sqrt{\frac{GM}{a}}, \qquad \Phi_* = \frac{GM}{a} \tag{3.3}$$

を得る。無次元の時刻・速度・ポテンシャルを物理単位へ戻すときは、それぞれ t_*、v_*、Φ_* を掛ければよい。

以下では、位置ベクトルから Plummer ポテンシャルと重力加速度を計算する関数を定義する。

Notebook 3 ｜ Plummer 球の重力場

初期条件・解析解・軌道計算で同じ G, M, a を使う。
`phi_plummer_pos` は、位置ベクトルからポテンシャルを計算する。`np.sum(x**2, axis=-1)` は、位置ベクトルの各成分の二乗和を求める。`axis=-1` は配列の最後の軸を指定しており、複数の時刻の位置をまとめて与えた場合にも、それぞれのポテンシャルを計算できる。
`acc_plummer` は、一つの位置ベクトルから重力加速度を計算する。`acc_plummer_model` は、設定した G, M, a を用いて、位置ベクトルだけから加速度を計算する補助関数である。後の時間積分では、この関数を使用する。

対応セル（重力場の定義）

```
import numpy as np

# 無次元単位を設定する
G = 1.0
M = 1.0
a = 1.0

# 位置ベクトルから Plummer ポテンシャルを計算する
def phi_plummer_pos(x, G, M, a):
    x = np.asarray(x)
    r2 = np.sum(x**2, axis=-1)
    return -G * M / np.sqrt(r2 + a**2)

# 一つの位置ベクトルから加速度を計算する
def acc_plummer(x, G, M, a):
    x = np.asarray(x)
    r2 = np.dot(x, x)
    return -G * M * x / (r2 + a**2)**1.5

# 時間積分で用いる加速度関数
def acc_plummer_model(x):
    return acc_plummer(x, G, M, a)
```

球対称ポテンシャル中では角運動量が保存されるため、恒星の軌道は一つの平面内に限られる。そこで、本章では x–y 平面内の軌道を扱う。

3.2 初期値問題としての運動方程式

軌道を計算するには、前節の加速度を使って位置と速度の変化を求める。位置を $\boldsymbol{x}$、速度を $\boldsymbol{v}$ とすると、運動方程式は

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{v}, \qquad \frac{d\boldsymbol{v}}{dt} = -\nabla\Phi(\boldsymbol{x}) \equiv \boldsymbol{g}(\boldsymbol{x}) \tag{3.4}$$

である。位置と速度を一つの**状態ベクトル**にまとめ、

$$\boldsymbol{y} = \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{v} \end{pmatrix} \tag{3.5}$$

と書くと、

$$\frac{d\boldsymbol{y}}{dt} = \boldsymbol{f}(\boldsymbol{y}) \tag{3.6}$$

という 1 階の常微分方程式（ordinary differential equation; ODE）の連立系になる。右辺は、現在の位置と速度からその変化率を返す関数であり、

$$\boldsymbol{f}(\boldsymbol{y}) = \begin{pmatrix} \boldsymbol{v} \\ \boldsymbol{g}(\boldsymbol{x}) \end{pmatrix} \tag{3.7}$$

である。本章の重力場は時間に依存しないため、右辺には時刻 t が明示的には現れない。この形にすれば、一般の常微分方程式の時間積分法を軌道計算に適用できる[2]

運動方程式だけでは、星がどの軌道をたどるかは決まらない。本章では、初期時刻 $t = 0$ における位置 $\boldsymbol{x}(0) = \boldsymbol{x}_0$ と速度 $\boldsymbol{v}(0) = \boldsymbol{v}_0$ を与え、その後の状態を求める。第 2 章で空間の両端に条件を与えた境界値問題とは、条件を与える場所と解の求め方が異なる。

初期値問題

微分方程式と、初期時刻に与える条件を組み合わせてその後の解を求める問題を、**初期値問題（initial-value problem）**と呼ぶ。

数値計算では、一定の時間刻み Δt を用い、

$$t_n = n\Delta t, \qquad n = 0, 1, 2, \ldots \tag{3.8}$$

という離散的な時刻で状態を求める。時刻 t_n の厳密な状態を $\boldsymbol{y}(t_n)$、その数値近似を $\boldsymbol{y}_n$ と書く。以下の時間積分法では、既知の $\boldsymbol{y}_n$ から次の時刻の $\boldsymbol{y}_{n+1}$ を求める操作を繰り返す。

3.3 Euler 法

最も基本的な時間積分法が Euler 法（Euler method）である。現在の状態の変化率を、次の時刻まで一定とみなして解を進める。この更新式は、時刻 t_n の厳密解を Taylor 展開すると理解できる。

$$\boldsymbol{y}(t_n + \Delta t) = \boldsymbol{y}(t_n) + \Delta t \left.\frac{d\boldsymbol{y}}{dt}\right|_{t_n} + \frac{\Delta t^2}{2} \left.\frac{d^2\boldsymbol{y}}{dt^2}\right|_{t_n} + O(\Delta t^3). \tag{3.9}$$

式(3.6)を使い、Δt^2 以上の項を省略して数値近似を更新すると、

$$\boxed{\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \Delta t\, \boldsymbol{f}(\boldsymbol{y}_n)} \tag{3.10}$$

を得る。

[2] 自己重力の N 体計算では、各粒子の加速度を全粒子の配置から求めるため、粒子の運動に伴って重力場も変化する。全粒子の位置と速度をまとめれば同じ常微分方程式系の形で書けるが、粒子同士の運動は結びついており、各粒子を独立した固定場中の軌道として積分するわけではない [8]。

この更新の意味を、1 変数の方程式 $dy/dt = f(y)$ で考える。t–y 平面では、始点での変化率 $k_1 = f(y_n)$ は、その点での傾きに対応する。Euler 法は、この傾きを保つ直線に沿って時間幅 Δt だけ進み、$y_{n+1} = y_n + \Delta t\, k_1$ と更新する。図3.1では、時間とともに増加する解を持つ $dy/dt = 1 + y^2$ を例に、$t_n = 0$、$y_n = 0$、$\Delta t = 1$ の 1 ステップを示す。この初期条件に対する厳密解は $y(t) = \tan t$ である。始点の傾きは $k_1 = 1$ なので、Euler 法の次の値は $y_{n+1} = 1$ となるが、厳密解は $y(1) = \tan 1 \simeq 1.5574$ である。

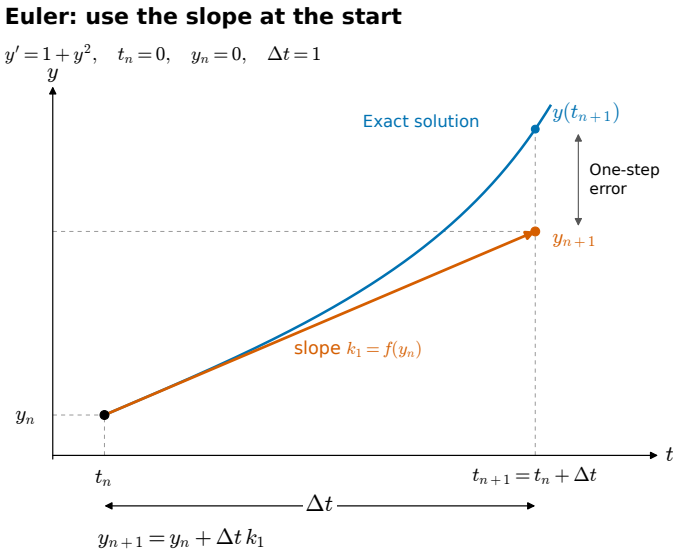


図 3.1 Euler 法の 1 ステップ。青い曲線は厳密解 $y(t) = \tan t$、朱色の直線は始点の傾き $k_1 = f(y_n)$ を用いた更新を表す。下の横矢印は時間幅 Δt、右の縦矢印は同じ終端時刻における厳密解と数値近似の差を示す。始点を厳密に与えているため、この差の大きさが 1 ステップの局所打ち切り誤差の大きさに対応する。

時刻 t_n の値が厳密であるとして、1 ステップで生じるずれを**局所打ち切り誤差**と呼ぶ。Euler 法では、省略した最初の項から、この誤差は $O(\Delta t^2)$ である。一方、更新を繰り返して得られる数値解には、それまでの局所的な誤差が伝播・累積した**大域誤差**が含まれる。滑らかな右辺に対して十分小さい時間刻みを用い、固定した終端時刻まで積分すると、Euler 法の大域誤差は $O(\Delta t)$ となり、1 次精度である [5]。この収束次数は §3.7で数値的に測定する。

式(3.10)を位置と速度へ書き戻すと、

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \Delta t\, \boldsymbol{v}_n, \tag{3.11}$$

$$\boldsymbol{v}_{n+1} = \boldsymbol{v}_n + \Delta t\, \boldsymbol{g}(\boldsymbol{x}_n). \tag{3.12}$$

これらの更新式を Python に対応させたものが Listing 3.1 である。

Listing 3.1　Euler 法の 1 ステップ

```
# 位置と速度の両方を、更新前の値から求める
x_new = x + dt * v
v_new = v + dt * acc_func(x)
```

ここで、`acc_func` は、位置を受け取って加速度を返す関数を表す引数である。Plummer 球の軌道計算では、前節で定義した `acc_plummer_model` を指定する。このように加速度関数を指定することで、同じ時間積分法を異なる重力ポテンシャルにも適用できる。また、`dt` は時間刻み Δt を表す。

Euler 法では、位置と速度の両方を時刻 t_n の値から計算する。したがって、速度の更新に用いる加速度は、更新後の位置 `x_new` ではなく、更新前の位置 `x` から求める。この 1 ステップの処理を時間ループで繰り返す実装は、§3.6で示す。

3.4 古典的 4 次 Runge–Kutta 法（RK4）

Euler 法は、1 ステップの始点での変化率だけを使う。Runge–Kutta 法では、時間区間内の状態を予測して右辺を複数回評価し、その結果を組み合わせて高い精度を得る。古典的 4 次 Runge–Kutta 法（RK4）では、次の 4 つの変化率を計算する。

$$\boldsymbol{k}_1 = \boldsymbol{f}(\boldsymbol{y}_n), \tag{3.13}$$

$$\boldsymbol{k}_2 = \boldsymbol{f}\left(\boldsymbol{y}_n + \frac{\Delta t}{2}\boldsymbol{k}_1\right), \tag{3.14}$$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(\boldsymbol{y}_n + \frac{\Delta t}{2}\boldsymbol{k}_2\right), \tag{3.15}$$

$$\boldsymbol{k}_4 = \boldsymbol{f}\left(\boldsymbol{y}_n + \Delta t\,\boldsymbol{k}_3\right). \tag{3.16}$$

$\boldsymbol{k}_1$ は始点での変化率、$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ は中間時刻、$\boldsymbol{k}_4$ は終点時刻の予測状態での変化率である。$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ は同じ中間時刻に対応するが、その評価に用いる状態の予測値が異なる。これらを使って、

$$\boxed{\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \frac{\Delta t}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right)} \tag{3.17}$$

と更新する [5]。

RK4 の幾何学的説明

図3.2では、図3.1と同じ $dy/dt = 1 + y^2$ の例を使い、4 つの傾きと最終更新を一つの t–y 平面に示す。図中の Y_2, Y_3, Y_4 は、それぞれ k_2, k_3, k_4 を評価するための予測値であり、

$$Y_2 = y_n + \frac{\Delta t}{2}k_1, \qquad Y_3 = y_n + \frac{\Delta t}{2}k_2, \qquad Y_4 = y_n + \Delta t\,k_3$$

と書ける。いずれも元の y_n を基準に作る値であり、Y_2 と Y_3 は同じ中間時刻 $t_n + \Delta t/2$ に対する異なる予測である。これらは厳密解の曲線上にあるとは限らない。

4 つの傾きがそろったら、重み付き平均 $\bar{k} = (k_1 + 2k_2 + 2k_3 + k_4)/6$ を求め、始点から $y_{n+1} = y_n + \Delta t\,\bar{k}$ と更新する。この例では、$Y_4 \simeq 1.3906$ に対して最終値は $y_{n+1} \simeq 1.5358$ となり、厳密解 $y(1) \simeq 1.5574$ に近い。最後の予測値 Y_4 をそのまま採用するのではなく、4 つの傾きを組み合わせて次の値を決める点が重要である。

中間状態の作り方と重み $1:2:2:1$ の組み合わせにより、厳密解の Taylor 展開と Δt の 4 次まで一致する。滑らかな右辺に対し、1 ステップの局所打ち切り誤差は $O(\Delta t^5)$ である。十分小さい時間刻みで固定した終端時刻まで積分したときの大域誤差は $O(\Delta t^4)$ となり、4 次精度である。ここでは更新式の使い方を学び、次数は円軌道の収束試験でも確かめる。

Notebook では、状態ベクトルを位置と速度の順に連結した 1 本の配列として扱う。`ndim` は空間の次元数で、本章の平面内の運動では 2 である。また、`y[n]` は時刻 t_n の状態ベクトルを表す。

Listing 3.2 に、RK4 法の 1 ステップの計算を示す。

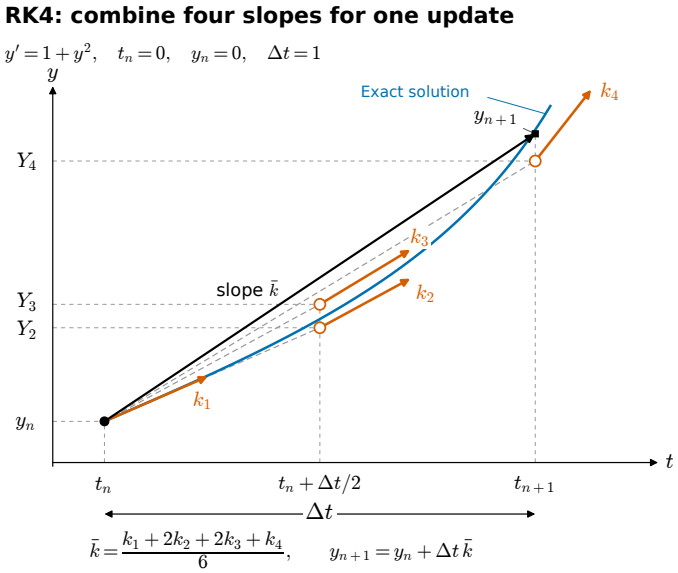


図 3.2 RK4 の 4 つの傾きと 1 ステップの更新。青い曲線は厳密解、白丸は予測値 Y_2, Y_3, Y_4、朱色の短い矢印は各評価点での傾き k_1, k_2, k_3, k_4 を表す。灰色の斜めの破線は始点から予測値を作る操作を示す。黒い矢印は平均傾き $\bar{k}$ を用いた最終更新であり、その終点の黒四角が y_{n+1} である。この矢印や予測値を結ぶ線は、ステップ途中の解の曲線を表すものではない。

Listing 3.2　古典的 4 次 Runge–Kutta 法の 1 ステップ

```
# 状態ベクトルから位置と速度を取り出し、時間微分を計算する
def rhs(state):
    x = state[:ndim]
    v = state[ndim:]
    return np.concatenate([v, acc_func(x)])

# 4 つの予測状態で変化率を求め、次の状態を計算する
yn = y[n]
k1 = rhs(yn)
k2 = rhs(yn + 0.5 * dt * k1)
k3 = rhs(yn + 0.5 * dt * k2)
k4 = rhs(yn + dt * k3)
y[n + 1] = yn + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0
```

ここで、`rhs` は式(3.7)に対応し、状態ベクトルからその時間微分を計算する関数である。`state[:ndim]` と `state[ndim:]` は、状態ベクトルから位置と速度をそれぞれ取り出す。また、`np.concatenate` は複数の配列を一つにつなぐ関数であり、ここでは速度と加速度をまとめて、状態ベクトルの時間微分を作る。

RK4 法は Euler 法より高次精度であるが、1 ステップで右辺を 4 回評価する。積分法を選ぶ際には、必要な精度だけでなく、計算コストも考慮する必要がある。例えば、$N_{\rm p}$ 個の粒子間の重力を直接計算する重力多体系では、全粒子対の重力を評価するため、1 回の加速度計算に $O(N_{\rm p}^2)$ の計算量がかかる [8]。このような計算では、時間積分法が必要とする加速度の評価回数も重要になる。

3.5 リープフロッグ法（蛙跳び法）

本章のように加速度が位置だけで決まる場合には、運動方程式を

$$\ddot{\boldsymbol{x}} = \boldsymbol{g}(\boldsymbol{x}) \tag{3.18}$$

と書ける。ドットは時間に関する微分を表す。この式は、位置と速度に関する 2 つの一階微分方程式

$$\dot{\boldsymbol{x}} = \boldsymbol{v}, \tag{3.19}$$

$$\dot{\boldsymbol{v}} = \boldsymbol{g}(\boldsymbol{x}) \tag{3.20}$$

に分けられる。

ここで、速度の変化率は位置だけで決まり、位置の変化率は速度だけで決まることに注目する。この性質を利用すると、時間積分を 2 つの簡単な操作に分けて考えられる。一つは、位置を固定して速度だけを更新する操作であり、キック（kick）と呼ぶ。もう一つは、速度を固定して位置だけを更新する操作であり、ドリフト（drift）と呼ぶ。リープフロッグ法は、この 2 つの操作を組み合わせて時間積分を行う方法である。

キックとドリフトを組み合わせる方法には、キック–ドリフト–キック（kick–drift–kick; KDK）形式と、ドリフト–キック–ドリフト（drift–kick–drift; DKD）形式がある。どちらも 2 次精度のリープフロッグ法である。本章では KDK 形式を用いる。この方法では、位置の更新（ドリフト）を 2 つの半ステップの速度更新（キック）ではさむ。この対称的な構成により、2 次精度の時間積分法となる。

KDK リープフロッグ法

まず速度を半ステップ更新し、

$$\boldsymbol{v}_{n+1/2} = \boldsymbol{v}_n + \frac{\Delta t}{2}\boldsymbol{g}(\boldsymbol{x}_n), \tag{3.21}$$

その補助速度で位置を 1 ステップ更新する。

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \Delta t\, \boldsymbol{v}_{n+1/2}. \tag{3.22}$$

次に、新しい位置で加速度を評価し、速度を残り半ステップ更新する。

$$\boldsymbol{v}_{n+1} = \boldsymbol{v}_{n+1/2} + \frac{\Delta t}{2}\boldsymbol{g}(\boldsymbol{x}_{n+1}). \tag{3.23}$$

すなわち、

$$\boldsymbol{v}_n \to \boldsymbol{v}_{n+1/2} \to \boldsymbol{x}_{n+1} \to \boldsymbol{v}_{n+1} \tag{3.24}$$

という計算順序で位置と速度を更新する。ここで $\boldsymbol{v}_{n+1/2}$ は計算途中の補助速度であり、この一連の操作で時刻 t_n から $t_{n+1} = t_n + \Delta t$ まで進む。一定の時間刻みでは、KDK 形式で隣り合うステップの半ステップのキックをまとめると、位置と速度を半ステップずれた時刻で交互に更新する形になる。リープフロッグ（蛙跳び）という名前は、この互い違いの更新に由来する。図3.3に、その時間的な配置を示す。

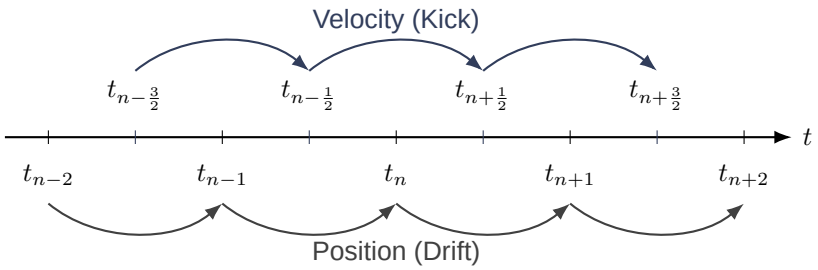


図 3.3 リープフロッグ法における位置と速度の時間的な配置。位置は整数時刻、速度は半整数時刻に対応する。上側の矢印は速度の更新（kick）、下側の矢印は位置の更新（drift）を表す。位置と速度が半ステップずれて交互に更新されることが、「蛙跳び」という名前の由来である。

式(3.21)–(3.23)は、Listing 3.3 の 3 行に対応する。

Listing 3.3　リープフロッグ法の 1 ステップ

```
# 速度を半ステップ、位置を 1 ステップ、速度を半ステップ更新する
v_half = v + 0.5 * dt * acc_func(x)
x_new = x + dt * v_half
v_new = v_half + 0.5 * dt * acc_func(x_new)
```

リープフロッグ法は、滑らかな加速度に対して 2 次精度の時間積分法である。その収束次数は §3.7の数値実験で確認する。更新式から誤差次数を求める計算は、以下の補足に示す。

Listing 3.3 では、各ステップの最初と最後に加速度を計算している。しかし、前のステップの最後に求めた加速度は、次のステップの最初にも使えるため、保存して再利用すれば計算回数を減らせる。本書では、更新式との対応を分かりやすくするため、加速度を毎回計算している。

補足｜リープフロッグ法が 2 次精度になる理由

時刻 t_n の位置と速度が厳密であるとして、1 ステップの誤差を調べる。最初の速度更新を位置更新へ代入すると、

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \Delta t\, \boldsymbol{v}_n + \frac{\Delta t^2}{2}\boldsymbol{g}(\boldsymbol{x}_n) \tag{3.25}$$

となる。これは位置の Taylor 展開を 2 次まで残した式である。滑らかな加速度なら、厳密解の次項は $\Delta t^3 \dot{\boldsymbol{g}}_n/6$ である。ここで $\dot{\boldsymbol{g}}_n$ は、軌道に沿った加速度の時間微分を t_n で評価した値を表す。したがって、位置の局所打ち切り誤差は $O(\Delta t^3)$ である。

2 回の速度更新をまとめると、

$$\boldsymbol{v}_{n+1} = \boldsymbol{v}_n + \frac{\Delta t}{2}\left[\boldsymbol{g}(\boldsymbol{x}_n) + \boldsymbol{g}(\boldsymbol{x}_{n+1})\right] \tag{3.26}$$

となる。一方、厳密な速度変化は

$$\boldsymbol{v}(t_{n+1}) - \boldsymbol{v}(t_n) = \int_{t_n}^{t_{n+1}} \boldsymbol{g}(\boldsymbol{x}(t))\, dt \tag{3.27}$$

である。式(3.26)は、加速度の時間積分に第 1 章の台形則を適用した形になっている。厳密な端点加速度を用いた場合、台形則による 1 区間の誤差は $O(\Delta t^3)$ である。

実際には、終点の加速度を数値的な位置 $\boldsymbol{x}_{n+1}$ で評価する。この位置には $O(\Delta t^3)$ の誤差があるため、滑らかな加速度では、その評価値の誤差も $O(\Delta t^3)$ となる。ただし、速度更新ではこれに $\Delta t/2$ を掛けるので、速度への寄与は $O(\Delta t^4)$ である。この寄与は台形則の主要な誤差より高次であり、速度の局所打ち切り誤差は $O(\Delta t^3)$ のままである。

固定した終端時刻までのステップ数は $O(\Delta t^{-1})$ である。滑らかな力に対して安定に積分できる範囲では、局所的な誤差の伝播・累積を含めた位置と速度の大域誤差は、ともに $O(\Delta t^2)$ となる。これがリープフロッグ法の 2 次精度に対応する。

3.6 円軌道による時間積分法の検証

ここまでに示した 3 つの更新式を繰り返して、実際に軌道を計算する。まず位置の解析解が分かる円軌道を用い、同じ初期条件と時間刻みで得た結果を比較する。

円軌道の解析解

半径 r_0 の円軌道の角速度を Ω とする。円運動に必要な向心加速度の大きさと重力加速度の大きさを等しくすると、

$$\Omega^2 r_0 = \frac{GMr_0}{(r_0^2 + a^2)^{3/2}} \tag{3.28}$$

であるから、

$$\Omega^2 = \frac{GM}{(r_0^2 + a^2)^{3/2}} \tag{3.29}$$

を得る。円運動速度と周期は、それぞれ

$$v_{\rm circ} = \Omega r_0, \qquad T_0 = \frac{2\pi}{\Omega} \tag{3.30}$$

である。Notebook 3 と同じく $G = M = a = r_0 = 1$ とすると、

$$\Omega = 0.59460356, \qquad v_{\rm circ} = 0.59460356, \qquad T_0 = 10.56701600 \tag{3.31}$$

となる。角速度の単位は t_*^{-1} で、速度と周期の単位は式(3.3)に示したものである。

初期条件

$$(x, y) = (r_0, 0), \qquad (v_x, v_y) = (0, v_{\rm circ}) \tag{3.32}$$

に対する解析的な位置は

$$x(t) = r_0 \cos \Omega t, \qquad y(t) = r_0 \sin \Omega t \tag{3.33}$$

であり、数値軌道を検証する基準になる。

時間ループと積分関数

軌道全体を求めるには、初期位置と初期速度を配列に保存し、1 ステップの更新を繰り返す。この処理を、Euler 法、RK4、リープフロッグ法について、それぞれ `integrate_euler`、`integrate_rk4`、`integrate_leapfrog` という関数にまとめる。各関数には、加速度を返す関数 `acc_func`、初期位置 `x0`、初期速度 `v0`、正の時間刻み `dt`、ステップ数 `nstep` を渡す。初期位置と初期速度は、同じ次元数の 1 次元配列とする。

`nstep` 回更新すると、初期時刻を含めて `nstep + 1` 個の時刻の値が得られる。返り値の `x` と `v` は、行が時刻、列が座標成分を表す配列である。したがって、`x[n]` は時刻 t_n の位置ベクトル、`x[-1]` は最終時刻の位置ベクトルに対応する。

Notebook 3 ｜時間ループと積分関数

> 配列の準備、初期値の保存、時間ループ、結果の返却という構造は 3 つの関数に共通する。時間ループ内の更新式が、Listing 3.1–3.3 に対応する。

対応セル（Euler 法）

```
def integrate_euler(acc_func, x0, v0, dt, nstep):
    # 小数を保持できる初期値と、全時刻の保存用配列を用意する
    x0 = np.asarray(x0, dtype=float)
    v0 = np.asarray(v0, dtype=float)
    ndim = len(x0)
    x = np.empty((nstep + 1, ndim))
    v = np.empty((nstep + 1, ndim))
    x[0], v[0] = x0, v0

    # 各ステップで、更新前の位置と速度から次の値を求める
    for n in range(nstep):
        x[n + 1] = x[n] + dt * v[n]
        v[n + 1] = v[n] + dt * acc_func(x[n])

    return x, v
```

対応セル（RK4）

```
def integrate_rk4(acc_func, x0, v0, dt, nstep):
    x0 = np.asarray(x0, dtype=float)
    v0 = np.asarray(v0, dtype=float)
    ndim = len(x0)
    y = np.empty((nstep + 1, 2 * ndim))
    y[0] = np.concatenate([x0, v0])

    # 状態の前半を位置、後半を速度として変化率を返す
    def rhs(state):
        x = state[:ndim]
        v = state[ndim:]
        return np.concatenate([v, acc_func(x)])

    # Listing 3.2 の更新を繰り返す
    for n in range(nstep):
        yn = y[n]
        k1 = rhs(yn)
        k2 = rhs(yn + 0.5 * dt * k1)
        k3 = rhs(yn + 0.5 * dt * k2)
        k4 = rhs(yn + dt * k3)
        y[n + 1] = yn + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0

    # 状態の履歴を、位置と速度の配列に分けて返す
    return y[:, :ndim], y[:, ndim:]
```

対応セル（リープフロッグ法）

```
def integrate_leapfrog(acc_func, x0, v0, dt, nstep):
    x0 = np.asarray(x0, dtype=float)
    v0 = np.asarray(v0, dtype=float)
    ndim = len(x0)
    x = np.empty((nstep + 1, ndim))
    v = np.empty((nstep + 1, ndim))
    x[0], v[0] = x0, v0

    # キック、ドリフト、キックの順に更新する
    for n in range(nstep):
        v_half = v[n] + 0.5 * dt * acc_func(x[n])
        x[n + 1] = x[n] + dt * v_half
        v[n + 1] = v_half + 0.5 * dt * acc_func(x[n + 1])

    return x, v
```

1 周期の軌道を比較する

これらの関数を使い、円軌道の 1 周期 T_0 を $N = 100$ ステップに分けて計算する。コード中の `nstep` がステップ数 N に対応し、時間刻みは $\Delta t = T_0/100$ である。

Notebook 3 ｜円軌道による時間積分法の比較

初期条件と周期を準備し、3 つの積分関数に同じ加速度・初期条件・時間刻みを渡す。返された位置配列を使って、図3.4を描く。

対応セル（初期条件と計算）

```
# 円軌道の角速度、円運動速度、周期を求める
r0 = 1.0
Omega0 = np.sqrt(G * M / (r0**2 + a**2)**1.5)
vc0 = Omega0 * r0
T0 = 2.0 * np.pi / Omega0
x0 = np.array([r0, 0.0])
v0 = np.array([0.0, vc0])

# 1 周期を 100 ステップで積分する
nstep = 100
dt = T0 / nstep
x_eu, v_eu = integrate_euler(acc_plummer_model, x0, v0, dt, nstep)
x_lf, v_lf = integrate_leapfrog(acc_plummer_model, x0, v0, dt, nstep)
x_rk, v_rk = integrate_rk4(acc_plummer_model, x0, v0, dt, nstep)
```

図3.4では、Euler 法の軌道が外側へずれる一方、リープフロッグ法と RK4 は解析的な円軌道に近く見える。ただし、一つの時間刻みで描いた軌道だけでは、誤差の大きさや収束次数を十分には判断できない。次節では、解析解からの位置のずれを直接測り、時間刻みを変えたときの誤差の減少を調べる。

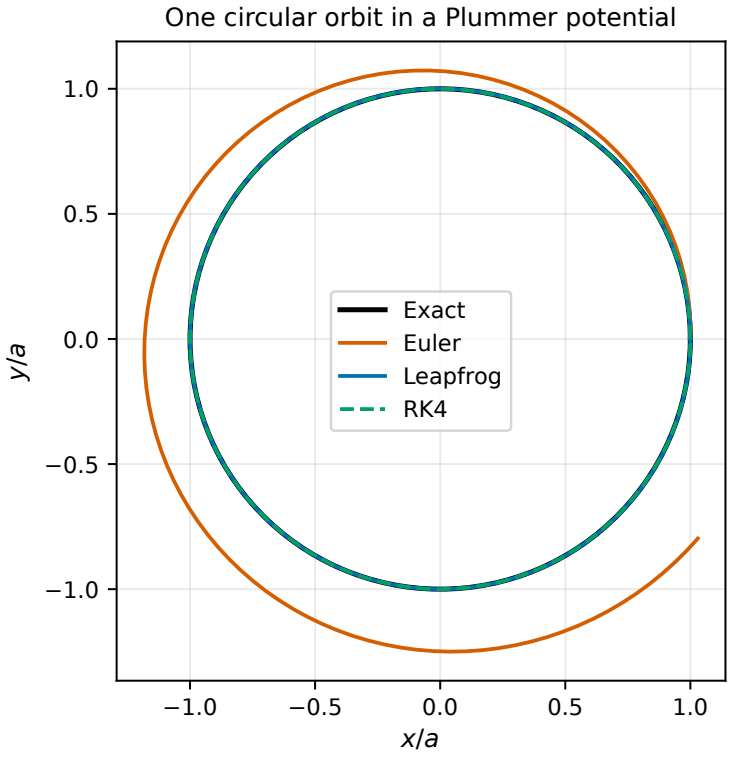


図 3.4　1 周期 T_0 を $N = 100$ ステップで積分した円軌道。時間刻みは $\Delta t = T_0/100$ である。解析解と 3 つの時間積分法を比較している。

3.7 時間積分法の収束次数

前節では、一つの時間刻みで 1 周期の軌道を描いた。本節では終端時刻を固定し、時間刻みを変えて同じ計算を繰り返す。第1.5節の格子収束試験で格子幅を変えたのと同じ考え方で、時間積分法の収束次数を測定する。

ここでは、円軌道を 90 度進んだ時刻 $T_0/4$ を終端時刻とする。この時刻の厳密な位置は

$$\boldsymbol{x}_{\mathrm{exact}}(T_0/4) = (0, r_0) \tag{3.34}$$

と分かっているので、数値的な位置とのずれ

$$\epsilon_x = |\boldsymbol{x}_{\mathrm{num}}(T_0/4) - \boldsymbol{x}_{\mathrm{exact}}(T_0/4)| \tag{3.35}$$

を直接測定できる。縦棒はベクトルの大きさを表し、ϵ_x は二つの位置の間の距離である。コードでは `np.linalg.norm` で計算する。

Notebook 3 では、$T_0/4$ までを $N = 20, 40, 80, 160, 320$ ステップで積分する。前節の 1 周期の計算とは異なり、ここでの時間刻みは $\Delta t = T_0/(4N)$ である。誤差が $\epsilon_x \simeq C(\Delta t)^p$ に従う範囲では、両対数プロットの傾きが収束次数 p となる。C は、終端時刻と初期条件を固定したときに時間刻みに依存しない係数である。

Notebook 3 ｜時間積分法の収束次数

積分関数を辞書 `methods` にまとめ、手法と時間刻みを変えて計算する。`errors` に位置誤差を保存し、5 種類の時間刻みすべてを用いて両対数上で直線近似する。

対応セル（誤差と収束次数の測定）

```
# 3 つの積分関数を、名前と対応させて登録する
methods = {
    "Euler": integrate_euler,
    "Leapfrog": integrate_leapfrog,
    "RK4": integrate_rk4,
}

# 終端時刻を固定し、試すステップ数と解析的な位置を用意する
N_values = np.array([20, 40, 80, 160, 320])
t_end = T0 / 4.0
x_exact_end = np.array([0.0, r0])
dt_values = t_end / N_values
errors = {}

# 最終時刻の位置のずれを距離として測る
for name, method in methods.items():
    err = []
    for N_i in N_values:
        dt_i = t_end / N_i
        x_i, v_i = method(acc_plummer_model, x0, v0, dt_i, int(N_i))
        err.append(np.linalg.norm(x_i[-1] - x_exact_end))
    errors[name] = np.array(err)

# 両対数上での直線の傾きを求める
orders = {}
for name, err in errors.items():
    orders[name] = np.polyfit(np.log(dt_values), np.log(err), 1)[0]
```

この計算から、

$$p_{\mathrm{Euler}} = 0.991, \qquad p_{\mathrm{leapfrog}} = 2.000, \qquad p_{\mathrm{RK4}} = 4.038 \tag{3.36}$$

を得る。Euler 法はほぼ 1 次、リープフロッグ法はほぼ 2 次、RK4 はほぼ 4 次精度で収束している。時間刻みを半分にすると、誤差はそれぞれおよそ $1/2$、$1/4$、$1/16$ に減少する。図3.5の点列の傾きは、この誤差の減少を表している。

3.8 保存量による数値軌道の診断

前節では、円軌道の解析解と比較して位置の誤差を測った。軌道計算では、このような直接比較に加えて、本来一定である保存量が数値計算でも保たれているかを調べることができる。本節では、半径が時間とともに変化する離心軌道を用い、エネルギーと角運動量から数値軌道を診断する。

単位質量あたりのエネルギーを**比エネルギー（specific energy）**と呼び、

$$E = \frac{1}{2}v^2 + \Phi(r) \tag{3.37}$$

と定義する。ここで $v = |\boldsymbol{v}|$ である。また、単位質量あたりの角運動量を**比角運動量（specific angular momentum）**と呼び、

$$\boldsymbol{L} = \boldsymbol{x} \times \boldsymbol{v} \tag{3.38}$$

と定義する。以下では、これらを単にエネルギー、角運動量と呼ぶ。ポテンシャルが時間に依存しないためエネルギーが保存され、球対称な重力場の力が中心方向を向くため角運動量が保

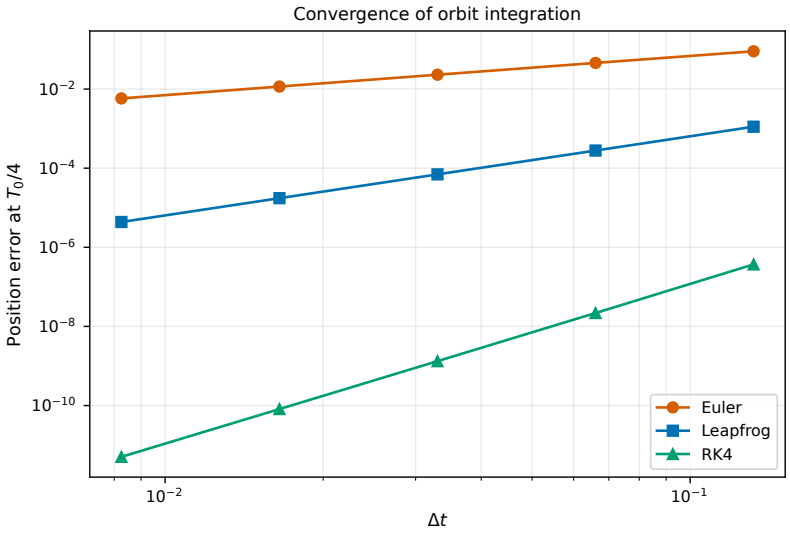


図 3.5　円軌道の $T_0/4$ 後の位置誤差と時間刻みの関係。両対数表示の傾きから、Euler 法・リープフロッグ法・RK4 の大域収束次数を測定する。

存される。したがって、厳密な運動では

$$\boxed{E = \text{const.}, \qquad \boldsymbol{L} = \text{const.}} \tag{3.39}$$

である [1]。本章の x–y 平面内の運動では、その平面に垂直な成分 $L_z = xv_y - yv_x$ を診断に用いる。

Notebook 3 では、$G = M = a = 1$ の単位で

$$\boldsymbol{x}_0 = (1, 0), \qquad \boldsymbol{v}_0 = (0, 0.45) \tag{3.40}$$

とする。初速度をこの半径の円運動速度より小さくすることで、半径が変化する軌道を作る。$\Delta t = T_0/100$ で、基準周期の 20 倍の時間 $20T_0$ まで 2000 ステップで積分する。T_0 は式(3.30)の円軌道の基準周期であり、この離心軌道の実測周期ではない。

計算した各時刻で保存量を評価し、初期値からの相対的なずれ

$$\frac{E(t) - E(0)}{|E(0)|}, \qquad \frac{L_z(t) - L_z(0)}{|L_z(0)|} \tag{3.41}$$

を求める。今回の初期条件では、どちらの分母も 0 ではない。これらの量が 0 からどれだけずれるかを調べることで、保存量が数値計算でどの程度保たれているかを確認できる。

Notebook 3 ｜離心軌道と保存量

前節で定義した `methods` を使い、同じ初期条件を 3 つの方法で積分する。`orbits` に軌道と保存量を保存し、`relative_errors` に式(3.41)の符号付きのずれを保存する。

対応セル（保存量を計算する関数）

```
# 全時刻の位置・速度配列から、各時刻の比エネルギーを返す
def energy_plummer(x, v, G, M, a):
    return 0.5 * np.sum(v**2, axis=1) + phi_plummer_pos(x, G, M, a)

# 平面内の運動に対する比角運動量の z 成分を返す
def angular_momentum_z(x, v):
    return x[:, 0] * v[:, 1] - x[:, 1] * v[:, 0]
```

対応セル（軌道と保存量の計算）

```
# 離心軌道の初期条件と計算時間を設定する
x0_e = np.array([1.0, 0.0])
v0_e = np.array([0.0, 0.45])
n_period = 20
steps_per_period = 100
dt = T0 / steps_per_period
nstep = n_period * steps_per_period
t = np.arange(nstep + 1) * dt

# 各手法の軌道と保存量の履歴を保存する
orbits = {}
for name, method in methods.items():
    x_i, v_i = method(acc_plummer_model, x0_e, v0_e, dt, nstep)
    E_i = energy_plummer(x_i, v_i, G, M, a)
    L_i = angular_momentum_z(x_i, v_i)
    orbits[name] = (x_i, v_i, E_i, L_i)
```

対応セル（保存量の相対変化）

```
# 符号付きの相対変化と、その絶対値の最大値を求める
relative_errors = {}
short_max_errors = {}
for name in methods:
    E_i, L_i = orbits[name][2:]
    relE = (E_i - E_i[0]) / abs(E_i[0])
    relL = (L_i - L_i[0]) / abs(L_i[0])
    relative_errors[name] = (relE, relL)
    max_E = np.max(np.abs(relE))
    max_L = np.max(np.abs(relL))
    short_max_errors[name] = {"energy": max_E, "Lz": max_L}
```

図3.6に計算した軌道を示す。Euler 法では軌道が外側へ広がる一方、リープフロッグ法と RK4 は中心付近の限られた範囲を運動する。この違いを、保存量の変化から調べる。

図3.7では、式(3.41)のエネルギーのずれの絶対値を、対数目盛で比較している。この時間刻みと積分時間では、最大絶対相対変化はリープフロッグ法で 2.655×10^{-4}、RK4 で 2.458×10^{-6} となり、RK4 の方が小さい。Euler 法の誤差はこれらより大きく、外側へ広がる軌道に対応してエネルギーも変化している。

角運動量のずれを図3.8に示す。リープフロッグ法では、角運動量のずれは非常に小さい。中心力の場合、キックで加える速度変化は位置ベクトルに平行であり、ドリフトで加える位置変化はその段階の速度に平行である。このため、それぞれの更新で $\boldsymbol{x} \times \boldsymbol{v}$ は変わらず、角運動量は浮動小数点演算による誤差を除いて保存される。

ただし、角運動量が保存されていても、数値軌道が正確とは限らない。例えば、円軌道では角運動量が一定でも、軌道位相に誤差が生じると、同じ時刻における恒星の位置は解析解からずれてしまう。そのため、角運動量の保存だけでなく、解析解との比較や時間刻みを変えた収束試験によっても、数値計算の精度を確かめる必要がある。

また、エネルギー誤差の大きさだけでなく、符号付きの時間変化にも違いがある。リープフロッグ法では誤差が振動するのに対し、RK4 では一方向へのずれが見られる。この符号付き誤差の拡大図は Notebook 3 で確認できる。次節では、積分時間を長くして両者の違いを調べ

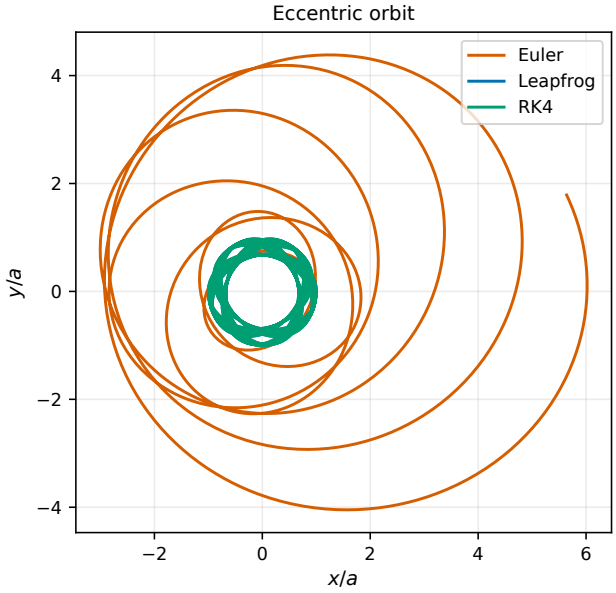


図 3.6　Plummer ポテンシャル中の離心軌道。初期条件は式(3.40)で、3 手法を同じ $\Delta t = T_0/100$ により $20T_0$ まで積分している。

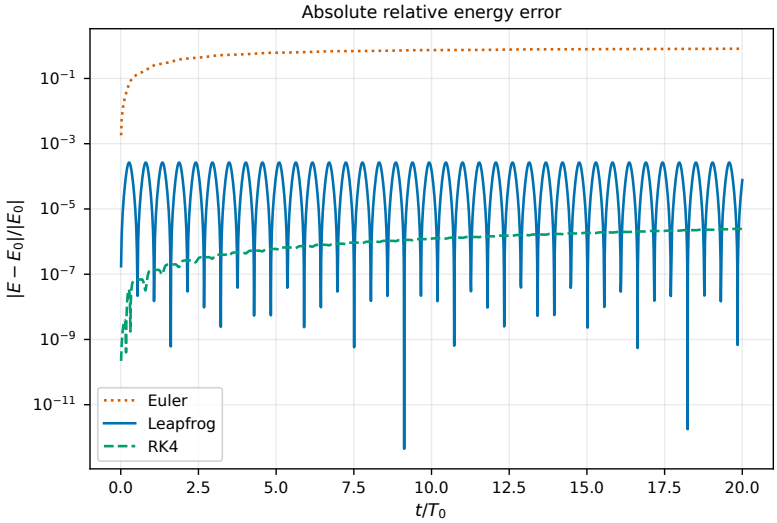


図 3.7　離心軌道における $|E(t) - E(0)|/|E(0)|$。条件は $\Delta t = T_0/100$、積分時間 $20T_0$ である。縦軸は対数目盛で、誤差が 0 の点は表示していない。リープフロッグ法の下向きの鋭い谷は、符号付き誤差が 0 に近づいて絶対値が一時的に小さくなる時刻に対応し、その瞬間に軌道全体が高精度になったことを意味しない。

る。保存量の選択は、解く問題によって変わる。自己重力の N 体計算では粒子間でエネルギーを交換するため、各粒子のエネルギーは一般には一定でない。外力がなく、粒子質量が一定の孤立系では、系全体のエネルギーなどを保存量の診断に用いる [8]。本章で各軌道のエネルギーを調べるのは、粒子が動いても変わらない、時間に依存しない重力場を与えているためである。

3.9 高次精度と長時間保存性の違い

前節の条件では、RK4 のエネルギー誤差はリープフロッグ法より小さかった。しかし、その大小だけでは、さらに長い時間まで積分したときの誤差を判断できない。本節では同じ離心軌道を長時間積分し、誤差が振動する場合と一方向にずれる場合の違いを比較する。

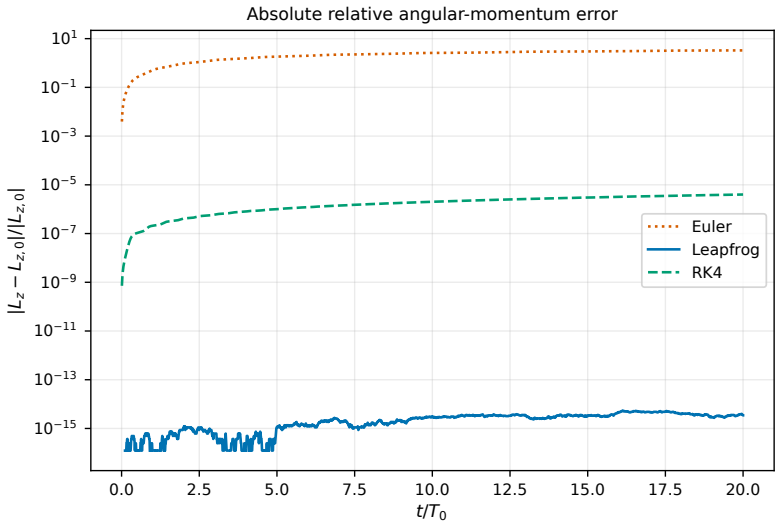


図 3.8　同じ計算における $|L_z(t) - L_z(0)|/|L_z(0)|$。縦軸は対数目盛で、誤差が 0 の点は表示していない。リープフロッグ法では角運動量が理論上厳密に保存されるが、実際の計算では浮動小数点演算によるわずかなずれが生じる。

Notebook 3 では、$\Delta t = T_0/50$ で $1000T_0$ まで積分する。T_0 は円軌道の基準周期である。前節より積分時間が長いだけでなく、時間刻みも 2 倍なので、時間経過だけの影響を調べる際は、この長時間計算の最初の $20T_0$ と全体の $1000T_0$ を比べる。手法間の比較は同じ時間刻みでの比較であり、同じ計算コストでの比較ではない。

符号付きのエネルギー誤差が振動する場合、最終値は計算を終えるタイミングによって変わる。そこで、時間変化に加えて、終端時刻 T までの出力時刻 t_n で測った最大絶対相対変化

$$\epsilon_{E,\max}(T) \equiv \max_{0 \le t_n \le T} \frac{|E(t_n) - E(0)|}{|E(0)|} \tag{3.42}$$

を用い、計算中に誤差がどこまで大きくなったかを比較する。

Notebook 3 ｜ RK4 とリープフロッグ法の長時間挙動

前節で定義した `x0_e`、`v0_e` を初期条件に用い、RK4 とリープフロッグ法で計算する。`long_results` には全時刻の符号付き誤差を、`long_max_errors` には二つの時間範囲の最大誤差を保存する。

対応セル（長時間積分）

```
# 基準周期あたり 50 ステップで、1000T0 まで積分する
n_period_long = 1000
steps_per_period_long = 50
dt_long = T0 / steps_per_period_long
nstep_long = n_period_long * steps_per_period_long
t_long = np.arange(nstep_long + 1) * dt_long

# 同じ離心軌道の符号付き相対エネルギー誤差を保存する
long_results = {}
for name in ["Leapfrog", "RK4"]:
    method = methods[name]
    x_i, v_i = method(acc_plummer_model, x0_e, v0_e, dt_long, nstep_long)
    E_i = energy_plummer(x_i, v_i, G, M, a)
    long_results[name] = (E_i - E_i[0]) / abs(E_i[0])
```

対応セル（最大誤差の比較）

```
# 同じ計算の最初の 20T0 と全体で、最大誤差を測る
n_early = 20 * steps_per_period_long
long_max_errors = {}
for name in ["Leapfrog", "RK4"]:
    err = long_results[name]
    max_early = np.max(np.abs(err[:n_early + 1]))
    max_all = np.max(np.abs(err))
    long_max_errors[name] = {"early": max_early, "all": max_all}
```

実行結果

表 3.1 同じ長時間計算から求めた $\epsilon_{E,\max}(T)$。どちらの行も $\Delta t = T_0/50$ であり、§3.8の $\Delta t = T_0/100$ とは異なる。

T/T_0	リープフロッグ法	RK4
20	1.055×10^{-3}	7.822×10^{-5}
1000	1.055×10^{-3}	3.948×10^{-3}

図3.9に、同じ計算の符号付き相対エネルギー誤差を示す。リープフロッグ法では誤差がほぼ一定の範囲で振動し、RK4 ではおおむね負の方向へのずれが続く。

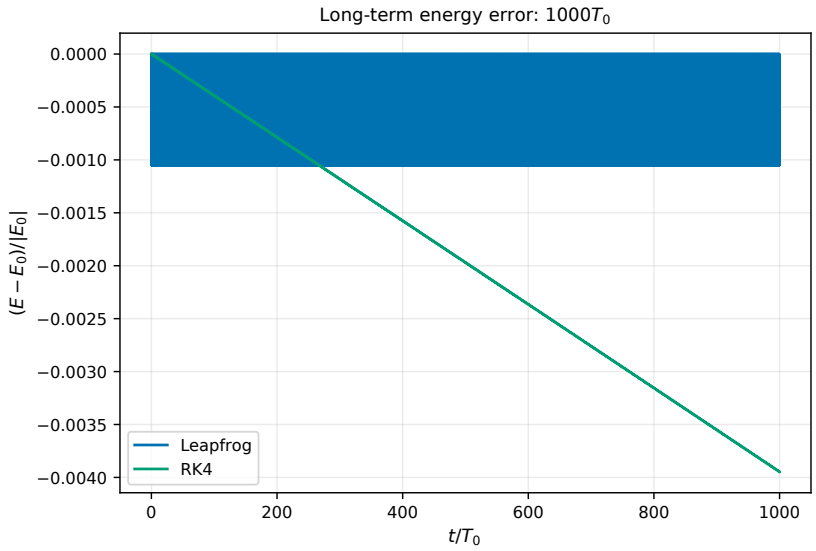


図 3.9 $\Delta t = T_0/50$ で $1000T_0$ まで積分した符号付き相対エネルギー誤差。リープフロッグ法はほぼ一定の範囲で振動し、RK4 には負の方向への長期的なずれが見られる。多数の振動が重なるため、リープフロッグ法の曲線は帯状に見える。

表3.1では、リープフロッグ法の最大誤差は最初の $20T_0$ で約 1.055×10^{-3} に達し、その後もほとんど増えない。一方、RK4 の誤差は初めは小さいが、時間とともに増加する。$20T_0$ までの最大誤差は RK4 の方が小さく、$1000T_0$ までではリープフロッグ法の方が小さい。ただし、大小が入れ替わる時刻は時間刻みに強く依存する。例えば $\Delta t = T_0/100$ では、$1000T_0$ まで積分しても RK4 の最大誤差の方が小さい。誤差の大小を比較するときは、時間刻みと積分時間を必ず併記する必要がある。

リープフロッグ法は 2 次精度だが、時間に依存しない保存力による滑らかな束縛軌道を、十分小さい固定時間刻みで積分すると、エネルギー誤差が一方向に蓄積しにくい性質を持つ [9]。これは、物理的エネルギーを各ステップで厳密に保存するという意味ではない。一方、RK4 が

4 次精度であることは、固定した終端時刻で時間刻みを小さくしたときの誤差の減少を表す。収束次数と長時間の保存量の振る舞いは別の性質なので、必要な精度、積分時間、計算コストに応じて方法を選ぶことが重要である。

補足｜リープフロッグ法とシンプレクティック性

リープフロッグ法の長時間挙動を理解する背景には、保存力による運動が持つ構造がある。位置と速度を組にした状態を表す空間を**位相空間**と呼ぶ。本章のような保存力による運動は、Hamilton 系と呼ばれる枠組みで記述できる。この系に備わる位相空間の構造を離散的な時間更新でも保つ方法が、**シンプレクティック積分法（symplectic integrator）**であり、固定時間刻みのリープフロッグ法はその一例である。

この性質は、エネルギーを各ステップで厳密に保存することや、任意の時間刻みで安定に計算できることとは異なる。精度の次数だけとも区別する必要がある。実際の N 体計算や SPH 計算では、粒子の状態に応じた**可変時間刻み（adaptive timestep）**や、粒子ごとに異なる時間刻み（**独立時間刻み; individual timestep**）を用いる場合がある。その場合、本章の共通の固定時間刻みで見た長時間の保存性はそのままでは保証されず、時間刻みの決め方と更新手順を組み合わせた検証が必要になる。

理論的背景は Notebook 3 の補足と松本ほか §9.6[9] を参照する。

補足｜宇宙流体計算の時間積分：RK2 と RK3

本章で扱った時間積分は、星の軌道計算だけでなく、空間を離散化した流体の方程式にも用いられる。例えば有限体積法では、密度・運動量密度・エネルギー密度などのセル平均値を持たせる。全セルのこれらの量をまとめたベクトルを $\boldsymbol{U}$ とすると、空間方向を離散化した後の時間発展は

$$\frac{d\boldsymbol{U}}{dt} = \boldsymbol{f}(\boldsymbol{U}) \tag{3.43}$$

と書ける。ここでは、右辺が時刻に明示的に依存しない場合を考える。$\boldsymbol{f}(\boldsymbol{U})$ は、現在の状態からセル境界を通る流束や重力などの効果を評価し、各セルの時間変化率を計算する関数である。式(3.6)と同じ形なので、常微分方程式の時間積分法を適用できる。

格子法の宇宙流体計算では、2 次や 3 次の Runge–Kutta 法も用いられる [10, 9]。ここでは、その具体例として 2 段 2 次の RK2 と 3 段 3 次の RK3 を示す。$\boldsymbol{U}^n$ は時刻 t_n の数値状態を表し、上付きの (1)、(2) は計算途中の補助状態を表す。

2 次精度の Heun 法（RK2）は、

$$\begin{aligned}
\boldsymbol{U}^{(1)} &= \boldsymbol{U}^n + \Delta t\, \boldsymbol{f}(\boldsymbol{U}^n), \\
\boldsymbol{U}^{n+1} &= \frac{1}{2}\boldsymbol{U}^n + \frac{1}{2}\left[\boldsymbol{U}^{(1)} + \Delta t\, \boldsymbol{f}(\boldsymbol{U}^{(1)})\right]
\end{aligned}$$

と書ける。最初に Euler 法で予測値 $\boldsymbol{U}^{(1)}$ を求め、その予測値でも時間変化率を評価する。2 行目に 1 行目を代入すると、始点と予測した終点での変化率を平均して更新する式になる。

代表的な 3 次精度の公式（RK3）では、最初の予測値 $\boldsymbol{U}^{(1)}$ は上の Heun 法と共通である。その後、次の 2 段で更新する。

$$\begin{aligned}
\boldsymbol{U}^{(2)} &= \frac{3}{4}\boldsymbol{U}^n + \frac{1}{4}\left[\boldsymbol{U}^{(1)} + \Delta t\, \boldsymbol{f}(\boldsymbol{U}^{(1)})\right], \\
\boldsymbol{U}^{n+1} &= \frac{1}{3}\boldsymbol{U}^n + \frac{2}{3}\left[\boldsymbol{U}^{(2)} + \Delta t\, \boldsymbol{f}(\boldsymbol{U}^{(2)})\right].
\end{aligned}$$

ここで示した RK2 では 2 回、RK3 では 3 回、右辺の関数を評価する。いずれも、Euler 法による予測と既知の状態との重み付き平均を組み合わせた更新として読める。

SPH 法へ拡張して速度依存の力や散逸を扱う場合には、本章の保存力に対するリープフロッグ法の保存性はそのままでは保証されない。RK2・RK3 でも、空間離散化と組み合わせた時間刻みの制約を満たす必要がある。詳しくは藤井・立川 [10]、松本ほか [9] を参照すること。

付録 A　軸対称円盤の質量積分と回転曲線

第 1 章では、球対称な密度分布から内包質量を求め、数値積分法の精度を検証した。本付録では、Ryden §6.5 の薄い軸対称円盤を題材として、既習の数値積分を別の問題へ適用する [1]。球殻から円環へ質量要素を変え、同じ累積台形則で指数円盤の内包質量を求める。解析解との比較と格子収束試験によって、実装を再利用した場合も精度を確かめる。

後半では、円盤の回転曲線と球対称近似を比較する。ここでは解析式を用い、数値積分の誤差を混ぜずに、質量分布の仮定による違いを調べる。数値手法を再利用できることと、物理の関係式をそのまま使えることは別である。本付録は第 1 章の発展内容であり、第 2・3 章や他の付録を先に読む必要はない。

本付録で学ぶこと

- 軸対称円盤の質量を求める問題について、数値積分の被積分関数と積分区間を設定できる。
- 被積分関数を入れ替え、第 1 章の累積台形則の実装を別の質量分布に再利用できる。
- 解析解との比較と格子収束試験から、累積積分の誤差と収束次数を評価できる。
- 数値積分の離散化誤差と、有限の計算領域の外にある質量を区別できる。
- 数値計算の誤差と、物理モデルの仮定によって生じる違いを区別できる。

数値実験は Notebook A で行う。積分公式と累積積分の実装には、第 1 章で学んだものを使う [5]。本文では、被積分関数の設定、既習の実装の使い方、数値結果の検証を説明する。

A.1 球対称系と軸対称円盤の質量要素

球対称系では、半径 r、厚さ dr の球殻の質量は

$$dM = 4\pi r^2 \rho(r)\, dr \tag{A.1}$$

であった。一方、無限に薄い軸対称円盤では、円盤面内で中心から測った半径を R とし、単位面積あたりの質量を**面密度（surface density）** $\Sigma(R)$ で表す。半径 R、幅 dR の薄い**円環（annulus）**の面積は、円周に幅を掛けた

$$dA = 2\pi R\, dR \tag{A.2}$$

なので、その質量は

$$dM = 2\pi R \Sigma(R)\, dR \tag{A.3}$$

となる。半径 R より内側の内包質量は

$$\boxed{M(<R) = \int_0^R 2\pi R' \Sigma(R')\, dR'.} \tag{A.4}$$

ここで R' は積分変数である。数値積分に渡す被積分関数は $\Sigma(R')$ そのものではなく、$2\pi R'\Sigma(R')$、積分区間は $[0, R]$ である。球対称系の $4\pi r'^2 \rho(r')$ をこの関数に入れ替えれば、同じ 1 次元の積分法を使える。

A.2 指数円盤の内包質量

数値積分の検証には、内包質量を解析的に求められる**指数円盤（exponential disk）**

$$\Sigma(R) = \Sigma_0 e^{-R/R_d} \tag{A.5}$$

を使う。Σ_0 は中心面密度、R_d は面密度が中心値の $1/e$ になる半径である [1]。

全質量と中心面密度

円盤の全質量を M_d とすると、式(A.4)の上端を無限大にして

$$M_d = 2\pi\Sigma_0 \int_0^\infty R' e^{-R'/R_d}\, dR' \tag{A.6}$$

である。$u = R'/R_d$ と変数変換すれば、$R' = R_d u$、$dR' = R_d\, du$ なので

$$M_d = 2\pi\Sigma_0 R_d^2 \int_0^\infty u e^{-u}\, du. \tag{A.7}$$

部分積分から

$$\int u e^{-u}\, du = -u e^{-u} + \int e^{-u}\, du = -(u+1)e^{-u} + C \tag{A.8}$$

となる。C は積分定数である。上端と下端で評価すると定積分の値は 1 なので、

$$M_d = 2\pi\Sigma_0 R_d^2, \qquad \Sigma_0 = \frac{M_d}{2\pi R_d^2} \tag{A.9}$$

を得る。Notebook A では M_d と R_d を指定して Σ_0 を決める。

有限半径までの内包質量

同じ変数変換を有限の上端 R に対して使うと、

$$\begin{aligned} M(<R) &= 2\pi\Sigma_0 R_d^2 \int_0^{R/R_d} u e^{-u}\, du \\ &= M_d \left[1 - e^{-R/R_d} \left(1 + \frac{R}{R_d} \right) \right] \end{aligned} \tag{A.10}$$

となる。式(A.8)を上端と下端で評価し、式(A.9)を使った。これは Ryden の式 (6.161) に対応する解析解であり、$R = 0$ で 0、$R \to \infty$ で M_d となる [1]。以下では、この式を有限半径までの数値積分を検証する基準にする。

被積分関数と実装

被積分関数は

$$f_M(R) \equiv \frac{dM}{dR} = 2\pi R \Sigma(R) = \frac{M_d}{R_d} \frac{R}{R_d} e^{-R/R_d} \tag{A.11}$$

である。面密度は中心で最大だが、f_M は $R = 0$ で 0 になる。一定幅の円環の面積が中心へ向かって小さくなるためである。図A.1では、面密度と被積分関数をそれぞれの基準量で割り、半径に対する形を比較する。

式(A.5)と式(A.11)を関数にすると、Listing A.1 になる。`sigma_exponential` が面密度、`dMdR_exponential` が数値積分に渡す被積分関数を返す。

Listing A.1　指数円盤の面密度と被積分関数

```
def sigma_exponential(R, Md=1.0, Rd=1.0):
    Sigma0 = Md / (2.0 * np.pi * Rd**2)
    return Sigma0 * np.exp(-R / Rd)

def dMdR_exponential(R, Md=1.0, Rd=1.0):
    return 2.0 * np.pi * R * sigma_exponential(R, Md, Rd)
```

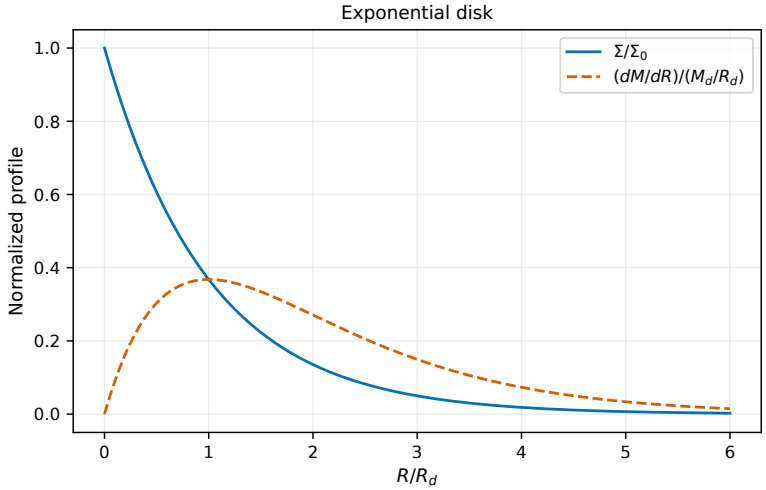


図 A.1　指数円盤の面密度と、一定の動径幅あたりの質量。面密度は Σ_0、被積分関数 dM/dR は M_d/R_d で規格化した。両者は異なる物理量なので、ここでは半径に対する形の違いを比較する。

A.3 累積台形則による数値積分

計算領域 $0 \le R \le R_{\max}$ を N 個の区間に分け、

$$R_i = i\Delta R, \qquad \Delta R = \frac{R_{\max}}{N}, \qquad i = 0, 1, \ldots, N \tag{A.12}$$

とする。第 1 章の累積台形則を f_M に使えば、内包質量の数値近似 $M_i \simeq M(< R_i)$ は

$$M_0 = 0, \qquad M_{i+1} = M_i + \frac{\Delta R}{2}\left[f_M(R_i) + f_M(R_{i+1})\right] \tag{A.13}$$

で求められる。$i = 0, 1, \ldots, N-1$ の順に更新すれば、各半径について 0 から積分をやり直す必要はない。

実装には第 1 章の Listing 1.5 の `cumulative_trapezoid_manual` を使う。この関数は、引数 `x` に格子、`y` に被積分関数の値を受け取り、各区間の台形面積を `np.cumsum` で累積する。ここでは `x` に R_i、`y` に $f_M(R_i)$ を渡す。積分する関数が変わるだけで、累積加算の実装は変わらない（§1.7）。

Notebook A ｜内包質量の数値積分

$G = M_d = R_d = 1$ とし、$0 \le R \le 10R_d$ を 400 区間に分ける。`R` は両端を含む 401 個の格子点である。`mass_exponential_exact` は式(A.10)を返す関数で、完全な定義は Notebook A に置く。

対応セル（計算部分）

```
# 0 <= R <= 10 Rd を N=400 区間に分ける
N = 400
R_max = 10.0 * Rd
R = np.linspace(0.0, R_max, N + 1)

# 被積分関数 dM/dR = 2 pi R Sigma(R) を格子点で評価する
dMdR = dMdR_exponential(R, Md, Rd)

# 累積台形則と、同じ格子点での解析解を比較する
M_num = cumulative_trapezoid_manual(R, dMdR)
M_exact = mass_exponential_exact(R, Md, Rd)
```

```

# 積分誤差と、計算領域の外にある質量を別々に測る
mass_error_outer = (M_num[-1] - M_exact[-1]) / Md
mass_outside = (Md - M_exact[-1]) / Md
```

同じ格子点で得た数値解と解析解を図A.2に示す。二つの曲線はほぼ重なり、外側の格子点では

$$\frac{M_{\rm num}(<R_{\rm max})}{M_d} = 0.9994484978, \qquad \frac{M_{\rm exact}(<R_{\rm max})}{M_d} = 0.9995006008 \tag{A.14}$$

となる。両者の差が、同じ上端まで積分したときの数値積分の誤差である。

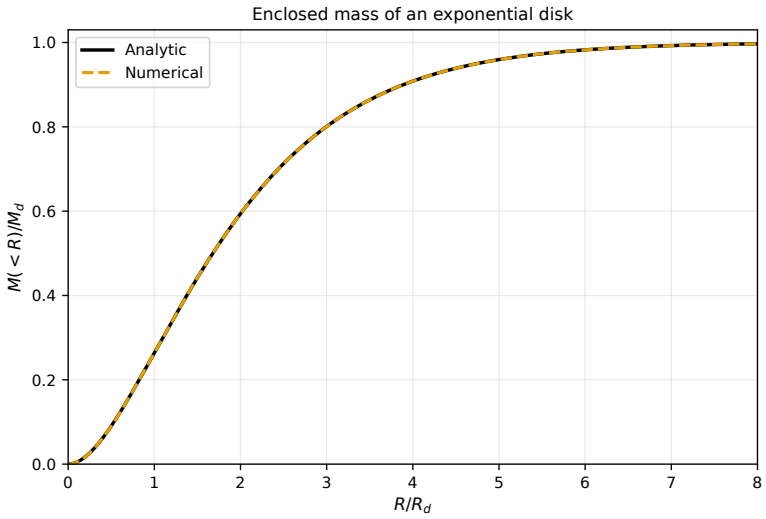


図 A.2　指数円盤の内包質量の解析解と累積台形積分。計算は $0 \le R \le 10R_d$ を 400 区間に分けて行い、図には $R \le 8R_d$ の範囲を示す。質量は円盤の全質量 M_d で規格化している。

積分誤差と計算領域の外にある質量

一方、解析解でも $M_{\rm exact}(<R_{\rm max}) < M_d$ である。その差は数値積分の誤差ではなく、計算領域の外にある質量であり、式(A.10)から

$$\frac{M_d - M_{\rm exact}(<R_{\rm max})}{M_d} = e^{-R_{\rm max}/R_d}\left(1 + \frac{R_{\rm max}}{R_d}\right) \tag{A.15}$$

と求められる。$R_{\rm max}$ を固定したまま格子を細かくすると、数値解はその上端までの解析解へ近づくが、領域外の質量は変わらない。全質量 M_d を有限半径までの積分で近似する場合には、積分の離散化誤差とは別に、この寄与も考える必要がある。

A.4 累積台形則の格子収束

一つの格子で解析解と重なるだけでは、誤差が期待した次数で減るかは分からない。計算領域を $R_{\rm max} = 10R_d$ に固定し、全格子点での誤差を

$$\epsilon_M \equiv \max_{0\le i\le N} \frac{|M_{\rm num}(<R_i) - M_{\rm exact}(<R_i)|}{M_d} \tag{A.16}$$

で測る。これは全質量 M_d で規格化した最大絶対誤差であり、各点の内包質量に対する相対誤差ではない。一定の M_d を分母に使うことで、内包質量が 0 の中心も評価に含められる。

第 1 章で学んだ台形則の誤差評価から、滑らかな被積分関数を固定区間で積分すると、離散化誤差が支配する範囲では

$$\epsilon_M \simeq C \left(\frac{\Delta R}{R_d} \right)^p, \qquad p = 2 \tag{A.17}$$

が期待される。ここで C は格子幅によらない係数である。累積積分でも、各上端まで同じ台形則で積分するため、この誤差評価を使える。R_d を固定しているので、$\log \Delta R$ に対する $\log \epsilon_M$ の傾きも p となる。

検証手順｜累積台形則の格子収束

1. 計算領域と M_d, R_d を固定し、区間数を $N = 25, 50, 100, 200, 400, 800$ と変える。
2. 各格子で累積積分と解析解を比較し、式(A.16)の最大誤差を測る。
3. 両対数で誤差が規則的に減少する範囲を確かめ、その傾きを 2 と比較する。

Notebook A ｜累積台形則の格子収束

`exponential_mass_error` は 1 つの格子の幅と最大誤差を返す。区間数を 2 倍ずつ増やし、格子幅を半分にして計算を繰り返す。

対応セル（計算部分）

```
# 1 つの格子で内包質量の最大誤差を測る
def exponential_mass_error(N, R_max=10.0, Md=1.0, Rd=1.0):
    R = np.linspace(0.0, R_max, N + 1)
    dMdR = dMdR_exponential(R, Md, Rd)
    M_num = cumulative_trapezoid_manual(R, dMdR)
    M_exact = mass_exponential_exact(R, Md, Rd)
    dR = R[1] - R[0]
    err = np.max(np.abs(M_num - M_exact)) / Md
    return dR, err

# 領域とモデルを固定し、区間数を 2 倍ずつ増やす
N_values = np.array([25, 50, 100, 200, 400, 800])
dR_values = []
mass_errors = []
for N_i in N_values:
    dR_i, err_i = exponential_mass_error(int(N_i), 10.0 * Rd, Md, Rd)
    dR_values.append(dR_i)
    mass_errors.append(err_i)

dR_values = np.array(dR_values)
mass_errors = np.array(mass_errors)

# log(error)-log(dR) の直線の傾きが収束次数 p
order_mass = np.polyfit(np.log(dR_values), np.log(mass_errors), 1)[0]
```

図A.3では、6 格子の誤差がほぼ 2 次の傾きで減少している。これらすべての点を両対数で直線近似すると、

$$p = 1.998 \tag{A.18}$$

を得る。球対称系から円盤へ題材を変えても、累積台形則の約 2 次の収束を確認できる。格子をさらに細かくする場合も、当てはめには離散化誤差が規則的に減少する範囲を使い、丸め誤差で停滞した点や誤差が 0 の点は含めない。

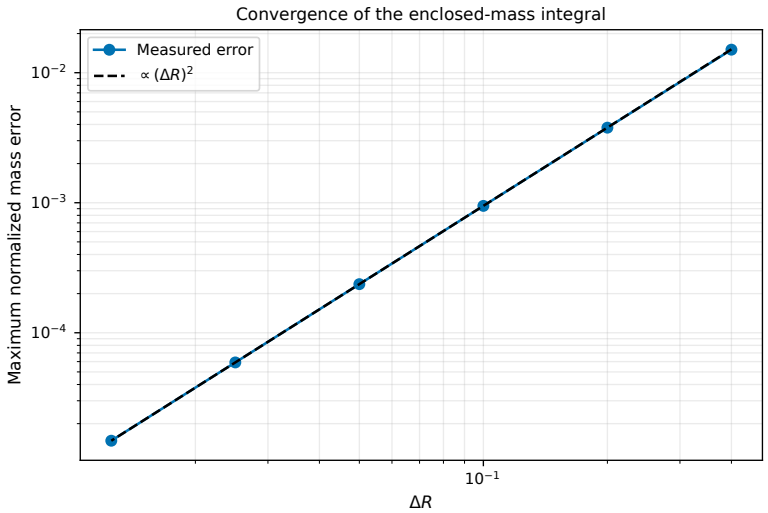


図 A.3　指数円盤の累積台形積分の格子収束。縦軸は式(A.16)の誤差、横軸は $R_d = 1$ の単位での格子幅である。破線は最も細かい格子での誤差を基準にした $(\Delta R)^2$ の線である。

A.5 指数円盤の回転曲線と球対称近似

ここまでは、同じ積分問題に対する数値解と解析解の差を調べた。次に、物理モデルの仮定によって結果がどう変わるかを見る。第 1 章の球対称系では、Newton の球殻定理から

$$v_{\mathrm{circ}}^2(r) = \frac{GM(<r)}{r} \tag{A.19}$$

と書けた。しかし円盤面内では、半径 R より外側の質量も重力に寄与する。球殻のように外側の質量の寄与が打ち消し合うわけではないため、その半径より内側の総質量だけでは円運動速度を決められない。

薄い指数円盤の回転曲線は、**修正 Bessel 関数（modified Bessel functions）**を用いて

$$v_{\mathrm{disk}}^2(R) = 4\pi G\Sigma_0 R_d\, y^2 \left[I_0(y)K_0(y) - I_1(y)K_1(y)\right], \qquad y = \frac{R}{2R_d} \tag{A.20}$$

と表せる。I_n と K_n は、それぞれ第 1 種・第 2 種の n 次修正 Bessel 関数であり、ここでは $n = 0, 1$ を使う。この式は、Ryden の式 (6.149) にある薄い円盤の Hankel 積分を指数円盤について評価した形に対応する。Ryden の図 6.6 には、その回転曲線が示されている [1]。本付録では式(A.20)の導出には立ち入らず、比較に使う解析式として扱う。Notebook A では `scipy.special.iv` と `kv` で関数値を求め、Hankel 積分を直接数値積分する方法は発展課題に残す。

比較曲線は、同じ内包質量を球対称分布のものとみなして

$$v_{\mathrm{sph}}^2(R) \equiv \frac{GM_{\mathrm{exact}}(<R)}{R} \tag{A.21}$$

と定義する。内包質量には数値積分値ではなく解析解を使い、累積台形則の誤差を比較に混ぜない。二つの曲線の違いは、数値積分法ではなく、質量分布を球対称とみなすことによって生じる。

Notebook A ｜指数円盤と球対称近似の比較

$10^{-4}R_d \le R \le 6R_d$ に等間隔の 4000 点を置き、二つの解析式を同じ半径で評価する。`np.argmax` で、それぞれの最大値をとる格子点を選ぶ。作図と出力の整形は Notebook A に置く。

対応セル（計算部分）

```python
from scipy.special import iv, kv

# 回転曲線を評価する半径と y = R/(2 Rd)
R_rot = np.linspace(1.0e-4 * Rd, 6.0 * Rd, 4000)
y = R_rot / (2.0 * Rd)

# 修正 Bessel 関数による解析式を評価する
Bessel_term = iv(0, y) * kv(0, y) - iv(1, y) * kv(1, y)
vc2_disk = 4.0 * np.pi * G * Sigma0 * Rd * y**2 * Bessel_term

# 解析的な内包質量から球対称の比較曲線を作る
M_rot = mass_exponential_exact(R_rot, Md, Rd)
vc2_sph = G * M_rot / R_rot

# それぞれの最大値をとる格子点を求める
i_disk = np.argmax(vc2_disk)
i_sph = np.argmax(vc2_sph)
Rpeak_disk = R_rot[i_disk]
Rpeak_sph = R_rot[i_sph]
vc2peak_disk = vc2_disk[i_disk]
vc2peak_sph = vc2_sph[i_sph]
```

表A.1に結果を示す。R_{peak} は最大値をとる格子点の半径、v_{peak} はそこでの円運動速度である。補間や最適化によって連続的な最大位置を求めた値ではない。球対称近似では、最大位置が内側へずれ、最大速度も小さくなる。

表 A.1　指数円盤の回転曲線と、同じ内包質量を用いた球対称近似の比較。速度の二乗は GM_d/R_d で規格化している。

重力場のモデル	R_{peak}/R_d	$v^2_{\mathrm{peak}}/(GM_d/R_d)$
指数円盤	2.150	0.3870
球対称近似	1.793	0.2984

この違いは、回転曲線を評価する点を増やしてもなくならない。数値積分が高い精度でできることと、物理の関係式が対象に適用できることは、分けて判断する必要がある。図A.4の実線と破線は、両曲線を共通の値 $v^2_{\mathrm{max,disk}}$ で割って示す。各曲線をそれぞれの最大値で割ったものではないので、振幅の違いも比較できる。

A.6 Mestel 円盤と平坦な回転曲線

一つのモデルで関係式が成り立っても、他のモデルで使えるとは限らない。面密度が $\Sigma(R) \propto 1/R$ の Mestel 円盤では、$2\pi R\Sigma(R)$ が一定なので、

$$M(<R) \propto R, \qquad \frac{GM(<R)}{R} = \text{const.} \tag{A.22}$$

となる。無限に広がる Mestel 円盤では、円盤の重力場から求めた円運動速度も一定であり、$v^2_{\mathrm{circ}} = GM(<R)/R$ が成立する。ただし、これはこの分布に特有の結果であり、一般の円盤に使える関係ではない。

Ryden の式 (6.150) では有限半径で打ち切ったモデルを導入し、式 (6.158)–(6.159) で無限円盤との違いを説明している [1]。有限半径で打ち切ると、外側の質量の寄与が変わり、端の影響によって同じ等式は厳密には成り立たない。本付録と Notebook A では有限 Mestel 円盤の重力場は計算せず、無限円盤の平坦な回転曲線を形の比較にだけ使う。

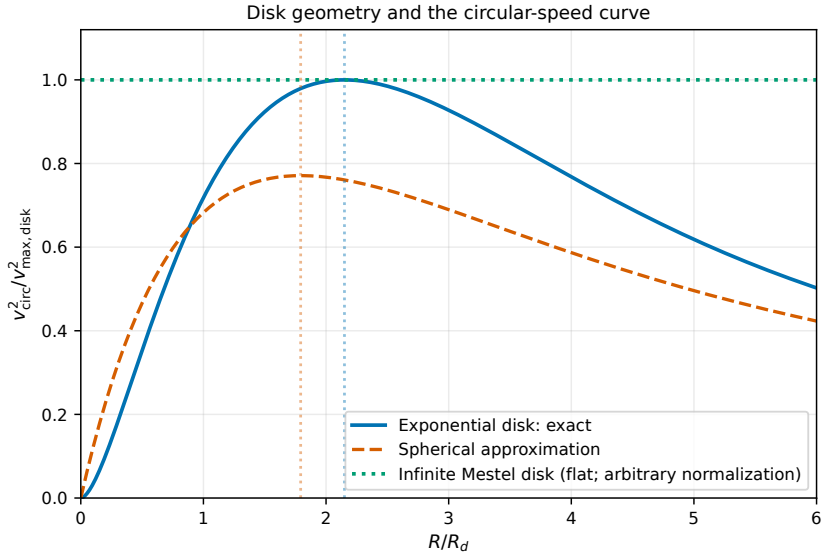


図 A.4　指数円盤の回転曲線と球対称近似。縦軸は速度の二乗を指数円盤の最大値 $v_{\rm max,disk}^2$ で割ったもので、表A.1の規格化とは異なる。Mestel 円盤の点線は、無限円盤の平坦な形を示す任意規格化の線である。

数値計算上の注意｜ Mestel 円盤の比較線は任意規格化である

図A.4の点線は、指数円盤と同じ全質量や長さスケールから計算したものではない。平坦な回転曲線の形を示すため、指数円盤の最大値に合わせている。有限 Mestel 円盤の計算結果や、指数円盤と振幅まで物理的に対応する曲線として解釈してはいけない。

付録 B　軸対称 Poisson 方程式の数値解法

第 2 章では、球対称 Poisson 方程式を 1 次元の境界値問題として解き、有限差分法の精度を検証した。本付録では、Ryden §6.5 の軸対称系を題材として、同じ中心差分の考え方を (R, z) の 2 次元格子へ広げる [1]。2 次元 Poisson 方程式の数値解法は、本書で加えた数値計算上の発展である。近傍点を結ぶ差分式から疎行列を構成し、境界条件を組み込んで解く手順を学ぶ。

検証には、密度とポテンシャルの解析式が分かる Miyamoto–Nagai モデルを使う。対称軸の処理と格子点の番号付けを確認したうえで、解析解との比較、残差、格子収束試験によって結果を判断する。本付録は第 2 章の発展内容であり、第 3 章や付録Cの軌道計算の前提ではない。

本付録で学ぶこと

- 軸対称 Poisson 方程式を、計算領域と境界条件を定めた 2 次元の境界値問題として設定できる。
- 中心差分を 2 次元へ拡張し、対称軸での滑らかさを使って軸上の差分式を導ける。
- 格子点と未知数番号を対応させ、既知の境界値を右辺に組み込んだ疎行列の連立一次方程式を構成できる。
- 解析解との比較と格子収束試験から、2 次元有限差分解の誤差と収束次数を評価できる。
- 残差とポテンシャルの誤差を区別し、境界条件の近似誤差を除いて離散化誤差を検証する理由を説明できる。

数値実験は、単独で実行できる Notebook B で行う。本文では差分式と短い実装の対応を示し、完全なソルバーと作図は Notebook に置く。数値計算法は幸谷第 8 章と §14.2.3 の疎行列・2 次元 Poisson 方程式の説明を参考にする [5]。

B.1 軸対称 Poisson 方程式と検証用モデル

円筒座標 (R, ϕ, z) で軸対称性 $\partial/\partial\phi = 0$ を仮定すると、Poisson 方程式は

$$\boxed{\frac{\partial^2\Phi}{\partial R^2} + \frac{1}{R}\frac{\partial\Phi}{\partial R} + \frac{\partial^2\Phi}{\partial z^2} = 4\pi G\rho(R, z)} \tag{B.1}$$

となる。球対称の場合と異なり、R だけでなく z による変化も求める必要がある。既知の密度 $\rho(R, z)$ を右辺に与え、境界条件と合わせて未知の $\Phi(R, z)$ を決める。

検証用の Miyamoto–Nagai モデル

密度とポテンシャルの両方が解析的に分かれば、密度を入力として得た数値解を、ポテンシャルの解析式と比較できる。ここでは、全質量を M、長さのパラメータを $a > 0$、$b > 0$ とする Miyamoto–Nagai モデルを使う。

$$B \equiv \sqrt{z^2 + b^2} \tag{B.2}$$

とおくと、ポテンシャルは

$$\Phi(R, z) = -\frac{GM}{\sqrt{R^2 + (a + B)^2}} \tag{B.3}$$

であり、対応する密度は

$$\rho(R, z) = \frac{b^2 M}{4\pi}\,\frac{aR^2 + (a + 3B)(a + B)^2}{B^3\left[R^2 + (a + B)^2\right]^{5/2}} \tag{B.4}$$

である。$b > 0$ とすることで、対称軸と円盤面を含めて滑らかなモデルを扱う。

Notebook B の既定値は

$$G = M = a = 1, \qquad b = 0.25 \tag{B.5}$$

である。以下では、長さの基準を a、ポテンシャルの基準を GM/a として結果を示す。図B.1に、この設定での密度分布を示す。

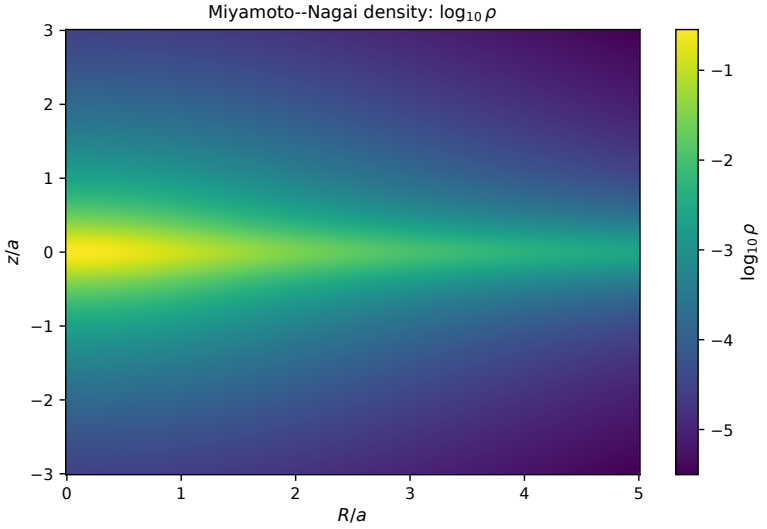


図 B.1　検証用の Miyamoto–Nagai モデルの密度分布。式(B.5)の設定で $\log_{10}\rho$ を表示した。密度は R と z の両方に依存する。この図の描画用格子と、Poisson 方程式を解く格子は別に用意する。

B.2 2 次元有限差分

計算領域を $0 \le R \le R_{\max}$、$-Z_{\max} \le z \le Z_{\max}$ とする。R 方向を N_R 区間、z 方向を N_Z 区間に分け、

$$\Delta R = \frac{R_{\max}}{N_R}, \qquad \Delta z = \frac{2Z_{\max}}{N_Z} \tag{B.6}$$

とおく。両端を含む格子点は

$$R_i = i\Delta R \quad (i = 0, \dots, N_R), \qquad z_j = -Z_{\max} + j\Delta z \quad (j = 0, \dots, N_Z) \tag{B.7}$$

である。N_R, N_Z は格子点数ではなく区間数であり、全格子点数は $(N_R+1)(N_Z+1)$ となる。格子点上の関数値を $\Phi_{i,j} = \Phi(R_i, z_j)$ と書く。

第 2 章の 3 点ステンシル（定義 2.2）を各方向へ適用する。$R_i > 0$ の内部点では、中心点と上下左右の 4 点を使う 5 点ステンシルになる（図B.2）。

$R_i > 0$ の内部点

第 2 章で導いた中心差分を R 方向と z 方向に用いると、

$$\left.\frac{\partial^2\Phi}{\partial R^2}\right|_{i,j} = \frac{\Phi_{i+1,j} - 2\Phi_{i,j} + \Phi_{i-1,j}}{(\Delta R)^2} + O((\Delta R)^2), \tag{B.8}$$

$$\left.\frac{\partial\Phi}{\partial R}\right|_{i,j} = \frac{\Phi_{i+1,j} - \Phi_{i-1,j}}{2\Delta R} + O((\Delta R)^2), \tag{B.9}$$

$$\left.\frac{\partial^2\Phi}{\partial z^2}\right|_{i,j} = \frac{\Phi_{i,j+1} - 2\Phi_{i,j} + \Phi_{i,j-1}}{(\Delta z)^2} + O((\Delta z)^2) \tag{B.10}$$

である。式(B.1)へ代入し、打ち切り誤差を除くと、解くべき差分方程式は

$$c_-\Phi_{i-1,j} + c_0\Phi_{i,j} + c_+\Phi_{i+1,j} + c_z\Phi_{i,j-1} + c_z\Phi_{i,j+1} = 4\pi G\rho_{i,j} \tag{B.11}$$

となる。以下の差分方程式で求める $\Phi_{i,j}$ は、連続なポテンシャルの数値近似である。各点に掛かる係数をまとめると、

$$c_- = \frac{1}{(\Delta R)^2} - \frac{1}{2R_i\Delta R}, \tag{B.12}$$

$$c_+ = \frac{1}{(\Delta R)^2} + \frac{1}{2R_i\Delta R}, \tag{B.13}$$

$$c_z = \frac{1}{(\Delta z)^2}, \tag{B.14}$$

$$c_0 = -\frac{2}{(\Delta R)^2} - \frac{2}{(\Delta z)^2} \tag{B.15}$$

である。動径方向では $(1/R)\partial\Phi/\partial R$ の項があるため、内側と外側の係数 c_-, c_+ は異なる。

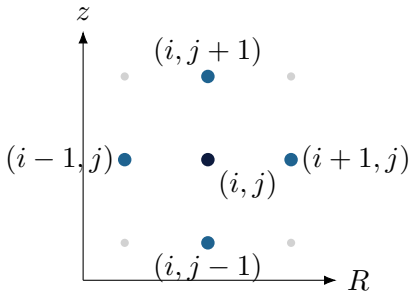


図 B.2 内部格子点の 5 点ステンシル。差分方程式は中心点と上下左右の近傍点だけを結合する。

Listing B.1 では、まず z 方向の中心係数を設定し、R 方向の寄与を加える。`cm`, `cp`, `cz`, `center` が、それぞれ c_-, c_+, c_z, c_0 に対応する。

Listing B.1 内部点の差分係数

```
cz = 1.0 / dz**2
center = -2.0 / dz**2
cp = 1.0 / dR**2 + 1.0 / (2.0 * Ri * dR)
cm = 1.0 / dR**2 - 1.0 / (2.0 * Ri * dR)
center += -2.0 / dR**2
```

対称軸 $R = 0$ の処理

式(B.1)の $(1/R)\partial\Phi/\partial R$ を、そのまま $R = 0$ で計算することはできない。軸上でも滑らかな軸対称ポテンシャルでは

$$\left.\frac{\partial\Phi}{\partial R}\right|_{R=0} = 0 \tag{B.16}$$

なので、この項は $0/0$ の形になる。軸へ近づく極限を先に求め、有限な式に置き換える。

z を固定し、対称軸を横切る直線上に符号付き座標 ℓ を取ると、$R = |\ell|$ である。軸対称性によって ℓ と $-\ell$ の値は等しい。第 2 章 §2.4と同じく、滑らかな関数の Taylor 展開には偶数次の項だけが残るため、

$$\Phi(R, z) = \Phi(0, z) + \frac{1}{2}\Phi_{RR}(0, z)R^2 + O(R^4) \tag{B.17}$$

と書ける。ここで Φ_{RR} は R による二階微分である。R で微分して R で割ると、

$$\frac{1}{R}\frac{\partial\Phi}{\partial R} = \Phi_{RR}(0, z) + O(R^2) \tag{B.18}$$

なので、動径方向の二つの項を合わせた極限は

$$\left[\frac{\partial^2\Phi}{\partial R^2}+\frac{1}{R}\frac{\partial\Phi}{\partial R}\right]_{R=0}=2\Phi_{RR}(0,z) \tag{B.19}$$

となる。

さらに、対称性を用いて仮想点を $\Phi_{-1,j}=\Phi_{1,j}$ とすれば、

$$\Phi_{RR}(0,z_j)=\frac{\Phi_{1,j}-2\Phi_{0,j}+\Phi_{-1,j}}{(\Delta R)^2}+O((\Delta R)^2)=\frac{2(\Phi_{1,j}-\Phi_{0,j})}{(\Delta R)^2}+O((\Delta R)^2) \tag{B.20}$$

である。したがって、軸上で動径方向の演算子全体を近似する式は

$$\boxed{2\Phi_{RR}(0,z_j)\simeq\frac{4(\Phi_{1,j}-\Phi_{0,j})}{(\Delta R)^2}} \tag{B.21}$$

となる。この差分も、軸上での滑らかさのもとで 2 次精度である。z 方向の中心差分を加えた軸上の差分方程式は

$$\left[-\frac{4}{(\Delta R)^2}-\frac{2}{(\Delta z)^2}\right]\Phi_{0,j}+\frac{4}{(\Delta R)^2}\Phi_{1,j} + c_z\Phi_{0,j-1}+c_z\Phi_{0,j+1}=4\pi G\rho_{0,j} \tag{B.22}$$

である。

Listing B.2 は、Listing B.1 の動径方向の係数を軸上のものに置き換えたコードである。`center` は、ここでも z 方向の寄与から初期化する。仮想点は対称性によって消去したので、負の R の配列は作らない。

Listing B.2　対称軸 $R=0$ の差分係数

```
cz = 1.0 / dz**2
center = -2.0 / dz**2
cp = 4.0 / dR**2
center += -4.0 / dR**2
```

数値計算上の注意｜$1/R$ をそのまま評価しない

ここでの $0/0$ は、滑らかなポテンシャルを円筒座標で表したことによる見かけの特異点である。R を小さな正数に置き換えるのではなく、対称軸近傍の展開から得た式(B.21)を使う。軸対称性だけでなく、軸上で滑らかであるという仮定も必要である。

B.3 2 次元格子での連立一次方程式の組み立て

第 2 章と同じく、差分式を連立一次方程式

$$A\boldsymbol{\Phi}=\boldsymbol{b} \tag{B.23}$$

にまとめる。ただし、2 次元配列のままでは行列の列番号に対応しないため、未知格子点 (i,j) に 1 次元の番号を付ける。右辺ベクトル $\boldsymbol{b}$ は、密度の項と既知の境界値から作る。モデルの長さ b とは別の量である。

既知の境界と未知格子点

$R=R_{\max}$ と $z=\pm Z_{\max}$ には、式(B.3)の解析値を与える。この 3 辺の値は未知数に含めない。一方、対称軸 $R=0$ のポテンシャル値は既知ではなく、式(B.22)で解く未知数に含める。したがって、未知格子点の添字は

$$i=0,\ldots,N_R-1,\qquad j=1,\ldots,N_Z-1$$

である。z 方向の未知点数を

$$n_z = N_Z - 1 \tag{B.24}$$

とし、各 i について j の小さい順に並べると、

$$\boxed{p = i\,n_z + (j-1)} \tag{B.25}$$

で 1 次元番号へ写せる。p は 0 から $N_R n_z - 1$ までの番号であり、$\boldsymbol{\Phi}$ の第 p 成分が $\Phi_{i,j}$ に対応する。$j = 0$ は境界点なので、最初の未知点 $j = 1$ がオフセット 0 になる。

Listing B.3　2 次元格子の 1 次元番号付け

```
nZ_u = NZ - 1

def idx(i, j):
    return i * nZ_u + (j - 1)
```

Notebook B では、完全なソルバーの前に 16 個の未知点の番号表を表示する。これは図B.3と同じ $N_R = 4, N_Z = 5$ の例である。逆の対応は p を n_z で割った商と余りから求められる。短い実装確認も同じセルの前に置いている。

非零係数の記録と境界値の移項

定義 B.1　疎行列

要素の大部分が 0 である行列を**疎行列（sparse matrix）**と呼ぶ。有限差分法では各格子点が近傍点だけと結合するため、係数行列は典型的に疎行列になる。

5 点ステンシルでは、一つの行に入る非零係数は高々 5 個である。そこで、0 も含めた全要素ではなく、非零要素の行番号・列番号・値だけを記録する。Notebook の `rows[k]`, `cols[k]`, `data[k]` が、その三つ組に対応する。

格子点 (i, j) の差分方程式を行番号 $p =$ `idx(i,j)` に割り当てる。例えば、外側の近傍点 $(i+1, j)$ が未知点なら、その係数 c_+ を列番号 $q =$ `idx(i+1,j)` に入れる。一方、$i = N_R - 1$ なら近傍点は既知の外側境界なので、行列の列を作らず、

$$b_p = 4\pi G \rho_{i,j} - c_+ \Phi^{\mathrm{exact}}_{N_R,j} \tag{B.26}$$

のように右辺へ移項する。z 方向にも既知の境界点があれば、その寄与も同じ符号で差し引く。

Listing B.4 は、$R_i > 0$ の点における外側近傍の処理である。`rhs[p]` は密度の項 $4\pi G\rho_{i,j}$ で初期化しておき、既知の境界値の寄与を順に差し引く。

Listing B.4　未知の近傍点と既知の境界点の扱い

```
if i + 1 == NR:
    rhs[p] -= cp * phi_mn(R[NR], z[j], G, M, a, b)
else:
    rows.append(p)
    cols.append(idx(i + 1, j))
    data.append(cp)
```

同様の操作を各近傍点について行い、中心点の係数を `rows` と `cols` がともに p の要素として加える。集めた三つ組から CSR 形式の疎行列を作り、`spsolve` に行列と右辺を渡す（Listing B.5）。CSR は非零要素を行ごとにまとめて保持する形式であり、ここでは SciPy の関数に変換を任せる [5]。

Listing B.5　疎行列の構成と求解

```
shape = (n_unknown, n_unknown)
A = sparse.csr_matrix((data, (rows, cols)), shape=shape)
sol = spsolve(A, rhs)
```

`sol` は 1 次元の解ベクトルである。既知の境界値を 2 次元配列 `Phi` に設定し、未知点へ `Phi[i, j] = sol[idx(i, j)]` を代入すれば、格子全体のポテンシャルに戻せる。疎行列の直接法における消去と代入、および消去中に非零要素が増えるフィルインについては、付録Dを参照する。

計算手順｜軸対称 2 次元 Poisson 方程式を解く

1. 格子、密度、既知の境界値を用意する。
2. 未知格子点 (i, j) を 1 次元番号 p に対応させる。
3. 内部点と対称軸の差分式を使い分け、未知点の係数を記録する。
4. 既知の境界値に掛かる項は右辺へ移項する。
5. 記録した非零要素から疎行列を作り、連立一次方程式を解く。
6. 解ベクトルを 2 次元配列に戻し、既知の境界値と合わせる。

数値計算上の注意｜解析的な境界値を使う理由

> 本付録では、有限の計算領域の 3 辺に解析値を与え、境界条件の近似誤差を除く。領域の外にも密度は分布しており、外側の質量を捨てたモデルを解くわけではない。解析的な境界値には、その寄与も含まれている。これにより、解析解との差から差分による誤差を調べられる。ただし、丸め誤差や連立一次方程式の求解に伴う誤差までなくなるわけではない。

B.4 疎行列の大きさと構造

Notebook B の最初の数値解は、

$$N_R = 40, \qquad N_Z = 64, \qquad R_{\max} = 5a, \qquad Z_{\max} = 4a \tag{B.27}$$

で求める。このとき $\Delta R = \Delta z = 0.125a$ である。未知数の個数は、対称軸を含む N_R 個の動径点と、上下端を除いた $N_Z - 1$ 個の鉛直点の積

$$N_{\mathrm{unknown}} = N_R(N_Z - 1) = 2520 \tag{B.28}$$

となる。密行列なら $N_{\mathrm{unknown}}^2 = 2520^2$ 個の要素を保存するが、実際の非零要素数は

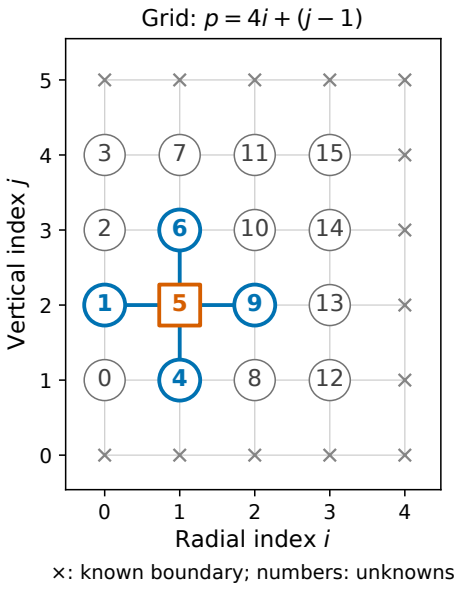
$$N_{\mathrm{nz}} = 12394 \tag{B.29}$$

であり、非零要素率は

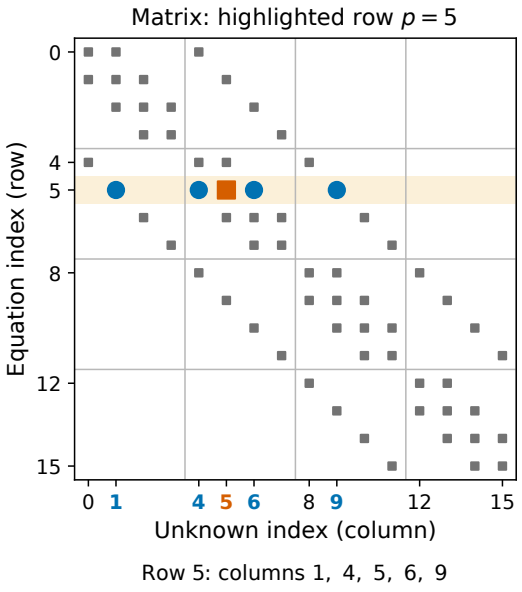
$$\frac{N_{\mathrm{nz}}}{N_{\mathrm{unknown}}^2} = 1.952 \times 10^{-3} \tag{B.30}$$

にすぎない。Notebook では、この個数を疎行列の `nnz` から直接確認する。

図B.3では、非零要素の位置を読み取るために小さい格子を使う。式(B.25)から、z 方向の近傍点の番号は $p \pm 1$、R 方向は $p \pm n_z$ となる。例えば $(i, j) = (1, 2)$ は $p = 5$ に対応し、近傍点を含む列番号は $1, 4, 5, 6, 9$ である。ただし、これらは近傍点が未知点の場合の対応であり、既知の境界点は列に含めない。小さい行列も、数値解に使うものと同じソルバーから生成している。



(a) Grid numbering



(b) Nonzero matrix entries

図 B.3 小さい格子と疎行列の対応。説明用に $N_R = 4, N_Z = 5$（$n_z = 4$、未知数 16 個）とした。左図の数字は未知数番号 p、× は既知の境界点である。赤い四角で示す $(i, j) = (1, 2)$ は $p = 5$ に対応し、青い丸の近傍点を含めた 5 点は、右図の行番号 5（0 始まり）の列番号 $1, 4, 5, 6, 9$ に対応する。右図の空白は零要素、灰色はその他の非零要素である。

番号が隣り合っていても、格子上で隣り合うとは限らない。例えば、この図の $p = 3$ と $p = 4$ は動径点の切り替わりに当たり、z 方向の近傍点ではない。係数を入れる前に (i, j) の範囲で境界を判定するのは、このような誤った結合を避けるためでもある。

数値計算上の注意｜行列の記憶量と求解の計算量を分ける

密行列の要素数は未知数の 2 乗で増えるが、ここで組み立てる疎行列の非零要素数は未知数にほぼ比例する。ただし、これだけで求解の計算時間も比例すると結論することはできない。疎行列の直接法ではフィルインも生じるため、分解中の記憶量と計算量は元の非零要素数だけでは決まらない。

補足｜シミュレーション天文学で使う重力計算法

本付録では、Poisson 方程式を有限差分で離散化し、疎行列の連立一次方程式を解いた。これは境界値問題の設定から数値解の検証までを学ぶための一例であり、すべての重力計算で同じ手順を使うわけではない。

- **滑らかな固定重力場：**複雑な質量分布の重力場には、角度依存性を球面調和関数で表す**多重極展開（multipole expansion）**や、密度とポテンシャルを対応する基底関数の和で表す**基底関数展開法（basis-function expansion）**などを使える。あらかじめ構成した展開式や補間関数を微分すれば、任意の位置で加速度を評価できる。固定場での軌道積分中に、空間上の Poisson 方程式を解き直す必要はない。多重極展開は、第 2 章で紹介した遠方の境界値だけでなく、質量分布の内部の重力場を表すことにも用いられる。
- **格子上の自己重力計算：**一様格子では、**高速 Fourier 変換（fast Fourier transform; FFT）**を利用して Poisson 方程式を解く方法があり、周期境界条件と組み合わせやすい。粗い格子と細かい格子を組み合わせて離散方程式を解く**多重格子法（multigrid**

method）は、適合格子細分化法（adaptive mesh refinement; AMR）を用いる計算でも利用される。有限差分法は方程式の**離散化**、多重格子法は離散方程式の**求解**の方法であり、両者は組み合わせて使える。FFT も有限差分法と対立するものではなく、有限差分で得た離散方程式を解くために利用する場合がある。

- **粒子分布からの重力計算：ツリー法（tree method）**では、遠方の粒子群からの重力を、多重極展開を使ってまとめて近似する。**PM 法（particle-mesh method）**では、粒子の質量を格子へ割り当て、格子上でポテンシャルと加速度を求め、加速度を粒子位置へ補間する。格子上のポテンシャルを差分して加速度を求める実装もあり、PM 法も有限差分法と排他的な方法ではない。重力を長距離成分と短距離成分に分け、前者を PM 法、後者をツリー法で計算する TreePM 法もある。

以上は用途に沿った例であり、排他的な分類ではない。基底関数展開を自己重力系の時間発展に用いることもできる。研究では質量分布、境界条件、必要な空間分解能、計算規模に応じて方法を選ぶ。方法が変わっても、参照解との比較や、格子幅・展開の項数などの近似の細かさを変えた検証は必要である。詳しくは Binney & Tremaine（2008）の §2.4・§2.8・§2.9[2]、松本ほか [9]、富阪ほか編 [8] を参照すること。

B.5 数値解と誤差

Notebook B の `solve_mn_poisson` は、ここまでの格子作成、係数行列と右辺の構成、求解、2 次元配列への復元を一つにまとめた関数である。同じ格子点で数値解と解析解を評価し、ポテンシャルの誤差と離散方程式の残差を別々に求める。

Notebook B ｜ 2 次元 Poisson 方程式の数値解

式(B.27)の領域と区間数で解く。`Phi_num[i,j]` に合わせ、`meshgrid` の `indexing="ij"` によって解析解の配列も第 1 軸を R、第 2 軸を z にする。

対応セル（計算部分）

```
# 計算領域と各方向の区間数を指定する
NR, NZ = 40, 64
Rmax = 5.0 * amn
Zmax = 4.0 * amn
Rmn, zmn, Phi_num, A_mn, rhs_mn, sol_mn = solve_mn_poisson(
    NR=NR, NZ=NZ, Rmax=Rmax, Zmax=Zmax, G=G, M=Mmn, a=amn, b=bmn)

# 数値解と同じ格子上で解析ポテンシャルを評価する
RRmn, ZZmn = np.meshgrid(Rmn, zmn, indexing="ij")
Phi_exact_mn = phi_mn(RRmn, ZZmn, G, Mmn, amn, bmn)

# 解の誤差と、離散方程式の残差を別々に測る
err_phi = np.max(np.abs(Phi_num - Phi_exact_mn)) / (G * Mmn / amn)
residual = A_mn @ sol_mn - rhs_mn
residual_max = np.max(np.abs(residual))
```

ここで用いる誤差は、全格子点での絶対誤差の最大値を、一定の基準量 GM/a で割った

$$\epsilon_\Phi = \frac{\max_{i,j} |\Phi_{\mathrm{num},i,j} - \Phi_{\mathrm{exact},i,j}|}{GM/a} = 4.260482 \times 10^{-3} \tag{B.31}$$

である。各点の $|\Phi_{\mathrm{exact}}|$ で割った相対誤差ではない。境界には解析値を設定しているため、境界点での差は 0 になる。数値ポテンシャルを図B.4、符号付きの誤差を図B.5に示す。

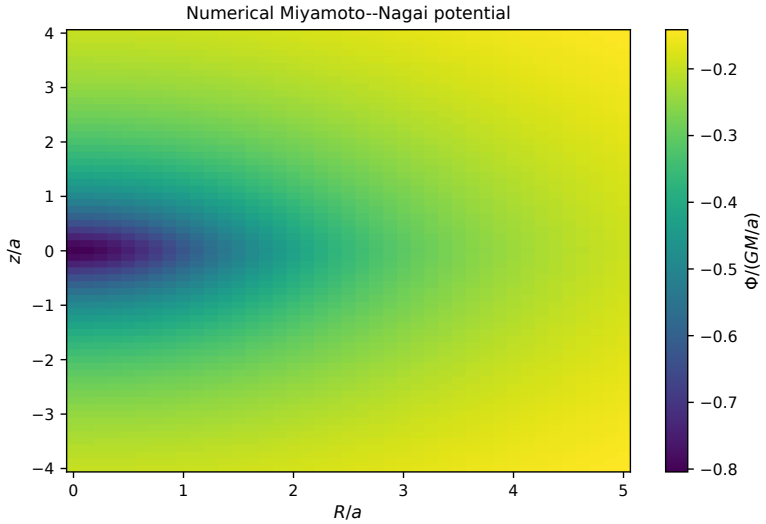


図 B.4　有限差分法で求めた Miyamoto–Nagai ポテンシャル。$N_R = 40$, $N_Z = 64$ とし、$R = R_{\max}$ と $z = \pm Z_{\max}$ には解析値を与えた。対称軸の値は差分方程式を解いて求めている。

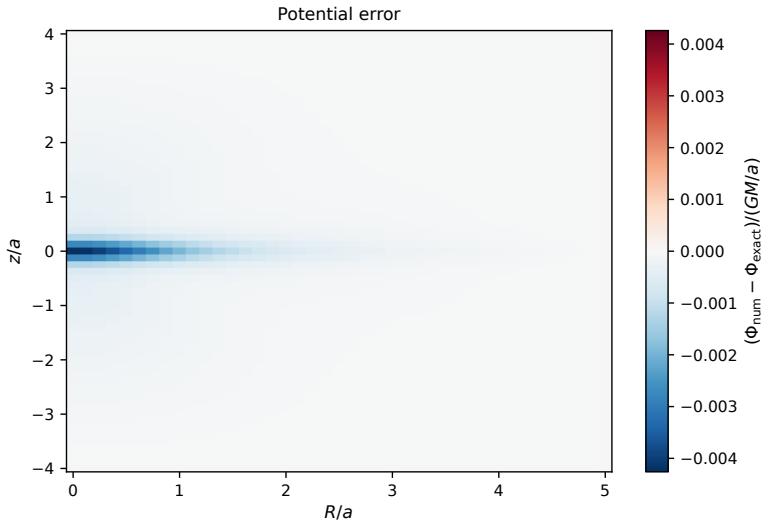


図 B.5　同じ格子上の数値解と解析解の差を GM/a で規格化したもの。符号付き誤差を、0 を中心とする対称なカラースケールで表示している。境界点での差は 0 である。

一方、未知点の解ベクトルについて計算した最大残差は

$$\max_p |(A\boldsymbol{\Phi} - \boldsymbol{b})_p| = 9.083446 \times 10^{-14} \tag{B.32}$$

である。これは、上で定義した差分係数を使い、$G = M = a = 1$ で計算した値である。残差は離散方程式の左右のずれであり、ポテンシャルの誤差とは単位も異なる。方程式全体を定数倍すれば残差も定数倍されるため、異なる規格化の残差をそのまま比較しない。

数値計算上の注意｜残差が小さくても離散化誤差は残る

第 2 章の残差の定義と同じく、ここで小さい残差が示すのは、数値解が組み立てた連立一次方程式をよく満たすことである。その方程式は連続な Poisson 方程式そのものではないため、解析解との差は残る。ポテンシャルの精度は、式(B.31)と格子収束試験で別に確認する。

B.6 2 次元 Poisson 方程式の格子収束

一つの格子で解と残差を確認した後は、領域・境界条件・モデルを固定して格子を細かくする。内部点と対称軸で使った差分は、滑らかな解に対して各方向で 2 次精度である。二つの格子幅を一定の比率で小さくすれば、離散化誤差が支配する十分細かい格子で

$$\epsilon_\Phi \propto h^2, \qquad h \equiv \max(\Delta R, \Delta z) \tag{B.33}$$

が期待される。ただし、第 2 章で述べたように、差分公式の打ち切り誤差の次数だけで数値解全体の収束が保証されるわけではない。ここでも、境界を含むソルバー全体の結果から次数を測定する。

検証手順｜ 2 次元格子の収束試験

1. $R_{\max} = 5a$, $Z_{\max} = 4a$、モデルのパラメータ、解析的な境界条件を固定する。
2. N_R と N_Z を同じ比率で増やし、両方向の格子幅を小さくする。
3. 各格子の数値解を同じ格子上の解析解と比較し、ϵ_Φ を計算する。
4. 両対数表示の傾きから収束次数を測定し、期待する 2 次収束と比較する。

Notebook B では $(N_R, N_Z) = (40, 64), (60, 96), (80, 128), (100, 160)$ とする。この組では常に $\Delta R = \Delta z$ であり、h/a は $0.125, 0.08333\ldots, 0.0625, 0.05$ となる。R 方向だけを細かくして Δz を固定する比較とは異なる。

Notebook B ｜ 2 次元 Poisson 方程式の格子収束

各格子で得た ϵ_Φ と h を配列に保存する。`polyfit` では、この 4 格子の両対数データに直線を当てはめ、その傾きを収束次数とする。

対応セル（計算部分）

```
# 領域とモデルを固定し、両方向の区間数を同じ比率で増やす
grid_list = [(40, 64), (60, 96), (80, 128), (100, 160)]
h_list = []
err_list = []

for NR_i, NZ_i in grid_list:
    R_i, z_i, Phi_i, _, _, _ = solve_mn_poisson(
        NR=NR_i, NZ=NZ_i, Rmax=Rmax, Zmax=Zmax,
        G=G, M=Mmn, a=amn, b=bmn)

    # 同じ格子上の解析解から、規格化した最大絶対誤差を測る
    RR_i, ZZ_i = np.meshgrid(R_i, z_i, indexing="ij")
    Phi_ex_i = phi_mn(RR_i, ZZ_i, G, Mmn, amn, bmn)
    h_i = max(R_i[1] - R_i[0], z_i[1] - z_i[0])
    err_i = np.max(np.abs(Phi_i - Phi_ex_i)) / (G * Mmn / amn)
    h_list.append(h_i)
    err_list.append(err_i)
```

```
18
19 h_list = np.array(h_list)
20 err_list = np.array(err_list)
21
22 # epsilon_Phi が h^p に比例する範囲では、両対数の傾きが p
23 order_2d = np.polyfit(np.log(h_list), np.log(err_list), 1)[0]
```

この 4 格子から得られる次数は

$$p = 2.048 \tag{B.34}$$

であり、図B.6のように誤差はほぼ 2 次で減少する。これは、ここで用いた滑らかなモデル、解析的な境界条件、格子幅の範囲に対する実測値である。別の密度分布や境界条件へ変更する場合も、同じ検証を繰り返す必要がある。

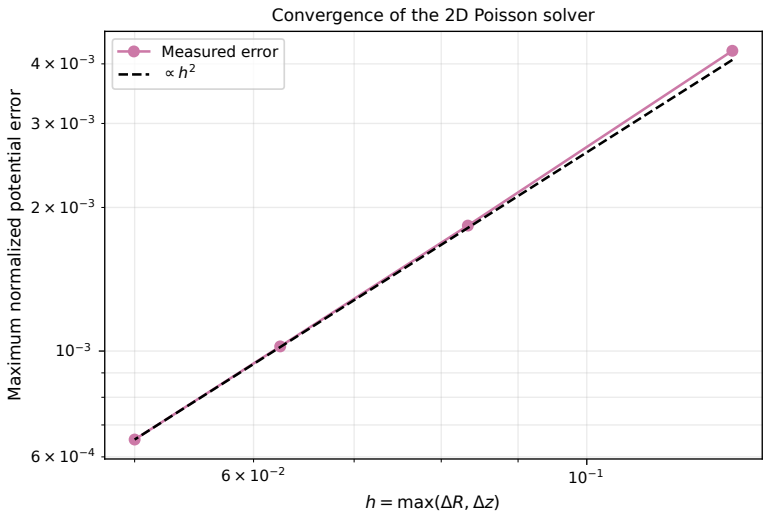


図 B.6　領域と境界条件を固定した 2 次元 Poisson 方程式の格子収束。実線は規格化した最大絶対誤差、破線は最も細かい格子の誤差に合わせた h^2 の比較線である。4 格子から測定した収束次数は $p = 2.048$ となる。

付録 C　軸対称・非軸対称ポテンシャル中の軌道

第 3 章では、球対称な重力場の中で軌道を計算し、時間積分法を検証した。本付録では、加速度関数を差し替えて同じ積分法を再利用する。ただし、重力場を変えると、検証に使える保存量や比較の基準も変わる。Ryden §7.2–7.3[1] の軸対称・非軸対称ポテンシャルを題材として、数値軌道と近似解の比較、時間刻みを変えた収束試験、保存量による診断を学ぶ。本付録は第 3 章の発展であり、他の付録を先に読む必要はない。

本付録で学ぶこと

- 加速度関数と初期条件を設定し、第 3 章の時間積分法を異なる重力場へ再利用できる。
- ポテンシャルの対称性に応じて保存量を選び、数値軌道の診断に用いることができる。
- 厳密解が得られない問題で、小振幅近似を近似的な参照解として用い、数値軌道との差を評価できる。
- 時間刻みと初期振幅を別々に変え、時間積分の誤差と小振幅近似との差を区別できる。
- 非軸対称系の角運動量の変化を数値誤差と区別し、有限時間の軌道から判断できる範囲を説明できる。

数値実験は Notebook C で行う。積分関数は第 3 章のものを使い、本文では加速度の実装と検証方法を中心に説明する。

C.1 異なる重力場への時間積分法の適用

本付録のポテンシャルは時間に依存せず、計算する粒子の運動によっても変わらないものとする。重力加速度 $\boldsymbol{g}(\boldsymbol{x}) = -\nabla\Phi$ は位置だけで決まるため、第 3 章のキック–ドリフト–キック（KDK）形式のリープフロッグ法をそのまま使える。`acc_func(x)` は 1 点の位置配列を受け取り、同じ次元の加速度配列を返す関数である。積分器と重力場の計算を分けておけば、モデルごとに時間ループを書き直す必要はない。

円筒座標を (R, ϕ, z) とし、$R = \sqrt{x^2 + y^2}$、ϕ は方位角、z は対称軸方向の座標とする。ポテンシャルを R, z で表す場合も、Notebook C では Cartesian 座標の位置と速度を更新する。返される配列の形は `(nstep + 1, ndim)` で、初期状態を含む。半径や保存量は、この配列から各出力時刻で計算する。

第 3 章と同様に、比エネルギーと比角運動量の z 成分を

$$E = \frac{1}{2}|\boldsymbol{v}|^2 + \Phi(\boldsymbol{x}), \qquad L_z = xv_y - yv_x \tag{C.1}$$

とする。時間に依存しないポテンシャルでは E が保存する。また、$dL_z/dt = xg_y - yg_x$ は z 軸まわりのトルクなので、軸対称な力では 0 となり、L_z も保存する。したがって、数値的な E や L_z の変化を誤差とみなせるかどうかは、モデルの対称性に応じて判断する必要がある。以下では第 3 章と同様に、文脈が明らかな場合はこれらを単にエネルギー、角運動量と呼ぶ。

以下では解析的な加速度を用い、付録Bの数値 Poisson 解は入力しない。これにより、空間離散化や補間の誤差を混ぜずに、時間積分を検証する。

C.2 軸対称ポテンシャルとエピサイクル運動

対数ポテンシャルと加速度の実装

平坦な回転曲線を持つ対数ポテンシャル（logarithmic potential）

$$\Phi(R,z) = \frac{1}{2}v_0^2 \ln\left(\frac{R^2 + z^2/q^2}{\ell^2}\right) \tag{C.2}$$

を用いる。$v_0 > 0$ は速度のスケール、$q > 0$ は等ポテンシャル面の形を決める定数であり、$q < 1$ では扁平な形になる。ℓ は対数の引数を無次元にする固定の基準長さで、以下のコードでは $\ell = 1$ とする。この選択はポテンシャルの加法定数を定めるだけで、加速度には影響しない。原点ではポテンシャルと加速度が特異になるため、ここでは原点を通らない、ほぼ円軌道を扱う。

$R^2 = x^2 + y^2$ を代入して

$$D = x^2 + y^2 + \frac{z^2}{q^2} \tag{C.3}$$

と置く。例えば $\partial D/\partial x = 2x$、$\partial D/\partial z = 2z/q^2$ なので、

$$g_x = -\frac{v_0^2 x}{D}, \qquad g_y = -\frac{v_0^2 y}{D}, \qquad g_z = -\frac{v_0^2 z}{q^2 D} \tag{C.4}$$

となる。この 3 成分を 1 つの配列で返す関数が Listing C.1 である。引数 x は 3 成分の位置配列であり、式中のスカラー x とは区別する。

Listing C.1　対数ポテンシャルの加速度

```
def acc_log_axisym(x, v0=1.0, q=0.8):
    # ポテンシャルの勾配を Cartesian 座標で評価する。
    x = np.asarray(x)
    den = x[0]**2 + x[1]**2 + x[2]**2 / q**2
    return -v0**2 * np.array([x[0], x[1], x[2] / q**2]) / den
```

半径方向のエピサイクル振動数

円軌道のまわりの小さな振動を、エピサイクル運動（epicyclic motion）として近似する。Ryden は z 方向の比角運動量を j_z と書くが、本書では L_z と表す。軸対称性によって

$$L_z = R^2\dot{\phi} = \text{const.} \tag{C.5}$$

なので、半径・鉛直方向の運動は有効ポテンシャル（effective potential）

$$\Phi_{\text{eff}}(R,z) = \Phi(R,z) + \frac{L_z^2}{2R^2} \tag{C.6}$$

で記述できる。これは Ryden 式 (7.35) に対応する [1]。半径方向の運動方程式は $\ddot{R} = -\partial\Phi_{\text{eff}}/\partial R$ である。

円軌道の案内中心半径（guiding radius）を $R_g > 0$ とする。$z = 0$ では $\Phi(R,0) = v_0^2 \ln(R/\ell)$ であるから、円軌道の条件は

$$\left.\frac{\partial \Phi_{\text{eff}}}{\partial R}\right|_{(R_g,0)} = \frac{v_0^2}{R_g} - \frac{L_z^2}{R_g^3} = 0 \tag{C.7}$$

となる。以下では正の向きに回る円軌道を選ぶ。比角運動量と角速度は次式で与えられる。

$$L_z = R_g v_0, \qquad \Omega = \frac{v_0}{R_g} \tag{C.8}$$

半径方向の変位を $\xi = R - R_g$ と書く。有効ポテンシャルを ξ の 2 次まで Taylor 展開すると、円軌道の条件によって 1 次の項が消え、

$$\Phi_{\text{eff}}(R,0) \simeq \Phi_{\text{eff}}(R_g,0) + \frac{1}{2}\kappa^2\xi^2 \tag{C.9}$$

となる。係数は

$$\kappa^2 = \left.\frac{\partial^2 \Phi_{\rm eff}}{\partial R^2}\right|_{(R_g,0)} = -\frac{v_0^2}{R_g^2} + \frac{3L_z^2}{R_g^4} = \frac{2v_0^2}{R_g^2} \tag{C.10}$$

である。したがって、線形化した運動方程式は $\ddot{\xi} = -\kappa^2 \xi$ となり、

$$\boxed{\kappa = \sqrt{2}\,\Omega} \tag{C.11}$$

を得る。初期変位を ξ_0、初期半径速度を 0 とすると、

$$R(t) - R_g \simeq \xi_0 \cos(\kappa t) \tag{C.12}$$

が比較に用いる近似解である。

鉛直振動数

$z = 0$ のまわりでも Taylor 展開を行う。ポテンシャルは z の偶関数なので 1 次の項はなく、$R = R_g$ で

$$\Phi(R_g, z) \simeq \Phi(R_g, 0) + \frac{1}{2}\nu_z^2 z^2 \tag{C.13}$$

となる。$\partial\Phi/\partial z = v_0^2 z/[q^2(R^2 + z^2/q^2)]$ をさらに微分して $z = 0$ と置くと、

$$\nu_z^2 = \left.\frac{\partial^2 \Phi}{\partial z^2}\right|_{(R_g,0)} = \frac{v_0^2}{q^2 R_g^2}, \qquad \boxed{\nu_z = \frac{v_0}{q R_g}} \tag{C.14}$$

を得る。これは Ryden 式 (7.47) を対数ポテンシャルに適用したものである [1]。初期鉛直速度も 0 とすると、

$$z(t) \simeq z_0 \cos(\nu_z t) \tag{C.15}$$

となる。この近似では $|\xi|/R_g \ll 1$、$|z|/(qR_g) \ll 1$ とし、半径方向と鉛直方向の結合を高次の項として省いている。式(C.12)と式(C.15)は、元の運動方程式の厳密解ではない。

初期条件と数値軌道の比較

Notebook C では $v_0 = 1$、$q = 0.8$、$R_g = 1$ とし、

$$\Omega = 1.000000, \qquad \kappa = 1.414214, \qquad \nu_z = 1.250000 \tag{C.16}$$

を用いる。初期変位を $\xi_0 = z_0 = 0.05 R_g$ とし、

$$\boldsymbol{x}_{\rm init} = (R_g + \xi_0, 0, z_0), \qquad \boldsymbol{v}_{\rm init} = \left(0, \frac{R_g v_0}{R_g + \xi_0}, 0\right) \tag{C.17}$$

とする。初期半径をずらしても、方位角方向の速度をこのように選べば $L_z = R_g v_0$ を保てる。基準周期 $T_\phi = 2\pi/\Omega$ に対して、$\Delta t = T_\phi/300$ で $12T_\phi$ まで積分する。T_ϕ は案内中心の円軌道の周期であり、数値軌道の実測周期ではない。

Notebook C ｜ほぼ円軌道とエピサイクル運動

> `x_amp` が ξ_0、`z_amp` が z_0 に対応する。近似解と数値軌道に同じ初期変位を用い、比較する時刻も揃える。加速度関数と初期条件を積分器へ渡す部分は、モデルを変えても共通である。

対応セル（初期条件と基準周期）

```
# 案内中心からの初期変位と、保つべき比角運動量を設定する。
Rg = 1.0
x_amp = 0.05 * Rg
z_amp = 0.05 * Rg
Lz_circ = Rg * vlog
R_init = Rg + x_amp
vphi_init = Lz_circ / R_init
x0_ax = np.array([R_init, 0.0, z_amp])
v0_ax = np.array([0.0, vphi_init, 0.0])

# 近似解の振動数と、案内中心の円軌道の周期を求める。
Omega = vlog / Rg
kappa = np.sqrt(2.0) * Omega
nu_z = vlog / (qlog * Rg)
Tphi = 2.0 * np.pi / Omega
```

対応セル（時間積分と診断量）

```
# 同じ加速度関数を各時間ステップで使い、12Tphi まで積分する。
dt_ax = Tphi / 300.0
nstep_ax = int(round(12.0 * Tphi / dt_ax))
t_ax = np.arange(nstep_ax + 1) * dt_ax
acc_ax = lambda x: acc_log_axisym(x, vlog, qlog)
x_ax, v_ax = integrate_leapfrog(acc_ax, x0_ax, v0_ax, dt_ax, nstep_ax)

# 各時刻の半径と、保存量の診断に用いる配列を求める。
R_ax = np.sqrt(x_ax[:, 0]**2 + x_ax[:, 1]**2)
E_ax = energy_log_axisym(x_ax, v_ax, vlog, qlog)
Lz_ax = Lz_3d(x_ax, v_ax)
```

図C.1と図C.2では、最初の $3T_\phi$ の数値軌道と近似解を比較している。両者はほぼ同じ振動を示すが、有限の初期振幅では完全には一致しない。この差の解釈には、後述の時間刻みと初期振幅を変える検証が必要になる。

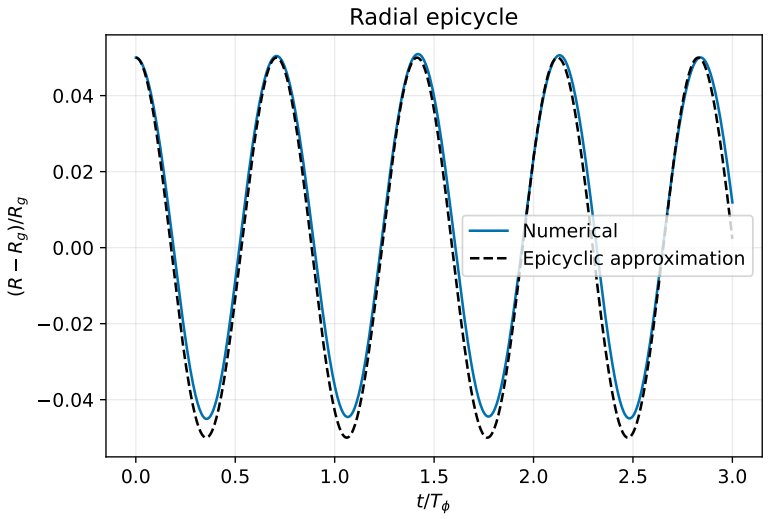


図 C.1　数値軌道の半径方向変位と、小振幅近似 $\xi_0 \cos(\kappa t)$ の比較。初期変位と初期半径速度を揃え、同じ時刻で評価している。

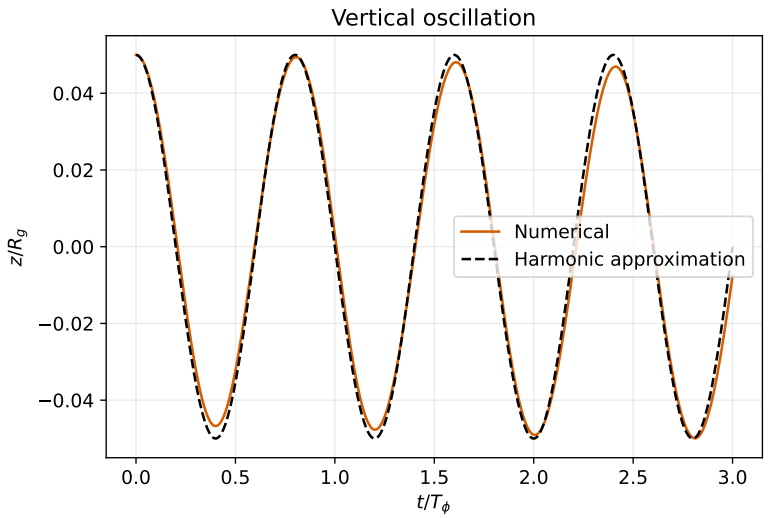


図 C.2　数値軌道の鉛直変位と、小振幅近似 $z_0\cos(\nu_z t)$ の比較。半径・鉛直方向の結合は近似解に含まれていない。

保存量の診断には、初期値を $E_0 = E(0)$、$L_{z,0} = L_z(0)$ として、

$$\epsilon_E = \frac{\max_n |E(t_n) - E_0|}{|E_0|}, \qquad \epsilon_{L_z} = \frac{\max_n |L_z(t_n) - L_{z,0}|}{|L_{z,0}|} \tag{C.18}$$

を用いる。この例では両分母は 0 ではなく、$0 \le t \le 12T_\phi$ の全出力時刻で測ると、

$$\epsilon_E = 2.113 \times 10^{-5}, \qquad \epsilon_{L_z} = 7.772 \times 10^{-15} \tag{C.19}$$

となる。軸対称な力ではキックによる L_z の変化が 0 であり、ドリフトも角運動量を変えない。このため、リープフロッグ法は L_z を丸め誤差の範囲で保つ。第 3 章 §3.8の球対称系では角運動量ベクトル全体について成り立った性質が、ここでは z 成分について成り立つ。ただし、L_z がよく保たれていても、各時刻の位置の精度は別に確認する必要がある。

数値計算上の注意｜エネルギーの基準と相対誤差

ポテンシャルの加法定数を変えると、軌道は同じでも E_0 が変わる。エネルギーの相対誤差を比較するときは、その基準を固定する。$E_0 = 0$ または $L_{z,0} = 0$ の場合には式(C.18)は使わず、絶対的な変化や、別に定めた非零のスケールで規格化した変化を用いる。

時間刻みと初期振幅を変えた検証

第 3 章の円軌道では厳密解との位置の差を時間積分誤差として測った。一方、ここで用いた小振幅近似との差には、時間積分の誤差と、線形化で省いた項の影響が含まれる。そこで、まず初期振幅と終端時刻を固定し、時間刻み Δt と $\Delta t/2$ の数値軌道どうしを比較する。

共通する出力時刻で位置の最大差を

$$D_{\Delta t} = \max_{t_n} \frac{|\boldsymbol{x}_{\Delta t}(t_n) - \boldsymbol{x}_{\Delta t/2}(t_n)|}{R_g} \tag{C.20}$$

と定める。離散化誤差が支配する範囲で、同じ時刻の数値解が $\boldsymbol{x}_{\Delta t} = \boldsymbol{x}_{\rm exact} + \boldsymbol{C}(\Delta t)^p + \cdots$ と書ければ、差の主要項は $\boldsymbol{C}(1 - 2^{-p})(\Delta t)^p$ となる。このため、

$$p_{\rm self} = \log_2 \frac{D_{\Delta t}}{D_{\Delta t/2}} \tag{C.21}$$

から収束次数を推定できる。$D_{\Delta t}$ は 2 つの数値解の差であり、厳密解に対する誤差そのものではない。また、共通の実装上の誤りがあっても数値解どうしは収束し得るため、加速度の式や保存量の確認も併用する。

Listing C.2 は式(C.20)の計算部分である。細かい時間刻みの配列から `[::2]` で 1 点おきに取り出すと、粗い配列の時刻と一致する。`axis=1` は各時刻の位置ベクトルの大きさを求める指定である。

Listing C.2　共通する時刻での位置の差と収束次数

```
self_diffs = []
for x_coarse, x_fine in zip(trial_orbits[:-1], trial_orbits[1:]):
    dx = x_coarse - x_fine[::2]
    self_diffs.append(np.max(np.linalg.norm(dx, axis=1)) / Rg)
self_diffs = np.asarray(self_diffs)
p_self = np.log2(self_diffs[:-1] / self_diffs[1:])
```

Notebook C では $\xi_0 = z_0 = 0.05R_g$、終端時刻 $3T_\phi$ を固定し、1 基準周期あたり 300、600、1200、2400 ステップで積分する。結果を表C.1に示す。刻みを半分にすると差は約 4 分の 1 になり、リープフロッグ法の 2 次収束と整合する。

表 C.1　同じ初期条件・終端時刻で得た数値軌道どうしの比較。最後の行は $D_{\Delta t/2}$ に必要な、さらに細かい計算を行っていないため、次数を表示しない。

$\Delta t/T_\phi$	$D_{\Delta t}$	p_{self}
$1/300$	4.528×10^{-4}	2.000
$1/600$	1.132×10^{-4}	2.000
$1/1200$	2.830×10^{-5}	—

次に $\Delta t = T_\phi/2400$ と終端時刻 $3T_\phi$ を固定し、初期振幅だけを小さくする。$\alpha = \xi_0/R_g = z_0/R_g$ とし、近似解との差を

$$A_R = \frac{\max_n |R(t_n) - R_g - \xi_0\cos(\kappa t_n)|}{|\xi_0|}, \qquad A_z = \frac{\max_n |z(t_n) - z_0\cos(\nu_z t_n)|}{|z_0|} \tag{C.22}$$

で測る。初期振幅で規格化するのは、振幅を小さくしたため絶対差が小さくなっただけなのか、振幅に比べても一致が改善したのかを区別するためである。この比較では $\alpha > 0$ とする。

表 C.2　初期振幅を変えた小振幅近似との比較。括弧内は 1 基準周期あたりのステップ数である。時間刻みを半分にしたときの変化は、初期振幅を半分にしたときの変化に比べて小さい。

α	A_R (1200)	A_R (2400)	A_z (1200)	A_z (2400)
0.0500	0.19061	0.19056	0.15024	0.15022
0.0250	0.07700	0.07690	0.05532	0.05530
0.0125	0.03341	0.03321	0.02216	0.02214

表C.2には $\Delta t = T_\phi/1200$ の結果も並べた。この範囲では、時間刻みをさらに小さくしても近似解との差はほとんど変わらない一方、初期振幅を小さくすると規格化した差も減る。したがって、近似解との不一致の主な原因を、時間積分ではなく小振幅近似に求められる。振幅をさらに小さくする場合には、固定した時間刻みの誤差が再び効く可能性があるため、刻みの確認も繰り返す。

Notebook C ｜時間刻みと初期振幅の比較

`epicycle_trial` は、振幅比と 1 基準周期あたりのステップ数を受け取り、$0 \le t \le 3T_\phi$ の時刻と位置を返す関数である。関数内では式(C.17)の初期条件を設定し、同じ積分器を呼ぶ。関数の完全な定義と出力整形は Notebook に置く。

対応セル（時間刻みを変えた軌道計算）

```
# 初期振幅を固定し、各時間刻みで独立に積分する。
steps_per_period = np.array([300, 600, 1200, 2400])
trial_orbits = []
for npp in steps_per_period:
    _, x_trial = epicycle_trial(0.05, npp, Rg, vlog, qlog)
    trial_orbits.append(x_trial)
```

得られた `trial_orbits` に Listing C.2 を適用する。初期振幅を変える比較では、時間刻みを 2 通りにして、式(C.22)を次のように計算する。

対応セル（初期振幅を変えた近似解との比較）

```
# 振幅を変えた比較を、2 つの十分小さい時間刻みで繰り返す。
amp_fractions = np.array([0.05, 0.025, 0.0125])
amp_error_coarse = []
amp_error_fine = []
for amp_fraction in amp_fractions:
    amp = amp_fraction * Rg
    for npp, errors in [(1200, amp_error_coarse), (2400, amp_error_fine)]:
        t_trial, x_trial = epicycle_trial(amp_fraction, npp, Rg, vlog, qlog)
        R_trial = np.sqrt(x_trial[:, 0]**2 + x_trial[:, 1]**2)
        radial_approx = amp * np.cos(kappa * t_trial)
        vertical_approx = amp * np.cos(nu_z * t_trial)
        A_R = np.max(np.abs(R_trial - Rg - radial_approx)) / amp
        A_z = np.max(np.abs(x_trial[:, 2] - vertical_approx)) / amp
        errors.append([A_R, A_z])
amp_error_coarse = np.asarray(amp_error_coarse)
amp_error_fine = np.asarray(amp_error_fine)
```

C.3 Miyamoto–Nagai ポテンシャル中の軌道

加速度関数の差し替え

別の軸対称モデルとして、Miyamoto–Nagai ポテンシャル

$$\Phi(R,z) = -\frac{GM}{\sqrt{R^2 + (a + \sqrt{z^2 + b^2})^2}} \tag{C.23}$$

を用いる。a と b は長さスケールであり、ここでは $G, M, a, b > 0$ とする。$B = \sqrt{z^2 + b^2}$、$D = \sqrt{x^2 + y^2 + (a + B)^2}$ と置くと、

$$\frac{\partial D}{\partial x} = \frac{x}{D}, \qquad \frac{\partial B}{\partial z} = \frac{z}{B}, \qquad \frac{\partial D}{\partial z} = \frac{(a + B)z}{BD} \tag{C.24}$$

である。$\Phi = -GM/D$ を微分して、

$$g_x = -\frac{GMx}{D^3}, \qquad g_y = -\frac{GMy}{D^3}, \qquad g_z = -\frac{GM(a + B)z}{BD^3} \tag{C.25}$$

を得る。鉛直方向では B も z に依存することに注意する。

Listing C.3 Miyamoto–Nagai ポテンシャルの加速度

```
def acc_mn_cart(x, G=1.0, M=1.0, a=1.0, b=0.25):
    # B と D の両方の z 依存性を含む解析的な加速度。
    X, Y, Z = x
    R2 = X**2 + Y**2
    B = np.sqrt(Z**2 + b**2)
    D = np.sqrt(R2 + (a + B)**2)

    ax = -G * M * X / D**3
    ay = -G * M * Y / D**3
    az = -G * M * (a + B) * Z / (B * D**3)
    return np.array([ax, ay, az])
```

Listing C.3 の関数を積分器へ渡せば、前節と同じ更新式で軌道を求められる。時間に依存しない軸対称系なので、ここでも

$$E = \text{const.}, \qquad L_z = \text{const.} \tag{C.26}$$

が検証に使う性質である。

初期条件と保存量による診断

長さ a、速度 $v_* = \sqrt{GM/a}$、時間 $t_* = a/v_*$ を用い、

$$\frac{\boldsymbol{x}_0}{a} = (3, 0, 0.5), \qquad \frac{\boldsymbol{v}_0}{v_*} = (0, 0.5, 0), \qquad \Delta t = 0.02 t_*, \qquad t_{\mathrm{end}} = 100 t_* \tag{C.27}$$

とする。Notebook C の既定値は $G = M = a = 1$、$b = 0.25$ であり、このとき $v_* = t_* = 1$ である。初期エネルギーは

$$E_0 = -0.17077859 \tag{C.28}$$

となる。式(C.18)で全出力時刻の最大絶対相対変化を測ると、

$$\epsilon_E = 7.330 \times 10^{-7}, \qquad \epsilon_{L_z} = 3.701 \times 10^{-15} \tag{C.29}$$

である。図C.3の軌道の形と合わせてこれらを確認する。L_z が丸め誤差の範囲で保たれることは、前節と同じキックとドリフトの性質による。位置の精度は、初期条件と終端時刻を固定した時間刻みの収束試験で別に調べる。

Notebook C ｜ Miyamoto–Nagai 軌道

> モデルのパラメータを変えても、初期条件と時間の単位が対応するように設定する。数値 Poisson 解は入力せず、解析的な加速度を使う。

対応セル（単位・初期条件の設定、積分、診断量）

```
# モデルを設定し、長さ・速度・時間の単位を揃える。
M_mn, a_mn, b_mn = 1.0, 1.0, 0.25
vscale_mn = np.sqrt(G * M_mn / a_mn)
tscale_mn = a_mn / vscale_mn
x0_mn = a_mn * np.array([3.0, 0.0, 0.5])
v0_mn = vscale_mn * np.array([0.0, 0.5, 0.0])

# 解析的な加速度で 100 時間単位まで積分する。
dt_mn = 0.02 * tscale_mn
nstep_mn = int(round(100.0 * tscale_mn / dt_mn))
t_mn = np.arange(nstep_mn + 1) * dt_mn
acc_mn = lambda x: acc_mn_cart(x, G, M_mn, a_mn, b_mn)
x_mn, v_mn = integrate_leapfrog(acc_mn, x0_mn, v0_mn, dt_mn, nstep_mn)

# 位置・速度から半径と保存量を計算する。
E_mn = energy_mn(x_mn, v_mn, G, M_mn, a_mn, b_mn)
Lz_mn = Lz_3d(x_mn, v_mn)
R_mn = np.sqrt(x_mn[:, 0]**2 + x_mn[:, 1]**2)
```

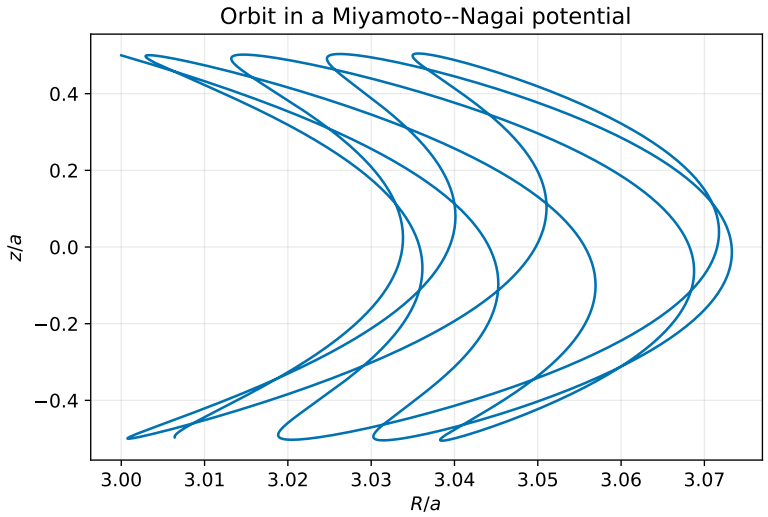


図 C.3　Miyamoto–Nagai ポテンシャル中の軌道の R–z 平面への投影。積分時間は $100t_*$、時間刻みは $0.02t_*$ である。線の形だけで数値精度を判断せず、保存量と時間刻み依存性を合わせて確認する。

数値計算上の注意｜同じモデルでも数値重力場の誤差は別に検証する

付録Bでは同じモデルの密度から Poisson 方程式を解くが、その数値解は本節では使わない。格子上のポテンシャルを軌道計算に使う場合には、空間離散化に加えて、補間や数値微分の誤差も含まれる。また、モデルの厚さ b/a を変える場合には、以前と同じ時間刻みで十分かを確かめ直す。

C.4 非軸対称ポテンシャル

加速度と保存量の選択

Ryden §7.3 の 2 次元の非軸対称対数ポテンシャル

$$\Phi(x,y) = \frac{1}{2}v_0^2 \ln\left[1 + \frac{x^2 + y^2/q^2}{r_c^2}\right] \tag{C.30}$$

を使う。これは Ryden 式 (7.68) に対応する [1]。$v_0, r_c > 0$、$0 < q \leq 1$ とし、$q \neq 1$ で非軸対称になる。$D = r_c^2 + x^2 + y^2/q^2$ と置くと、加速度は

$$g_x = -\frac{v_0^2 x}{D}, \qquad g_y = -\frac{v_0^2 y}{q^2 D} \tag{C.31}$$

となる。この場合は位置・速度・加速度を 2 成分の配列にするが、積分関数は同じでよい。

Listing C.4　非軸対称対数ポテンシャルの加速度

```
def acc_nonaxisym(x, v0=1.0, rc=1.0, q=np.sqrt(3.0)/2.0):
    # 2 次元の位置配列から、2 成分の加速度を返す。
    X, Y = x
    den = rc**2 + X**2 + Y**2/q**2
    return -v0**2 * np.array([X, Y/q**2]) / den
```

ポテンシャルは時間に依存しないため、

$$E = \text{const.} \tag{C.32}$$

である。一方、式(C.31)から

$$\frac{dL_z}{dt} = x g_y - y g_x = \frac{v_0^2 xy}{D}\left(1 - \frac{1}{q^2}\right) \tag{C.33}$$

なので、$q \neq 1$ では一般に

$$L_z \neq \text{const.} \tag{C.34}$$

となる。L_z の変化をすべて数値誤差と解釈してはいけない。$q = 1$ とすればトルクが 0 になることは、加速度の実装を検証する一つの方法である。

中心近傍の近似とその適用範囲

中心近くで $\eta = (x^2 + y^2/q^2)/r_c^2 \ll 1$ なら、$\ln(1+\eta) = \eta + O(\eta^2)$ より

$$\Phi(x,y) \simeq \frac{1}{2}\frac{v_0^2}{r_c^2}x^2 + \frac{1}{2}\frac{v_0^2}{q^2 r_c^2}y^2 \tag{C.35}$$

となる。この近似では $\ddot{x} = -\omega_x^2 x$、$\ddot{y} = -\omega_y^2 y$ であり、

$$\omega_x = \frac{v_0}{r_c}, \qquad \omega_y = \frac{v_0}{q r_c} \tag{C.36}$$

という異方的調和振動子（anisotropic harmonic oscillator）になる。$q < 1$ では 2 方向の振動数が異なり、中心付近の運動を理解する手掛かりとなる。ただし、以下の初期位置 $(2r_c, 0)$ では $\eta = 4$ であり、中心近傍の近似は成り立たない。この近似を、有限振幅の数値軌道全体の基準解には使わない。

同じエネルギーを持つ軌道の比較

Ryden 図 7.6 にならい、

$$q = \frac{\sqrt{3}}{2}, \qquad E = 1.5 v_0^2, \qquad \boldsymbol{x}_0 = (2r_c, 0) \tag{C.37}$$

を共通にする。速さと速度方向を別々に設定し、

$$v_{\rm init} = \sqrt{2[E - \Phi(2r_c, 0)]}, \qquad v_y = f v_{\rm init}, \qquad v_x = +\sqrt{v_{\rm init}^2 - v_y^2} \tag{C.38}$$

とする。$E > \Phi(2r_c, 0)$、$0 \leq f \leq 1$ とすれば、平方根の中は非負である。ここでは $f = 0.5$ と 0.9 を選び、初期位置とエネルギーを同じにしたまま、速度方向だけを変える。$\Delta t = 0.02 r_c/v_0$、積分時間 $300 r_c/v_0$ とし、Notebook C の既定値は $r_c = v_0 = 1$ である。

Notebook C ｜同じエネルギーを持つ 2 本の軌道

`v0_na`、`rc_na`、`q_na` をモデルのパラメータとし、`E_target=1.5*v0_na**2` とする。まずエネルギーから速さを決め、速度方向を変えた 2 組の初期条件を作る。

対応セル（同じ比エネルギーからの初速度の設定）

```
# 初期位置と比エネルギーを共通にして速さを決める。
x_start = np.array([2.0 * rc_na, 0.0])
phi_start = phi_nonaxisym(x_start, v0_na, rc_na, q_na)
if E_target <= phi_start:
    raise ValueError(" 速度方向を比較するには E_target > phi_start が必要です。")
speed = np.sqrt(2.0 * (E_target - phi_start))

# このモデルと初期位置で box・loop 軌道になる速度方向を選ぶ。
f_box = 0.50
vy_box = f_box * speed
vx_box = np.sqrt(speed**2 - vy_box**2)
v_box0 = np.array([vx_box, vy_box])
f_loop = 0.90
vy_loop = f_loop * speed
vx_loop = np.sqrt(speed**2 - vy_loop**2)
v_loop0 = np.array([vx_loop, vy_loop])
```

2 本の軌道には、同じ加速度・時間刻み・終端時刻を使う。

対応セル（時間積分と診断量）

```
# 時間の単位を rc/v0 に揃え、同じ刻みと終端時刻を使う。
dt_na = 0.02 * rc_na / v0_na
t_end_na = 300.0 * rc_na / v0_na
nstep_na = int(round(t_end_na / dt_na))
t_na = np.arange(nstep_na + 1) * dt_na
acc_na = lambda x: acc_nonaxisym(x, v0_na, rc_na, q_na)
x_box, v_box = integrate_leapfrog(acc_na, x_start, v_box0, dt_na, nstep_na)
x_loop, v_loop = integrate_leapfrog(acc_na, x_start, v_loop0, dt_na, nstep_na)

# E の変化は保存誤差として、Lz の変化は物理的な変化として調べる。
E_box = energy_nonaxisym(x_box, v_box, v0_na, rc_na, q_na)
E_loop = energy_nonaxisym(x_loop, v_loop, v0_na, rc_na, q_na)
L_box = angular_momentum_z(x_box, v_box)
L_loop = angular_momentum_z(x_loop, v_loop)
```

図C.4と図C.5に軌道を示す。今回の計算時間では、$f = 0.5$ の軌道は中心近傍まで達し、回り込む向きが変わる box 軌道である。$f = 0.9$ の軌道は、中心の周囲を一定の向きに回る loop

軌道である。この分類は今回のモデルと初期条件についての結果であり、f の値だけから任意の軌道を分類できるわけではない。

運動エネルギーが非負であるため、厳密な軌道が存在できる領域は $\Phi(x,y) \le E$ である。その境界 $\Phi = E$ は

$$x^2 + \frac{y^2}{q^2} = r_c^2 \left[\exp\left(\frac{2E}{v_0^2} \right) - 1 \right] \tag{C.39}$$

という楕円になり、図の破線に対応する。これは許される領域を示すもので、軌道が内部のすべての点を通るという意味ではない。

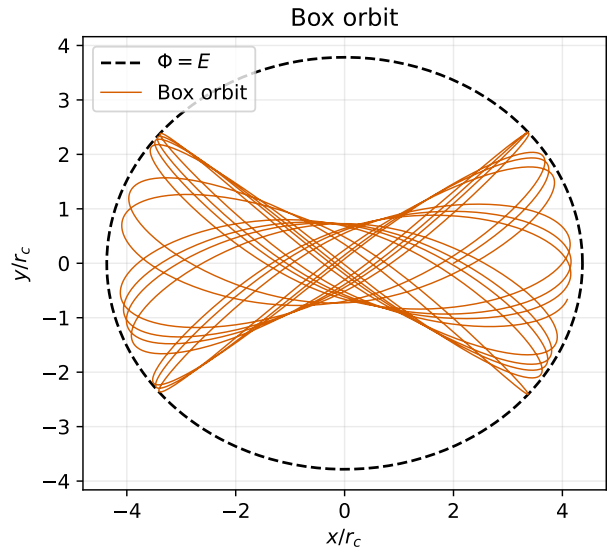


図 C.4　$f = 0.5$ の初速度で得た box 軌道。破線は式(C.39)のゼロ速度境界 $\Phi = E$ である。積分時間は $300 r_c / v_0$。

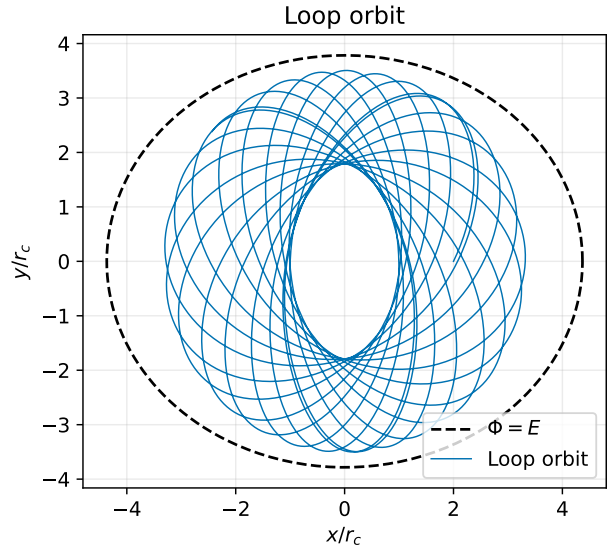


図 C.5　$f = 0.9$ の初速度で得た loop 軌道。図C.4と初期位置・比エネルギー・積分時間は同じであり、速度方向だけが異なる。

保存誤差と物理的な変化の区別

両軌道の初期エネルギーは

$$E_{\mathrm{box}}(0) = 1.50000000, \qquad E_{\mathrm{loop}}(0) = 1.50000000 \tag{C.40}$$

となり、設定どおり一致している。$0 \le t \le 300 r_c/v_0$ の全出力時刻で測ったエネルギーの最大絶対相対変化と角運動量の範囲を、表C.3に示す。この場合、E の変化は保存誤差として調べるが、L_z の変化には式(C.33)のトルクによる物理的な変化が含まれる。

表 C.3　非軸対称系の軌道の診断。ϵ_E は式(C.18)で定義した保存誤差、L_z の範囲は数値軌道に沿った変化であり、誤差の範囲ではない。

軌道	ϵ_E	$L_z/(r_c v_0)$ の範囲
box	7.769×10^{-5}	$[-1.2706, 1.2706]$
loop	2.739×10^{-5}	$[1.3312, 2.3772]$

図C.6では、box 軌道の L_z は正負を横切り、loop 軌道では符号が保たれる。ただし、loop 軌道でも L_z の大きさは一定ではない。回り込む向きが変わらないことと、角運動量が保存することは別である。数値的な変化と物理的な変化を区別するには、同じ初期条件・終端時刻で時間刻みを小さくし、エネルギー誤差が減っても $L_z(t)$ の変化が残るかを確かめる。対応する検証は Notebook C の演習で行う。

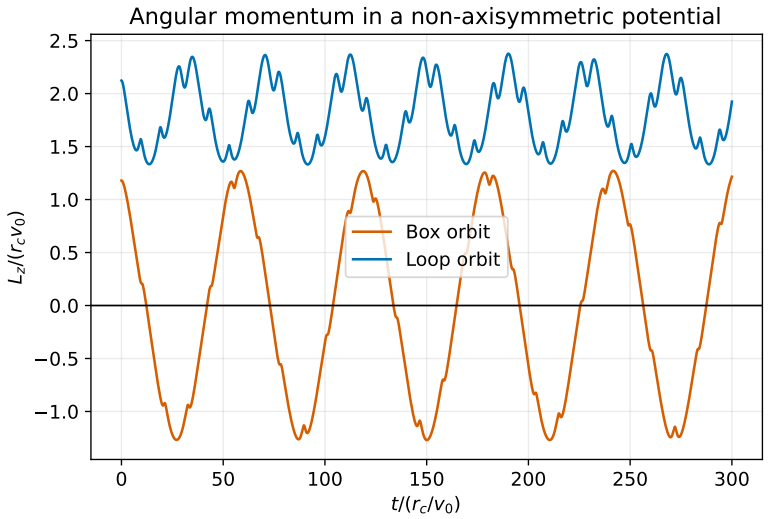


図 C.6　非軸対称対数ポテンシャル中の角運動量の時間変化。box 軌道では符号が変わり、loop 軌道では符号が保たれるが、どちらも一定ではない。

数値計算上の注意｜有限時間の出力から判断できる範囲

計算から直接得られる最小半径は、有限時間内の離散的な出力点で測った値である。出力点の間でさらに小さい半径に達する可能性があり、積分時間を延ばせば新しい最小値が見つかる可能性もある。積分時間と時間刻みを別々に変えて結果を比較する必要があるが、それだけで無限時間に中心へ任意に近づくことを証明したわけではない。

付録 D　連立一次方程式の解法

第 2 章では `numpy.linalg.solve`、付録Bでは `scipy.sparse.linalg.spsolve` を使い、Poisson 方程式を離散化して得た連立一次方程式を解いた。本付録は、その求解の仕組みを確認するための背景資料である。同じ小さな連立一次方程式を使い、Gauss の消去法、LU 分解、疎行列の消去、Jacobi 法を順に説明する。行列の構造と解法の種類を区別し、計算結果を残差でどう確認するかを学ぶ。数値計算法は幸谷第 7・8 章 [5] を参考にする。

本編と同様に、一般的なソルバーの自作は求めない。以下では m 個の未知数に対する正方行列の問題 $A\boldsymbol{u} = \boldsymbol{b}$ を扱い、解は一意に定まるとする。式と実装の対応は短い Listing で示す。付録 D 専用の Notebook は設けず、Notebook 2 末尾の補足「付録 D：連立一次方程式の解法」も参照できる。

D.1 Gauss の消去法：小さな例

第 2 章の三重対角行列と同じ構造を持つ、説明用の例を考える。ここでは計算手順を追うため、右辺を $\boldsymbol{b} = (1, 0, 1)^{\mathsf{T}}$ と選ぶ。この右辺は Plummer 球の密度や境界値から求めたものではない。連立一次方程式は

$$\underbrace{\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}}_{A} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \tag{D.1}$$

である。これは、$2u_1 - u_2 = 1$、$-u_1 + 2u_2 - u_3 = 0$、$-u_2 + 2u_3 = 1$ の 3 式をまとめたものである。

一つの式の定数倍を別の式に加える操作は、同じ定数倍を引けば元に戻せるため、解を変えない。この操作で未知数を順番に消す方法を **Gauss の消去法（Gaussian elimination）**と呼ぶ。係数と右辺を縦線で区切って並べ、R_i を第 i 行と書くと、まず

$$\left[\begin{array}{ccc|c} 2 & -1 & 0 & 1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 + \frac{1}{2}R_1} \left[\begin{array}{ccc|c} 2 & -1 & 0 & 1 \\ 0 & \frac{3}{2} & -1 & \frac{1}{2} \\ 0 & -1 & 2 & 1 \end{array}\right] \tag{D.2}$$

として、第 2 行の u_1 を消す。次に、**更新後の第 2 行**を使って

$$R_3 \leftarrow R_3 + \frac{2}{3}R_2 \quad \Longrightarrow \quad \left[\begin{array}{ccc|c} 2 & -1 & 0 & 1 \\ 0 & \frac{3}{2} & -1 & \frac{1}{2} \\ 0 & 0 & \frac{4}{3} & \frac{4}{3} \end{array}\right] \tag{D.3}$$

とすると、第 3 行の u_2 も消える。このように対角成分より下を 0 にする操作が**前進消去（forward elimination）**である。係数だけを変えると別の連立方程式になるため、右辺にも同じ行操作を行う。

得られた上三角の連立方程式は、下の行から順に解ける。

$$u_3 = \frac{3}{4} \cdot \frac{4}{3} = 1, \qquad u_2 = \frac{2}{3}\left(\frac{1}{2} + u_3\right) = 1, \qquad u_1 = \frac{1 + u_2}{2} = 1. \tag{D.4}$$

この操作を**後退代入（back substitution）**と呼ぶ。求めた解 $\boldsymbol{u} = (1, 1, 1)^{\mathsf{T}}$ を式(D.1)へ代入すると、元の 3 式を満たす。**前進消去で解きやすい形に変え、後退代入で解を求める**のが基本である。

第 2 章と同じライブラリでこの例を解き、元の方程式に代入して確認するコードを Listing D.1 に示す。この例では厳密解が分かるので、残差だけでなく解そのものの誤差も計算できる。

Listing D.1　小さな連立一次方程式の求解と検証

```
import numpy as np

A = np.array([[2.0, -1.0, 0.0], [-1.0, 2.0, -1.0], [0.0, -1.0, 2.0]])
b = np.array([1.0, 0.0, 1.0])
u_num = np.linalg.solve(A, b)

# 元の方程式の残差と、既知の厳密解からの誤差を別々に調べる
residual_max = np.max(np.abs(A @ u_num - b))
solution_error = np.max(np.abs(u_num - np.ones(3)))
```

有限精度の計算では、解が見かけ上 $(1, 1, 1)$ と表示されても、残差が厳密に 0 になるとは限らない。残差と解の誤差の違いは、D.4節でも確認する。

D.2 LU 分解：消去手順の行列表現

前節の行列 A は、次の二つの三角行列の積に分解できる。

$$\boxed{A = LU}, \qquad L = \begin{pmatrix} 1 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 \\ 0 & -\frac{2}{3} & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & -1 & 0 \\ 0 & \frac{3}{2} & -1 \\ 0 & 0 & \frac{4}{3} \end{pmatrix}. \tag{D.5}$$

L は対角成分より上が 0 の下三角行列、U は対角成分より下が 0 の上三角行列である。ここでは L の対角成分をすべて 1 に取っている。この分解を **LU 分解（LU decomposition）**と呼ぶ。U は前進消去の結果で、L には消去に使った倍率が記録されている。例えば第 2 行の消去は $R_2 \leftarrow R_2 - \ell_{21} R_1$ と書け、$\ell_{21} = -1/2$ である。これが L の第 2 行・第 1 列の値になる。同様に、更新後の第 2 行を使う倍率 $\ell_{32} = -2/3$ が L に入る。

$A\boldsymbol{u} = \boldsymbol{b}$ に $A = LU$ を代入し、$\boldsymbol{y} = U\boldsymbol{u}$ とおけば、

$$\underbrace{L\boldsymbol{y} = \boldsymbol{b}}_{\text{上の行から解く}} \qquad \longrightarrow \qquad \underbrace{U\boldsymbol{u} = \boldsymbol{y}}_{\text{下の行から解く}} \tag{D.6}$$

の 2 段階になる。下三角系を上から解く操作が**前進代入（forward substitution）**である。本例では

$$y_1 = 1, \qquad y_2 = 0 + \frac{1}{2} y_1 = \frac{1}{2}, \qquad y_3 = 1 + \frac{2}{3} y_2 = \frac{4}{3} \tag{D.7}$$

となり、$\boldsymbol{y}$ は式(D.3)の変形後の右辺に一致する。あとは前節と同じ後退代入で $\boldsymbol{u}$ を求める。**前進消去は行列を変形する操作、前進代入は下三角系を解く操作**である。

同じ A に対して右辺 $\boldsymbol{b}$ だけが変わる場合、一度求めた L, U を再利用できる。各右辺について前進代入と後退代入だけを行えばよいことが、LU 分解の利点である [5]。第 2 章の Poisson 方程式でも、格子と差分式を固定して密度や境界値だけを変えるなら、係数行列は変わらず右辺だけが変わる。ただし、格子を変える収束試験では行列も変わるので、同じ分解をそのまま使うことはできない。

行を入れ替える場合

一般の行列では、解が一意に定まっていても、消去の途中で割る成分が 0 になることがある。0 でなくても、他の成分に比べて小さい値で割ると、丸め誤差が増幅される場合がある。この割る成分を**枢軸（pivot、ピボット）**という。**部分枢軸選択（partial pivoting）**では、処理する

列の未処理部分から絶対値が最大の成分を選び、その行を現在の行と入れ替える。右辺も同じ順に入れ替える。行の入れ替えを表す行列を P と書けば、本書の記法では

$$PA = LU, \qquad L\boldsymbol{y} = P\boldsymbol{b}, \qquad U\boldsymbol{u} = \boldsymbol{y} \tag{D.8}$$

となる。行を入れ替えても未知数の順番は変わらない。本例では行交換は不要である [5]。

なぜ逆行列を作らずに `solve` を使うのか

数学的には $\boldsymbol{u} = A^{-1}\boldsymbol{b}$ と書けるが、解だけが必要なら逆行列全体を作る必要はない。逆行列を求めてから右辺を掛ける方法は、LU 分解と代入で解く方法より計算量が増える [5]。第 2 章と Listing D.1 の `np.linalg.solve(A, b)` も、逆行列を明示的に作るのではなく、LU 分解を用いて解を求める。

消去法や LU 分解のように、所定の消去・代入で解を求める方法を**直接法（direct method）**と呼ぶ。ただし、計算機では丸め誤差があるため、「直接法」は「誤差がない方法」という意味ではない。

D.3 疎行列の消去と代入

付録Bで定義した「疎行列」は、0 の要素が多いという**行列の構造**を表し、解法の名前ではない。疎行列にも前節の LU 分解を使える。違いは、0 の要素まで一律に保存・計算するのではなく、**必要な非零要素を扱いながら消去と代入を進める**ことである。まず前節と同じ三重対角系で、その計算を具体的に見る。

三重対角行列と Thomas 法

式(D.1)の行列では、第 1 列で消す必要があるのは第 2 行の -1 だけである。第 3 行の第 1 成分は初めから 0 なので、第 1 行を使って第 3 行を更新する必要はない。第 2 列でも、更新するのは第 3 行だけである。

この行列を、対角が 2、隣接する副対角が -1 の m 次三重対角行列へ拡張する。未知数は $u_1, \dots, u_m$ で、両端の式には範囲外の未知数を含めない。消去後の第 $i-1$ 行が

$$d_{i-1} u_{i-1} - u_i = y_{i-1}$$

になったとする。次の行 $-u_{i-1} + 2u_i - u_{i+1} = b_i$ に、その $1/d_{i-1}$ 倍を加えると、

$$\left(2 - \frac{1}{d_{i-1}}\right) u_i - u_{i+1} = b_i + \frac{y_{i-1}}{d_{i-1}} \tag{D.9}$$

となり、u_{i-1} が消える。つまり、更新が必要なのは対角成分と右辺だけである。

$$d_1 = 2, \quad y_1 = b_1, \qquad d_i = 2 - \frac{1}{d_{i-1}}, \quad y_i = b_i + \frac{y_{i-1}}{d_{i-1}} \quad (i = 2, \dots, m). \tag{D.10}$$

最後の行には u_{m+1} がないので $d_m u_m = y_m$ である。下の行から

$$u_m = \frac{y_m}{d_m}, \qquad u_i = \frac{y_i + u_{i+1}}{d_i} \quad (i = m-1, \dots, 1) \tag{D.11}$$

と代入すれば解が得られる。

前節の $\boldsymbol{b} = (1, 0, 1)^{\mathsf{T}}$ では、計算する数は次のとおりである。

表 D.1　3 未知数の例で、前進消去により更新される対角成分と右辺。

行 i	消去後の対角 d_i	消去後の右辺 y_i
1	2	1
2	$2 - 1/2 = 3/2$	$0 + 1/2 = 1/2$
3	$2 - 1/(3/2) = 4/3$	$1 + (1/2)/(3/2) = 4/3$

これは式(D.3)と同じ結果である。式(D.11)から、再び $u_3 = u_2 = u_1 = 1$ を得る。新しい解法へ置き換えたのではなく、**同じ消去法から 0 に対する不要な操作を省いている**。

三重対角行列の非零要素は $3m - 2$ 個である。この例では各行を 1 回ずつ消去・代入するため、演算回数と記憶量は m に比例する。このように三重対角系の構造を利用して前進消去と後退代入を行う手順は、**Thomas 法（Thomas algorithm）**と呼ばれる。

この例では、式(D.10)から

$$d_i = \frac{i+1}{i} > 0 \qquad (i = 1, \ldots, m) \tag{D.12}$$

となる。実際、$d_{i-1} = i/(i-1)$ を漸化式に代入すれば $d_i = (i+1)/i$ を得る。したがって、この係数行列では 0 で割ることはない。一般の三重対角行列では、係数に応じて漸化式が変わり、消去中に割る成分が 0 または非常に小さくなる場合もある。三重対角という構造だけで、行交換なしの消去が常に使えるわけではない。

一般の疎行列とフィルイン

三重対角の例では、消去後も隣接した成分だけを扱えた。一方、付録Bのような 2 次元の問題では、消去によって最初は 0 だった位置にも非零の値が生じる。ここでは、両方向の格子幅を h とする直交格子で、2 行 2 列の内部点を行ごとに番号付けした 4 未知数の例を使う。2 次元 Poisson 方程式の 5 点差分式に $-h^2$ を掛け、既知の境界値を右辺へ移すと、係数行列は次の形になる。

$$B = \begin{pmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{pmatrix} \tag{D.13}$$

付録Bの軸対称系とは係数が異なるが、隣接点が結び付く構造は共通である。

第 1 列を消すために $R_2 \leftarrow R_2 + \frac{1}{4} R_1$ とすると、第 2 行は

$$(-1, 4, 0, -1) + \frac{1}{4}(4, -1, -1, 0) = \left(0, \frac{15}{4}, -\frac{1}{4}, -1\right) \tag{D.14}$$

へ変わる。**第 2 行・第 3 列の 0 が $-1/4$ に変わった**。このように消去で新たに生じる非零要素を**フィルイン（fill-in）**と呼ぶ。疎行列の直接法でも、この成分を無視してよいわけではなく、保存して以後の計算に使う。

図D.1は、二つの例について前進消去の前後の非零要素の位置を比較したものである。三重対角行列では新たな非零位置は生じないが、行列 B では上三角行列 U_B の第 2 行・第 3 列にフィルインが残る。フィルインとは、元は 0 だった位置に値が生じることを指す。下三角部分を消しているため、図の前後で非零要素の総数が増えるとは限らない。

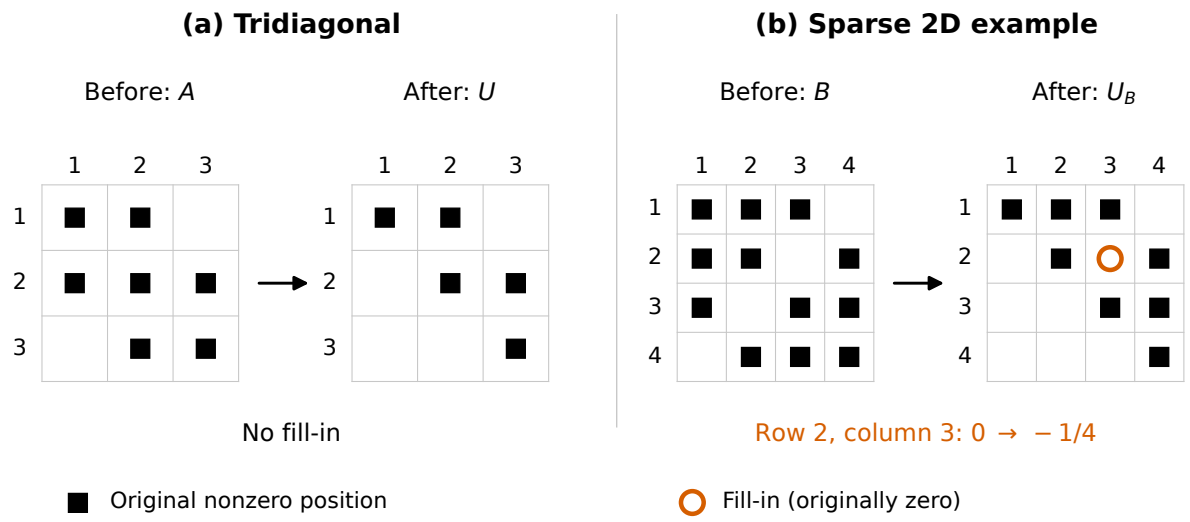


図 D.1　疎行列の前進消去と新たに生じる非零要素（フィルイン）。行交換を行わず、(a) 式(D.1)の A と消去後の U、(b) 式(D.13)の B と消去後の U_B を比較している。黒い四角は元の行列でも非零だった位置、朱色の白抜き丸は元は 0 だった位置に生じた非零要素を表し、空白は 0 である。(b) の第 2 行・第 3 列の値は、式(D.14)で求めた $-1/4$ である。記号は非零の位置だけを表し、係数の大きさや符号は表していない。

図D.1に示したのは U の非零位置だけであり、実際の LU 分解では L も保存する。必要な記憶量は、元の行列の非零要素数だけでは決まらない。疎行列の LU 分解では、非零要素の位置を管理しながら消去を行い、得られた三角行列に前進代入と後退代入を行う。非零要素の増加を抑えるために未知数の順を並べ替え、数値的な安定性のために行を入れ替えることもある。**基本はあくまで LU 分解と代入であり、疎な構造に合わせて計算と保存の方法を工夫している。**

第 2 章の `solve` と付録 B の `spsolve` の違い

第 2 章の `numpy.linalg.solve` は、三重対角行列を入力しても密行列として扱う。第 2 章では差分式と行列の対応を見やすくするためにこの方法を使ったが、(D.10)–(D.11)の手順を自動的に使っているわけではない。付録Bの `spsolve` は、ここで述べた疎行列の直接法を使う。幸谷第 8 章でも、`spsolve` による求解と、疎な LU 分解を保存して再利用する方法が説明されている [5]。`spsolve` は、次節で扱う反復法ではない。また、疎行列を入力しただけで演算回数が未知数の数に比例するとは限らず、フィルインや並べ替えの影響も受ける。

D.4 反復法：Jacobi 法

直接法では行列を変形したが、**反復法（iterative method）**では近似解を繰り返し更新する。ここでは幸谷第 8 章の **Jacobi 法（Jacobi method）**を、同じ 3 未知数の例へ適用する [5]。

反復式への書き換え

式(D.1)の各行を、それぞれ u_1, u_2, u_3 について解くと、

$$u_1 = \frac{1 + u_2}{2}, \qquad u_2 = \frac{u_1 + u_3}{2}, \qquad u_3 = \frac{1 + u_2}{2} \tag{D.15}$$

である。このままでは右辺にも未知数がある。そこで右辺には**現在の近似値**を入れ、左辺を**次の近似値**として計算する。k 回目の近似解を $\boldsymbol{u}^{(k)}$ とすると、

$$\boxed{\begin{aligned} u_1^{(k+1)} &= \frac{1 + u_2^{(k)}}{2}, \\ u_2^{(k+1)} &= \frac{u_1^{(k)} + u_3^{(k)}}{2}, \\ u_3^{(k+1)} &= \frac{1 + u_2^{(k)}}{2} \end{aligned}} \tag{D.16}$$

となる。上付きの (k) は反復回数であり、物理的な時刻ではない。Jacobi 法では、**右辺のすべての成分に、更新前の同じ k 回目の値を使う**。Listing D.2 では、更新前の配列 `u_old` と更新後の配列 `u_new` を分けて、最初の 1 回の更新を示す。NumPy は Listing D.1 と同様に `np` という名前で読み込んでおく。

Listing D.2　Jacobi 法の 1 回の更新

```
u_old = np.zeros(3)

# すべての成分を、更新前の同じ配列から計算する
u_new = np.empty_like(u_old)
u_new[0] = (1.0 + u_old[1]) / 2.0
u_new[1] = (u_old[0] + u_old[2]) / 2.0
u_new[2] = (1.0 + u_old[1]) / 2.0
u_old = u_new
```

1 行目の初期化は最初に 1 回だけ行う。以後は 4–8 行目を繰り返す。各反復で新しい配列を作り、三つの値をすべて計算してから次の反復へ渡す。計算途中で `u_old` の成分を書き換えて右辺に使うと、式(D.16)とは別の更新になる。

初期値からの反復計算

初期値を $\boldsymbol{u}^{(0)} = (0, 0, 0)^{\mathsf{T}}$ とする。1 回目は

$$\boldsymbol{u}^{(1)} = \left(\frac{1+0}{2}, \frac{0+0}{2}, \frac{1+0}{2} \right)^{\mathsf{T}} = \left(\frac{1}{2}, 0, \frac{1}{2} \right)^{\mathsf{T}},$$

2 回目は、この三つの値を使って

$$\boldsymbol{u}^{(2)} = \left(\frac{1+0}{2}, \frac{1/2+1/2}{2}, \frac{1+0}{2} \right)^{\mathsf{T}} = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right)^{\mathsf{T}}$$

となる。1 回目には端の二つだけが変わり、2 回目にはその影響が中央へ伝わっている。

以後の計算結果を表D.2に示す。残差は第 2 章と同じ $\boldsymbol{r}_{\mathrm{res}}^{(k)} = A\boldsymbol{u}^{(k)} - \boldsymbol{b}$ とし、その最大絶対成分も併記する。

表 D.2　Jacobi 法の近似解と最大残差。残差は、表示用に丸める前の近似解から計算した。

反復回数 k	$u_1^{(k)}$	$u_2^{(k)}$	$u_3^{(k)}$	$\max_i \lvert r_{\mathrm{res},i}^{(k)} \rvert$
0	0	0	0	1
1	0.5	0	0.5	1
2	0.5	0.5	0.5	0.5
3	0.75	0.5	0.75	0.5
4	0.75	0.75	0.75	0.25
10	0.968750	0.968750	0.968750	0.031250
20	0.999023	0.999023	0.999023	0.000977

直接法で得た $\boldsymbol{u} = (1,1,1)^{\mathsf{T}}$ に近づいている。この初期値では、偶数回の更新後に三つの成分が等しくなる。その共通の値を c とすると、式(D.16)を 2 回使った後の値は $(1+c)/2$ なので、厳密解 1 からのずれが半分になる。したがって、

$$\boldsymbol{u}^{(2\ell)} = (1 - 2^{-\ell})(1,1,1)^{\mathsf{T}} \qquad (\ell = 0, 1, \ldots) \tag{D.17}$$

であり、途中の奇数回の値も同じ厳密解へ近づく。これはこの行列と初期値に対する結果であり、一般の行列についての収束保証ではない。

収束判定と疎行列での計算

実際の計算では、元の行列と右辺を用いて残差を調べる。表D.2の例では、$k = 20$ で最大残差は $2^{-10} \simeq 9.77 \times 10^{-4}$ である。例えば最大残差が 10^{-3} 未満になったら止めると決めれば、20 回目に初めてこの条件を満たす。丸めて表示した値ではなく、計算中の配列から判定する。

ただし、残差の絶対値は方程式の規格化に依存する。A と $\boldsymbol{b}$ を同じ非零の定数倍にしても解は変わらないが、残差はその定数倍になる。許容値は、右辺の大きさや問題の単位に合わせて設定する必要がある。また、最大反復数も決め、条件を満たさずにその回数へ達した場合は、収束したとは扱わない。

残差は解の誤差そのものでもない。連立一次方程式の厳密解を $\boldsymbol{u}_*$、現在の誤差を $\boldsymbol{e}^{(k)} = \boldsymbol{u}^{(k)} - \boldsymbol{u}_*$ とすると、

$$\boldsymbol{r}_{\mathrm{res}}^{(k)} = A\boldsymbol{u}^{(k)} - \boldsymbol{b} = A(\boldsymbol{u}^{(k)} - \boldsymbol{u}_*) = A\boldsymbol{e}^{(k)} \tag{D.18}$$

である。このため、残差と解の誤差は一般には同じ大きさではなく、その関係は行列にも依存する [5]。本例のように厳密解が分かる場合は、Listing D.1 のように両方を確認できる。

一般の行列でも、各行を同じように書き直せば

$$u_i^{(k+1)} = \frac{b_i - \displaystyle\sum_{j \neq i} a_{ij} u_j^{(k)}}{a_{ii}} \qquad (a_{ii} \neq 0) \tag{D.19}$$

となる。$a_{ij} = 0$ の項は和に寄与しないので、疎行列では非零要素だけを使える。三重対角の例なら各点の更新に使うのは隣の二つの近似値だけで、L, U を作る必要はない。ただし、必要な反復回数は行列や許容値に依存し、大きな問題では収束が遅くなることもある。**疎行列という構造は、直接法でも反復法でも利用できる。**

数値計算上の注意｜反復の収束と格子収束は別の確認

反復回数を増やすときは、格子を固定した同じ連立一次方程式を解いている。反復が収束しても、その方程式を作った有限差分の離散化誤差は残る。第 2 章や付録Bの格子収束試験では、格子を細かくして別の連立一次方程式を作り、元の Poisson 方程式の解への近づき方を調べる。反復法を使う場合は、各格子で許容値をさらに小さくしても結果がほとんど変わらないことを確認し、求解が不十分なことによる誤差を格子収束に混ぜない。

Jacobi 法は反復法の最も基本的な例の一つである。ほかにも、Gauss–Seidel 法や、対称正定値行列に対する共役勾配法（conjugate gradient method）などがある。また、付録Bの補足で紹介した多重格子法は、異なる格子幅の問題を組み合わせて誤差を効率よく減らす方法である。本書では、これらの詳細には立ち入らない。

参考図書

[1] Barbara Ryden, “*Celestial and Stellar Dynamics*”, Cambridge University Press, 2025.
学部生向けに天体力学と恒星系力学を扱う入門的な教科書。本教材では、物理的な題材の主な出典として、§6.4–6.5 の球対称・軸対称系と、§7.1–7.3 の球対称・軸対称・非軸対称ポテンシャル中の軌道を参照する。

[2] James Binney & Scott Tremaine, “*Galactic Dynamics*”, 2nd ed., Princeton University Press, 2008.
銀河力学・恒星系力学を体系的に扱う標準的な教科書。第 2 章の James 法の補足では、§2.9 の Box 2.5 を参照する。付録 B の補足では、§2.4 の多重極展開、§2.8 の基底関数展開、§2.9 の N 体計算における重力計算法を参照する。

[3] R. A. James, “The solution of Poisson’s equation for isolated source distributions”, *Journal of Computational Physics*, 25(2), 71–93, 1977.
有限の格子で孤立系の Poisson 方程式を解くため、境界上の遮蔽質量と離散 Green 関数を利用した境界補正法を提案した論文。

[4] 福江純・和田桂一・梅村雅之『宇宙流体力学の基礎［改訂版］』、日本評論社、2022 年。
宇宙流体力学の物理的な基礎を体系的に扱う教科書。宇宙流体の基礎方程式、自己重力流体、波動、衝撃波、流体不安定性、重力不安定性などを学ぶことができる。本教材のあとに宇宙流体シミュレーションへ進む際に、数値計算で解く方程式とその物理的意味を学ぶための参考書とする。

[5] 幸谷智紀『Python 数値計算プログラミング』講談社、2021 年。
Python による数値計算法を、理論と実装の両面から扱う教科書。本教材では、数値積分、数値微分、有限差分法、常微分方程式の数値解法を主に参照する。付録 D では、第 7–8 章の連立一次方程式、疎行列、反復法を参照する。より入門的な説明には、杉浦『数値計算の基礎と応用 — 数値解析学への入門 —』や、三井田・須田『数値計算法』も参考になる。

[6] 杉浦洋『数値計算の基礎と応用 — 数値解析学への入門 —』新訂版、サイエンス社、2009 年。
数値計算法の基礎を簡潔にまとめた入門書。第 1 章では、数値積分則の次数、Taylor 展開による誤差評価、および局所的な近似誤差と大域的な誤差を区別する考え方の参考とする。

[7] 三井田惇郎・須田宇宙『数値計算法』第 3 版、森北出版、2023 年。
数値計算法の基礎を具体的な計算例とともに学べる入門書。第 1 章の補足では、モンテカルロ法と乱数生成、とくに逆変換法によるサンプリングの参考とする。

[8] 富阪幸治・花輪知幸・牧野淳一郎編『シミュレーション天文学』第 2 版、シリーズ現代の天文学第 14 巻、日本評論社、2026 年。
常微分方程式・偏微分方程式の数値解法から、流体・磁気流体、自己重力流体、重力多体系まで、天文シミュレーションの基礎と応用を広く扱う教科書。第 2 章では §5.3 の自己重力流体計算と多重極展開による境界値を参照する。第 1・3 章の重力多体系に関する補足では第 6 章、付録 B の補足では重力計算法を参考とする。

[9] 松本倫明・大須賀健・須佐元『輻射電磁流体シミュレーションの基礎』シリーズ〈宇宙物理学の基礎〉第 5 巻、日本評論社、2024 年。
宇宙流体シミュレーションの数値計算法を、格子法・粒子法から磁気流体・輻射流体まで広く扱う教科書。第 1 章では §2.5 の有限体積法とセル平均、第 2 章では §2.4–2.6 の安

定性・境界条件を参照する。第 3 章では §9.6 のリープフロッグ法と長時間挙動を参照し、ソフトニング（§9.5）と流体の時間積分に関する補足でも参考とする。付録 B では、重力計算法に関する説明を参考とする。

[10] 藤井孝藏・立川智章『Python で学ぶ流体力学の数値計算法』オーム社、2020 年。
圧縮性流体の数値計算法を、Python による実装とともに学べる教科書。第 2 章末の偏微分方程式の型・境界条件・陽的および陰的時間積分の補足、第 3 章と Notebook 3 の RK2・RK3 の補足で参考とする。

[11] Sverre J. Aarseth, "*Gravitational N-Body Simulations: Tools and Algorithms*", Cambridge University Press, 2003.
重力 N 体シミュレーションの数値計算法を詳しく扱う教科書。直接重力計算、時間積分、近接遭遇、正則化、個別時間刻み、精度と計算効率などを学ぶことができる。特に、星団のような衝突系の重力多体系計算を詳しく学ぶ際の参考になる。

[12] Walter Dehnen & Justin I. Read, "*N-body simulations of gravitational dynamics*", *European Physical Journal Plus*, 126, 55, 2011.
重力 N 体シミュレーションの数値計算法を概説したレビュー。衝突系と無衝突系の違いを踏まえ、それぞれで用いられる代表的な数値手法を整理している。重力多体系計算の全体像を把握するための参考になる。